

User Manual

for S32K1_S32M24X PWM Driver

Document Number: UM2PWMASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	13
3.5 Driver Limitations	19
3.6 Driver usage and configuration tips	20
3.7 Runtime errors	22
3.8 Symbolic Names Disclaimer	22
3.9 Usage in non AUTOSAR context	23
4 Configuration Parameters	25
4.1 Pwm	26
4.1.1 Module Pwm	30
4.1.2 Container PwmChannelConfigSet	30
4.1.3 Container PwmChannel	31
4.1.4 Parameter PwmChannelId	31
4.1.5 Parameter PwmChannelClass	31
4.1.6 Parameter PwmPeriodInTicks	32
4.1.7 Parameter PwmPeriodDefault	32
4.1.8 Parameter PwmDutycycleDefault	33
4.1.9 Parameter PwmPolarity	34
4.1.10 Parameter PwmIdleState	34
4.1.11 Parameter PwmNotification	34
4.1.12 Reference PwmChannelEcucPartitionRef	35
4.1.13 Reference PwmHwChannel	35
4.1.14 Reference PwmMcuClockReferencePoint	36
4.1.15 Container PwmFtm	36
4.1.16 Parameter PwmHwInstance	37
4.1.17 Container PwmFtmClkCfg	37
4.1.18 Parameter PwmFtmClockSource	37
4.1.19 Parameter PwmFtmClockPrescaler	38
4.1.20 Parameter PwmFtmClockPrescalerAlternate	38
4.1.21 Container PwmFtmGlobalChCfg	39
4.1.22 Parameter PwmFtmCounterMode	39
4.1.23 Parameter PwmFtmPeriod	40
4.1.24 Parameter PwmFtmPeriodDither	40
4.1.25 Parameter PwmFtmDeadTime	41
4.1.26 Parameter PwmFtmDeadTimePrescaler	41
4.1.27 Container PwmFtmInstCfg	42
4.1.28 Parameter PwmFtmDebugMode	42
4.1.29 Parameter PwmFtmWriteProtection	43
4.1.30 Parameter PwmFtmInitTrigger	43
4.1.31 Parameter PwmFtmInitTriggerMode	44
4.1.32 Parameter PwmFtmOverflowIrq	44

4.1.33	Parameter	PwmFtmReloadIrq	45
4.1.34	Container	PwmFtmOverflowIrqCallback	45
4.1.35	Parameter	PwmFtmOverflowIrqFunctionCallback	45
4.1.36	Parameter	PwmFtmOverflowIrqParameterCallback	46
4.1.37	Container	PwmFtmReloadIrqCallback	46
4.1.38	Parameter	PwmFtmReloadIrqFunctionCallback	46
4.1.39	Parameter	PwmFtmReloadIrqParameterCallback	47
4.1.40	Container	PwmFtmSyncCfg	47
4.1.41	Parameter	PwmFtmSyncMode	48
4.1.42	Parameter	PwmFtmHwTrig0	48
4.1.43	Parameter	PwmFtmHwTrig1	48
4.1.44	Parameter	PwmFtmHwTrig2	49
4.1.45	Parameter	PwmFtmAutoHwTrig	49
4.1.46	Parameter	PwmFtmMaxLoadPoint	49
4.1.47	Parameter	PwmFtmMinLoadPoint	50
4.1.48	Parameter	PwmFtmHalfCycleLoadPoint	50
4.1.49	Parameter	PwmFtmHalfCycleLoadPointValue	51
4.1.50	Parameter	PwmFtmReloadPointFrequency	51
4.1.51	Parameter	PwmFtmInvertControl	52
4.1.52	Parameter	PwmFtmSwOutControl	52
4.1.53	Parameter	PwmFtmOutputMask	53
4.1.54	Parameter	PwmFtmCounterInitial	53
4.1.55	Parameter	PwmFtmCounterSync	54
4.1.56	Container	FtmGlobalFaultCfg	54
4.1.57	Parameter	FtmFaultFunctionality	54
4.1.58	Parameter	FtmFaultFilterValue	55
4.1.59	Parameter	FtmFaultOutputState	55
4.1.60	Parameter	FtmFaultIrqUsed	56
4.1.61	Container	FtmFaultSettings	56
4.1.62	Parameter	FtmFaultId	57
4.1.63	Parameter	FtmFaultPolarity	57
4.1.64	Parameter	FtmEnableFaultFilter	57
4.1.65	Parameter	FtmFaultNotification	58
4.1.66	Container	PwmFtmSimCfg	58
4.1.67	Parameter	PwmFtmObeControl	59
4.1.68	Container	PwmFtmCh	59
4.1.69	Parameter	PwmFtmChId	59
4.1.70	Parameter	PwmFtmChPulseType	60
4.1.71	Parameter	PwmFtmChInitOutput	60
4.1.72	Parameter	PwmFtmChPolarity	61
4.1.73	Parameter	PwmFtmChDutyCycle	61
4.1.74	Parameter	PwmFtmChDutyCycleDither	62
4.1.75	Parameter	PwmFtmChInterrupt	62
4.1.76	Parameter	PwmFtmChPwmOutputEnable	62
4.1.77	Parameter	PwmFtmChSwCtrlEnable	63
4.1.78	Parameter	PwmFtmChExternTriggerEnable	63
4.1.79	Parameter	PwmFtmChSwCtrlValue	64
4.1.80	Parameter	PwmFtmChMatchLoadPointEnable	64
4.1.81	Parameter	PwmFtmPairChEnable	65
4.1.82	Container	PwmFtmChInterruptCallback	65
4.1.83	Parameter	PwmFtmChInterruptFunctionCallback	65
4.1.84	Parameter	PwmFtmChInterruptParameterCallback	66
4.1.85	Container	PwmFtmPairedCh	66
4.1.86	Parameter	PwmFtmPairedChDeadtimeEnable	67

4.1.87 Parameter PwmFtmPairedChSync	67
4.1.88 Parameter PwmFtmPairedChExtTrig	67
4.1.89 Parameter PwmFtmPairedChComplementary	68
4.1.90 Parameter PwmFtmDutyDitheringPaired	68
4.1.91 Parameter PwmFtmPairedChCombineMode	69
4.1.92 Parameter PwmFtmPairedChComplementaryMode	69
4.1.93 Parameter PwmFtmPairedChPhaseShift	69
4.1.94 Container PwmFlexio	70
4.1.95 Parameter PwmHwInstance	70
4.1.96 Container PwmFlexioChannels	71
4.1.97 Parameter FlexioChId	71
4.1.98 Parameter FlexioPinId	71
4.1.99 Parameter FlexioChPrescaler	72
4.1.100 Parameter FlexioChPrescalerAlternate	72
4.1.101 Parameter FlexioChDutyCycle	73
4.1.102 Parameter FlexioChPeriod	73
4.1.103 Parameter FlexioChPolarity	73
4.1.104 Reference FlexioMclChRef	74
4.1.105 Container FlexioChIrqCallback	74
4.1.106 Parameter FlexioChIrqFunctionCallback	74
4.1.107 Parameter FlexioChIrqParameterCallback	75
4.1.108 Container PwmHwResourceConfig	75
4.1.109 Parameter PwmHwResourceId	76
4.1.110 Parameter PwmIsrEnable	76
4.1.111 Container PwmGeneral	77
4.1.112 Parameter PwmMultipartitionEnabled	77
4.1.113 Parameter PwmDevErrorDetect	78
4.1.114 Parameter PwmDutycycleUpdatedEndperiod	78
4.1.115 Parameter PwmPeriodUpdatedEndperiod	78
4.1.116 Parameter PwmNotificationSupported	79
4.1.117 Parameter PwmEnableUserModeSupport	79
4.1.118 Parameter PwmLowPowerStatesSupport	80
4.1.119 Parameter PwmPowerStateAsynchTransitionMode	80
4.1.120 Parameter PwmEnableDualClockMode	80
4.1.121 Parameter PwmEnableExternalTrigger	81
4.1.122 Parameter PwmEnableMaskOutputs	81
4.1.123 Parameter PwmFaultSupported	82
4.1.124 Parameter PwmReloadNotificationSupported	82
4.1.125 Parameter PwmMultiChannelSync	82
4.1.126 Parameter PwmIndex	83
4.1.127 Reference PwmEcucPartitionRef	83
4.1.128 Reference PwmKernelEcucPartitionRef	84
4.1.129 Container PwmPowerStateConfig	84
4.1.130 Parameter PwmPowerState	85
4.1.131 Parameter PwmPowerStateReadyCbRef	85
4.1.132 Container PwmConfigurationOfOptApiServices	85
4.1.133 Parameter PwmDeInitApi	86
4.1.134 Parameter PwmGetOutputState	86
4.1.135 Parameter PwmSetDutyCycle	87
4.1.136 Parameter PwmSetOutputToIdle	87
4.1.137 Parameter PwmSetPeriodAndDuty	87
4.1.138 Parameter PwmVersionInfoApi	88
4.1.139 Parameter PwmGetChannelStateApi	88
4.1.140 Parameter PwmSetDutyCycle_NoUpdate	89

4.1.141 Parameter PwmSetPeriodAndDuty_NoUpdate	89
4.1.142 Parameter PwmSetDutyPhaseShift	89
4.1.143 Parameter PwmSetChannelDeadTime	90
4.1.144 Parameter PwmForceOutputToZeroApi	90
4.1.145 Parameter PwmEnableTrigger	91
4.1.146 Parameter PwmDisableTrigger	91
4.1.147 Parameter PwmResetCounter	92
4.1.148 Parameter PwmFastUpdateApi	92
4.1.149 Container CommonPublishedInformation	92
4.1.150 Parameter ArReleaseMajorVersion	93
4.1.151 Parameter ArReleaseMinorVersion	93
4.1.152 Parameter ArReleaseRevisionVersion	94
4.1.153 Parameter ModuleId	94
4.1.154 Parameter SwMajorVersion	94
4.1.155 Parameter SwMinorVersion	95
4.1.156 Parameter SwPatchVersion	95
4.1.157 Parameter VendorApiInfix	96
4.1.158 Parameter VendorId	96
4.2 FTM_PWM	97
4.2.1 Module Ftm_Pwm	97
4.3 FLEXIO_PWM	97
4.3.1 Module Flexio_Pwm	98
5 Module Index	99
5.1 Software Specification	99
6 Module Documentation	100
6.1 FlexIO Pwm IPL	100
6.1.1 Detailed Description	100
6.1.2 Data Structure Documentation	101
6.1.2.1 struct Flexio_Pwm_Ip_IplNotificationType	101
6.1.2.2 struct Flexio_Pwm_Ip_HldNotificationType	101
6.1.2.3 struct Flexio_Pwm_Ip_ChannelConfigType	102
6.1.3 Types Reference	102
6.1.3.1 Flexio_Pwm_Ip_IplCallbackType	102
6.1.3.2 Flexio_Pwm_Ip_HldCallbackType	102
6.1.4 Enum Reference	102
6.1.4.1 Flexio_Pwm_Ip_StatusType	102
6.1.4.2 Flexio_Pwm_Ip_InterruptType	103
6.1.4.3 Flexio_Pwm_Ip_OccurredIRQType	103
6.1.5 Function Reference	103
6.1.5.1 Flexio_Pwm_Ip_InitChannel()	104
6.1.5.2 Flexio_Pwm_Ip_DeInitChannel()	104
6.1.5.3 Flexio_Pwm_Ip_UpdatePeriodDuty()	104
6.1.5.4 Flexio_Pwm_Ip_GetOutputState()	105
6.1.5.5 Flexio_Pwm_Ip_UpdateInterruptMode()	105
6.1.5.6 Flexio_Pwm_Ip_GetPeriod()	106
6.1.6 Variable Documentation	106
6.1.6.1 Flexio_Pwm_Ip_aState	106
6.1.6.2 Flexio_Pwm_Ip_u8TimerIrqMask	106
6.2 Ftm Pwm IPL	107
6.2.1 Detailed Description	107
6.2.2 Data Structure Documentation	110
6.2.2.1 struct Ftm_Pwm_Ip_CallbackCfgType	110

6.2.2.2 struct Ftm_Pwm_Ip_SyncCfgType	110
6.2.2.3 struct Ftm_Pwm_Ip_InstanceCfgType	111
6.2.2.4 struct Ftm_Pwm_Ip_PairCfgType	112
6.2.2.5 struct Ftm_Pwm_Ip_ChannelConfigType	112
6.2.2.6 struct Ftm_Pwm_Ip_FaultChCfgType	113
6.2.2.7 struct Ftm_Pwm_Ip_UserCfgType	113
6.2.3 Macro Definition Documentation	114
6.2.3.1 FTM_PWM_IP_MAX_DUTY_CYCLE	114
6.2.3.2 FTM_PWM_IP_NO_EDGE	114
6.2.4 Types Reference	114
6.2.4.1 Ftm_Pwm_Ip_CallbackType	114
6.2.4.2 Ftm_Pwm_Ip_FaultCallbackType	114
6.2.5 Enum Reference	114
6.2.5.1 Ftm_Pwm_Ip_StatusType	114
6.2.5.2 Ftm_Pwm_Ip_ChannelModeType	115
6.2.5.3 Ftm_Pwm_Ip_ChannelStateType	115
6.2.5.4 Ftm_Pwm_Ip_PrescalerType	116
6.2.5.5 Ftm_Pwm_Ip_PowerStateType	116
6.2.5.6 Ftm_Pwm_Ip_NotifType	116
6.2.5.7 Ftm_Pwm_Ip_CounterModeType	117
6.2.5.8 Ftm_Pwm_Ip_ClkSourceType	117
6.2.5.9 Ftm_Pwm_Ip_PolarityType	118
6.2.5.10 Ftm_Pwm_Ip_OutputStateType	118
6.2.5.11 Ftm_Pwm_Ip_ClkPrescalerType	118
6.2.5.12 Ftm_Pwm_Ip_SyncType	119
6.2.5.13 Ftm_Pwm_Ip_SyncModeType	119
6.2.5.14 Ftm_Pwm_Ip_DeadtimePrescalerType	119
6.2.5.15 Ftm_Pwm_Ip_DebugModeType	120
6.2.5.16 Ftm_Pwm_Ip_InitTriggModeType	120
6.2.5.17 Ftm_Pwm_Ip_PairChPolarityType	120
6.2.5.18 Ftm_Pwm_Ip_PairDitherEdgeType	121
6.2.5.19 Ftm_Pwm_Ip_FaultControlModeType	121
6.2.5.20 Ftm_Pwm_Ip_FaultOutputStateType	121
6.2.6 Function Reference	122
6.2.6.1 Ftm_Pwm_Ip_Init()	122
6.2.6.2 Ftm_Pwm_Ip_DeInit()	122
6.2.6.3 Ftm_Pwm_Ip_UpdatePwmChannel()	122
6.2.6.4 Ftm_Pwm_Ip_FastUpdatePwmDuty()	123
6.2.6.5 Ftm_Pwm_Ip_UpdatePwmPeriod()	124
6.2.6.6 Ftm_Pwm_Ip_SwOutputControl()	124
6.2.6.7 Ftm_Pwm_Ip_SyncUpdate()	125
6.2.6.8 Ftm_Pwm_Ip_DisableNotification()	125
6.2.6.9 Ftm_Pwm_Ip_EnableNotification()	125
6.2.6.10 Ftm_Pwm_Ip_SetClockMode()	126
6.2.6.11 Ftm_Pwm_Ip_ResetCounter()	126
6.2.6.12 Ftm_Pwm_Ip_MaskOutputChannels()	126
6.2.6.13 Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel()	127
6.2.6.14 Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty()	127
6.2.6.15 Ftm_Pwm_Ip_GetOutputState()	128
6.2.6.16 Ftm_Pwm_Ip_SetPowerState()	128
6.2.6.17 Ftm_Pwm_Ip_GetChannelState()	128
6.2.6.18 Ftm_Pwm_Ip_SetPhaseShift()	129
6.2.6.19 Ftm_Pwm_Ip_SetDutyPhaseShift()	129
6.2.6.20 Ftm_Pwm_Ip_UnMaskOutputChannels()	130

6.2.6.21 Ftm_Pwm_Ip_EnableTrigger()	130
6.2.6.22 Ftm_Pwm_Ip_DisableTrigger()	131
6.2.6.23 Ftm_Pwm_Ip_SetChannelDeadTime()	131
6.2.7 Variable Documentation	131
6.2.7.1 Ftm_Pwm_Ip_pFaultIrqCallback	131
6.2.7.2 Ftm_Pwm_Ip_pReloadIrqCallback	132
6.2.7.3 Ftm_Pwm_Ip_aInstanceState	132
6.2.7.4 Ftm_Pwm_Ip_aPreviousChannelState	132
6.2.7.5 Ftm_Pwm_Ip_pOverflowIrqCallback	132
6.2.7.6 Ftm_Pwm_Ip_aChannelState	132
6.2.7.7 Ftm_Pwm_Ip_aNotifIrq	132
6.2.7.8 Ftm_Pwm_Ip_aPeriod	132
6.3 Pwm Driver	133
6.3.1 Detailed Description	133
6.3.2 Data Structure Documentation	138
6.3.2.1 struct Pwm_ChannelConfigType	138
6.3.2.2 struct Pwm_ConfigType	139
6.3.3 Macro Definition Documentation	141
6.3.3.1 PWM_DUTY_CYCLE_100_U16	141
6.3.3.2 PWM_E_INIT_FAILED	141
6.3.3.3 PWM_E_UNINIT	141
6.3.3.4 PWM_E_PARAM_CHANNEL	141
6.3.3.5 PWM_E_PERIOD_UNCHANGEABLE	142
6.3.3.6 PWM_E_ALREADY_INITIALIZED	142
6.3.3.7 PWM_E_PARAM_POINTER	142
6.3.3.8 PWM_E_NOT_DISENGAGED	142
6.3.3.9 PWM_E_POWER_STATE_NOT_SUPPORTED	142
6.3.3.10 PWM_E_TRANSITION_NOT_POSSIBLE	143
6.3.3.11 PWM_E_PERIPHERAL_NOT_PREPARED	143
6.3.3.12 PWM_E_PERIODVALUE	143
6.3.3.13 PWM_E_PARAM_NOTIFICATION	143
6.3.3.14 PWM_E_PARAM_NOTIFICATION_NULL	143
6.3.3.15 PWM_E_DUTYCYCLE_RANGE	144
6.3.3.16 PWM_E_CHANNEL_OFFSET_VALUE	144
6.3.3.17 PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE	144
6.3.3.18 PWM_E_PARAM_INSTANCE	144
6.3.3.19 PWM_E_UNEXPECTED_ISR	144
6.3.3.20 PWM_E_PARAM_PHASESHIFT_RANGE	145
6.3.3.21 PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED	145
6.3.3.22 PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED	145
6.3.3.23 PWM_E_TRIGGER_MASK	145
6.3.3.24 PWM_E_FORCE_OUTPUT_NOT_SUPPORTED	145
6.3.3.25 PWM_E_PARAM_CONFIG	146
6.3.3.26 PWM_E_DEADTIME_RANGE	146
6.3.3.27 PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED	146
6.3.3.28 PWM_E_GETOUTPUTSTATE_NOT_SUPPORTED	146
6.3.3.29 PWM_INIT_ID	146
6.3.3.30 PWM_DEINIT_ID	147
6.3.3.31 PWM_SETDUTYCYCLE_ID	147
6.3.3.32 PWM_SETPERIODANDDDUTY_ID	147
6.3.3.33 PWM_SETOUTPUTTOIDLE_ID	147
6.3.3.34 PWM_GETOUTPUTSTATE_ID	147
6.3.3.35 PWM_DISABLENOTIFICATION_ID	148
6.3.3.36 PWM_ENABLENOTIFICATION_ID	148

6.3.3.37 PWM_GETVERSIONINFO_ID	148
6.3.3.38 PWM_SETPOWERSTATE_ID	148
6.3.3.39 PWM_GETCURRENTPOWERSTATE_ID	148
6.3.3.40 PWM_GETTARGETPOWERSTATE_ID	149
6.3.3.41 PWM_PREPAREPOWERSTATE_ID	149
6.3.3.42 PWM_MAIN_POWERTRANSITIONMANAGER_ID	149
6.3.3.43 PWM_GETCHANNELSTATE_ID	149
6.3.3.44 PWM_FORCEOUTPUTTOZERO_ID	149
6.3.3.45 PWM_SETCLOCKMODE_ID	150
6.3.3.46 PWM_SYNCUPDATE_ID	150
6.3.3.47 PWM_SETPERIODANDDUTY_NO_UPDATE_ID	150
6.3.3.48 PWM_SETDUTYCYCLE_NO_UPDATE_ID	150
6.3.3.49 PWM_SETCHANNELDEADTIME_ID	150
6.3.3.50 PWM_MASKOUTPUT_ID	151
6.3.3.51 PWM_UNMASKOUTPUT_ID	151
6.3.3.52 PWM_ENABLETRIGGER_ID	151
6.3.3.53 PWM_DISABLETRIGGER_ID	151
6.3.3.54 PWM_RESETCOUNTERENABLE_ID	151
6.3.3.55 PWM_RESETCOUNTERDISABLE_ID	152
6.3.3.56 PWM_DISABLERELOADNOTIF_ID	152
6.3.3.57 PWM_ENABLERELOADNOTIF_ID	152
6.3.3.58 PWM_FASTUPDATEDUTYCYCLE_ID	152
6.3.3.59 PWM_SETDUTYPHASESHIFT_ID	152
6.3.3.60 PWM_DISABLEFAULTNOTIF_ID	153
6.3.3.61 PWM_ENABLEFAULTNOTIF_ID	153
6.3.4 Types Reference	153
6.3.4.1 Pwm_ChannelType	153
6.3.4.2 Pwm_InstanceType	153
6.3.4.3 Pwm_PeriodType	153
6.3.4.4 Pwm_DutyType	153
6.3.4.5 Pwm_NotifyType	154
6.3.5 Enum Reference	154
6.3.5.1 Pwm_OutputStateType	154
6.3.5.2 Pwm_EdgeNotificationType	154
6.3.5.3 Pwm_ChannelClassType	154
6.3.5.4 Pwm_PowerStateType	155
6.3.5.5 Pwm_PowerStateRequestResultType	155
6.3.5.6 Pwm_PrescalerType	155
6.3.6 Function Reference	156
6.3.6.1 Pwm_Init()	156
6.3.6.2 Pwm_DeInit()	157
6.3.6.3 Pwm_SetDutyCycle()	157
6.3.6.4 Pwm_SetPeriodAndDuty()	158
6.3.6.5 Pwm_SetOutputToIdle()	159
6.3.6.6 Pwm_GetOutputState()	160
6.3.6.7 Pwm_EnableNotification()	160
6.3.6.8 Pwm_DisableNotification()	161
6.3.6.9 Pwm_GetVersionInfo()	162
6.3.6.10 Pwm_GetChannelState()	162
6.3.6.11 Pwm_ForceOutputToZero()	162
6.3.6.12 Pwm_SetClockMode()	163
6.3.6.13 Pwm_SetChannelDeadTime()	163
6.3.6.14 Pwm_SetDutyCycle_NoUpdate()	163
6.3.6.15 Pwm_SetPeriodAndDuty_NoUpdate()	164

6.3.6.16 Pwm_SyncUpdate()	165
6.3.6.17 Pwm_SetDutyPhaseShift()	166
6.3.6.18 Pwm_EnableTrigger()	166
6.3.6.19 Pwm_DisableTrigger()	167
6.3.6.20 Pwm_ResetCounterEnable()	167
6.3.6.21 Pwm_ResetCounterDisable()	167
6.3.6.22 Pwm_FastUpdateDutyCycle()	168
6.3.6.23 Pwm_MaskOutputs()	168
6.3.6.24 Pwm_UnMaskOutputs()	169
6.3.6.25 Pwm_EnableReloadNotification()	169
6.3.6.26 Pwm_DisableReloadNotification()	170
6.3.6.27 Pwm_EnableFaultNotification()	170
6.3.6.28 Pwm_DisableFaultNotification()	171
6.3.6.29 Pwm_SetPowerState()	171
6.3.6.30 Pwm_GetCurrentPowerState()	172
6.3.6.31 Pwm_GetTargetPowerState()	172
6.3.6.32 Pwm_PreparePowerState()	173



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR PWM for S32K1_S32M24X. AUTOSAR PWM driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR PWM driver requirements and APIs are described in the AUTOSAR PWM driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

3.2 Driver Design Summary

The driver provides functions for initialization and control of the micro controller internal PWM stage (pulse width modulation). The PWM module generates pulses with variable pulse width. It allows the selection of the duty cycle and the signal period time. Each PWM channel is linked to a hardware channel capable of implementing PWM functionality, which belongs to the micro controller.

The S32K1 micro controllers contains FlexTimer(FTM) and Flexio.

The FTM module has the following major features in this release:

- FTM source clock is selectable with pre-scaler option.
- 16-bit counter.
- Each channel can be configured for edge-aligned or center-aligned PWM mode.
- Each pair of channels can be combined to generate a PWM signal with independent control of both edges of PWM signal.
- The FTM channels can operate as pairs with equal outputs, pairs with complementary outputs, or independent channels with independent outputs.

- The deadtime insertion is available for each complementary pair.

The Flexio module has the following major features in this release:

- Up to 4 channels chosen among the Flexio timers.
- Up to 8 pins that can be chosen for any channel.
- Timer 8-bit High PWM Mode.

3.3 Hardware Resources

Ftm's resource

Derivatives	Pwm Module	Pwm Channels available	Fault channel
s32k116_qfn32	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 4, Channel 5	N/A
s32k116_lqfp48	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2
s32k118_lqfp48	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2
s32k118_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
s32k142_lqfp48	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3	FLT_1
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_1

s32k142_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT↔_3
s32k142_lqfp100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
s32k142w_lqfp48	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3	FLT_1
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_1
s32k142w_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT↔_3
s32k144_lqfp48	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3	FLT_1

	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_1
s32k144_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
s32k144_lqfp100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
s32k144_mapbga100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
s32k144w_lqfp48	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_2
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3	FLT_1
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_1

s32k144w_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
s32k146_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 4	Channel 5, Channel 6	N/A
	Ftm 5	Channel 0, Channel 1, Channel 3, Channel 5	N/A
s32k146_lqfp100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 4	Channel 2, Channel 5, Channel 6	FLT_0, FLT_1
	Ftm 5	Channel 0, Channel 1, Channel 3, Channel 5	FLT_0, FLT_1
s32k146_mapbga100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT↔_3

	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 4	Channel 2, Channel 5, Channel 6	FLT_0, FLT_1
	Ftm 5	Channel 0, Channel 1, Channel 3, Channel 5	FLT_0, FLT_1
s32k146_lqfp144	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 4	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
	Ftm 5	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
s32k148_lfpq100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 4	Channel 2, Channel 5, Channel 6	FLT_0, FLT_1
	Ftm 5	Channel 0, Channel 1, Channel 3, Channel 5	FLT_0, FLT_1
s32k148_mapbga100	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 4	Channel 2, Channel 5, Channel 6	FLT_0, FLT_1

	Ftm 5	Channel 0, Channel 1, Channel 3, Channel 5	FLT_0, FLT_1
s32k148_lqfp144	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 4	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
	Ftm 5	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
	Ftm 6	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 7	FLT_0, FLT_1
	Ftm 7	Channel 0, Channel 1, Channel 2, Channel 3, Channel 5, Channel 7	FLT_0, FLT_1
s32k148_lqfp176	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 4	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
	Ftm 5	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
	Ftm 6	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
	Ftm 7	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1
s32m244_lqfp64	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 2, Channel 3, Channel 4, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3

s32m242_lqfp64	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 2, Channel 3, Channel 4, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
s32m242_lqfp64	Ftm 2	Channel 0, Channel 1, Channel 2, Channel 3, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 3	Channel 0, Channel 1, Channel 2, Channel 3, Channel 5, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 0	Channel 0, Channel 1, Channel 2, Channel 3, Channel 4, Channel 5	FLT_0, FLT_1, FLT_2, FLT_3
	Ftm 1	Channel 0, Channel 2, Channel 3, Channel 4, Channel 6, Channel 7	FLT_0, FLT_1, FLT_2, FLT_3

Flexio's resource

Derivatives	Module	Flexio timer available	Flexio Channels available
All Derivatives	Flexio 0	Timer_0, Timer_1, Timer_2, Timer_3	Channel_0, Channel_1, Channel_2, Channel_3, Channel_4, Channel_5, Channel_6, Channel_7

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR PWM Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the PWM Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the PWM driver.

Requirement	Status	Description	Notes
SWS_Pwm_00065	N/S	The Pwm SWS shall not define the code file structure.	Code file structure does always not define in the driver.
SWS_Pwm_70075	N/S	Pwm_Irq.c shall include Pwm.h .	Pwm_Irq.c is not needed. Autosar specific interrupt behaviour is implemented using a normal function placed in the Pwm.c file.
SWS_Pwm_00162	N/S	The Pwm Driver shall support synchronuous and asynchronous power state transitions, depending on the value of the configuration parameter PwmPowerStateAsynchTransitionMode.	Rejected: All PWM hardware do not support asynchronous power state transitions

Requirement	Status	Description	Notes
SWS_Pwm_00164	N/S	In case the configuration parameter PwmPowerState↔AsynchTransitionMode is set to TRUE, the preparation process shall continue in background after the relative API returns and its completion shall be notified by means of the configured callback.	Rejected: All PWM hardware do not support asynchronous power state transitions
SWS_Pwm_00175	N/S	The API shall report the development error PWM_E_TR↔ANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state.	Rejected: All PWM hardware do not support asynchronous power state transitions
SWS_Pwm_00195	N/S	The API shall report the development error PWM_E_TR↔ANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state.	Rejected: All PWM hardware do not support asynchronous power state transitions
SWS_Pwm_00188	N/S	The API shall report the development error PWM_E_TR↔ANSITION_NOT_POSSIBLE in case the requested power state cannot be directly reached from the current power state. All asynchronous operation needed to reach the target power state can be executed in background in the context of Pwm_Main↔_PowerTransitionManager.	Rejected: All PWM hardware do not support asynchronous power state transitions
SWS_Pwm_00189	N/S	Service Name: Pwm_Main_↔PowerTransitionManager Syntax: void Pwm_Main_↔PowerTransitionManager (void) Service ID [hex]: 0x0d Description: This API is cyclically called and supervises the power state transitions, checking for the readiness of the module and issuing the callbacks IoHwAb_Pwm_NotifyReady↔ForPowerState<Mode> (see PwmPowerStateReadyCbRef configuration parameter). Available via: SchM_Pwm.h	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
SWS_Pwm_00190	N/S	This API executes any non-immediate action needed to finalize a power state transition requested by Pwm_PreparePowerState() .	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00191	N/S	The rate of scheduling shall be defined by Pwm Main↔SchedulePeriod and shall be variable, as the function only needs to be called if a transition has been requested.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00192	N/S	This API shall also issue call-back notifications to the eventually registered users (IoHwAbs) as configured, only in case the asynch mode is chosen.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00193	N/S	In case the PWM module is not initialized, this function shall simply return without any further elaboration. This is needed to avoid to elaborate uninitialized variables. No development error shall be entered, because this condition can easily be verified during the startup phase (tasks started before the initialization is complete). Rationale: during the startup phase it can happen that the OS already schedules tasks, which call main functions, while some modules are not initialised yet. This is no real error condition, although need handling, i.e. returning without execution. Although the transition state monitoring functionality is mandatory, the implementation of this API is optional, meaning that if the HW allows for other ways to deliver notification and watch the transition state the implementation of this function can be skipped.	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
SWS_Pwm_00199	N/S	In case the PWM Driver is configured to support power state management with asynchronous transitions, this API shall be called to signal completion of the power transition preparation phase to the IoHwAbs module. This is a callback, this API is to be implemented in the IoHwAbs component.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_00153	N/S	These requirements are not applicable to this specification.	not a requirement
ECUC_Pwm_00143	N/S	Name: PwmPowerState↔ AsynchTransitionMode Parent Container: PwmGeneral Description: Enables / disables support of the PWM Driver to the asynchronous power state transition. Multiplicity: 0..1 Type: EcucBooleanParamDef↔ Default value: false Post-Build Variant Multiplicity: false Post-Build Variant Value: false Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local: dependency: This parameter shall only be configured if the parameter PwmLow↔PowerStatesSupport is set to true.	Rejected: All PWM hardwares do not support asynchronous power state transitions
ECUC_Pwm_00144	N/S	Container Name: PwmPower↔StateConfigParent Container: PwmGeneral Description: Each instance of this parameter defines a power state and the callback to be called when this power state is reached.↔ Configuration Parameters	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
ECUC_Pwm_00146	N/S	Name: PwmPowerState Parent Container: PwmPower↔StateConfig Description: Each instance of this parameter describes a different power state supported by the PWM HW. It should be defined by the HW supplier and used by the PWMDriver to reference specific HW configurations which set the PWM HW module in the referenced power state.: At least the power mode corresponding to full power state shall be always configured. Multiplicity: 1 Type: EcucIntegerParamDef (Symbolic Name generated for this parameter) Range: 0 .. 18446744073709551615 Default value: – Post-Build Variant Value: false Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local; dependency: This parameter shall only be configured if the parameter PwmLow↔PowerStatesSupport is set to true.	Rejected: All PWM hardwares do not support asynchronous power state transitions

Requirement	Status	Description	Notes
ECUC_Pwm_00145	N/S	Name: PwmPowerState↔ReadyCbkJRef Parent Container: PwmPower↔StateConfig Description: Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component. Multiplicity: 1 Type: EcucFunctionNameDef Default value: – maxLength: – minLength: – regularExpression: – Post-Build Variant Value: false Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: local: dependency: This parameter shall only be configured if the parameter PwmLow↔PowerStatesSupport is set to true.	Rejected: All PWM hardwares do not support asynchronous power state transitions
SWS_Pwm_CONSTR_00001	N/S	The ECUC partitions referenced by PwmKernelEcucPartitionRef shall be a subset of the ECUC partitions referenced by Pwm↔EcucPartitionRef.	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Pwm↔KernelEcucPartitionRef is not supported

Requirement	Status	Description	Notes
ECUC_Pwm_00150	N/S	Name: PwmKernelEcuc↔PartitionRef Parent Container: PwmGeneral Description: Maps the PWM kernel to zero or one ECUC partitions to assign the driver kernel to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the PWM driver is mapped to. Multiplicity: 0..1 Type: Reference to [Ecuc↔Partition] Post-Build Variant Multiplicity: true Post-Build Variant Value: true Multiplicity Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Value Configuration Class: Pre-compile time: X: All Variants Link time: – Post-build time: – Scope / Dependency: scope: ECU	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Pwm↔KernelEcucPartitionRef is not supported
SWS_Pwm_CONSTR_00002	N/S	If PwmEcucPartitionRef references one or more EC↔UC partitions, PwmKernel↔EcucPartitionRef shall have a multiplicity of one and reference one of these ECUC partitions as well.	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Pwm↔KernelEcucPartitionRef is not supported

3.5 Driver Limitations

The Pwm driver software has the following limitations :

For FlexTimer:

- When the corresponding FTMn_OBE_CTRL bit of instance is cleared via disabling **Ftm OBE Control** option on the configuration tool, then the output of channels belong to this instance will be tristated and can't set their state to the idle state after calling Pwm_DeInit function.
- On derivatives supporting the fault functionality, if Pwm_SetOutputToIdle/Ftm_Pwm_Ip_SwOutputControl is used and fault detection is also enabled for the channel, then when a fault is detected the output is forced to its safe value. However, when the fault condition has been cleared, the channel output can not to be reverted to the Idle state if inverting the polarity of channel is performed in function Ftm_Pwm_Ip_SwOutputControl.

- The Pwm_SetDutyPhaseShift function when using Modified_Combine_Mode is not support SyncUpdate = TRUE.

For FlexIO:

- FlexIO compare registers are not buffered so they are written directly, therefore the update of duty cycle and period will be immediate. Option PwmDutycycleUpdatedEndperiod and PwmPeriodUpdatedEndperiod is not supported for FlexIO.
- FlexIO notification only support for rising edge. Other cases are not supported, including falling edges and both edges notification.
- FlexIO does not support functionality of Pwm_SetOutputToIdle.
- FlexIO does not support for the duty cycle is (0)0% and (0x8000)100% and period is 0 as well When user call this function with parameter 0% , 100% duty cycle or 0 period ,it will generate a spike pulse.

3.6 Driver usage and configuration tips

How to configure in EB and S32DS:

Basically, PWM driver should be configured by EB tresos or S32DS in 3 steps(example with FTM):

Step 1: Configure FTM instances in PwmFtm tab

-Example with EB tresos

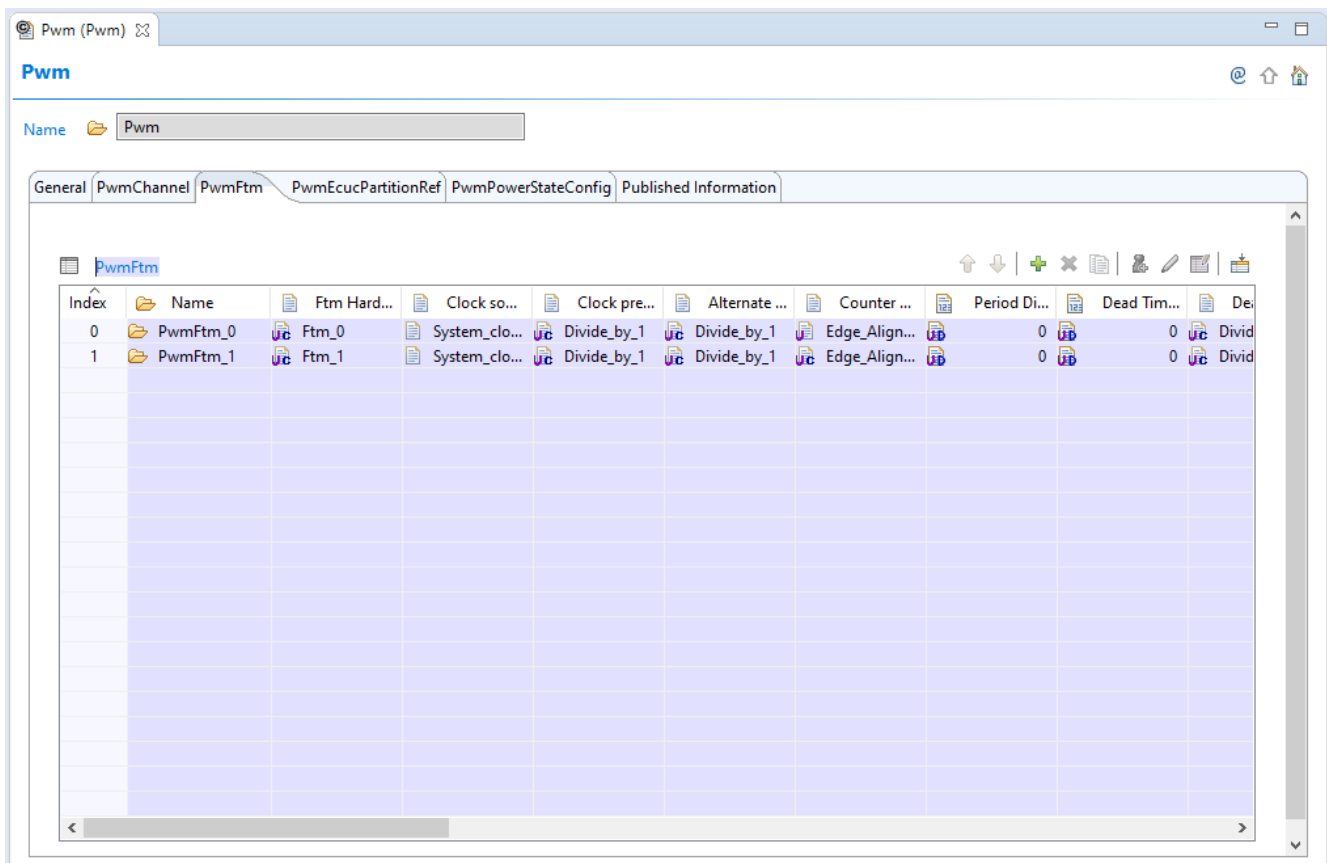


Figure 3.1 Configure FTM instances

Step 2: Configure FTM channels in PwmFtmCh tab

-Example with EB tresos

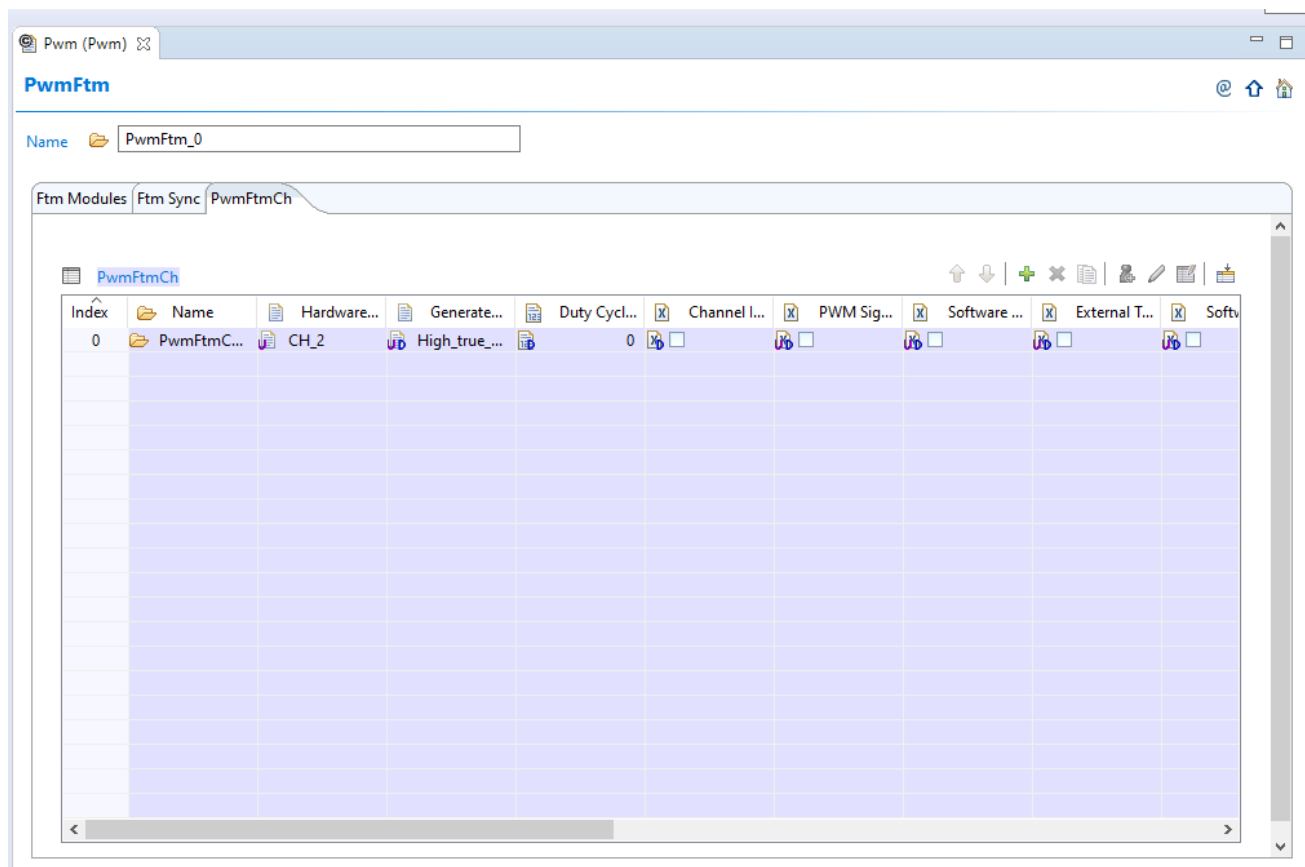


Figure 3.2 Configure FTM channels

Step 3: Configure logic channels in PwmChannel tab

-Example with EB tresos

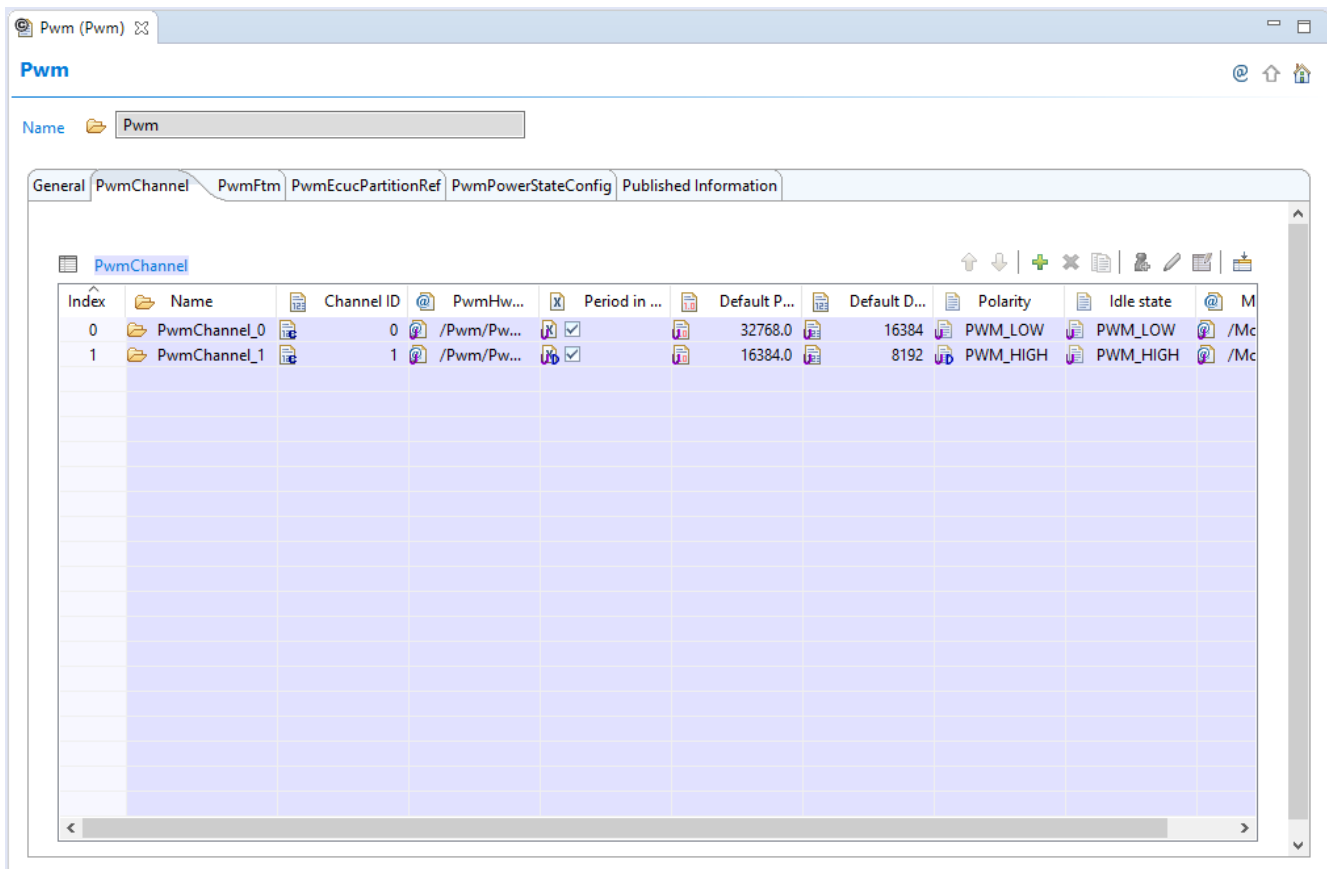


Figure 3.3 Configure logic channels

Note when using interrupt functions in the IPV layer:

To make it more convenient to use the 2 enable/disable interrupt functions when using the IPV layer, we have updated parameters of functions (Refer to Ftm_Pwm_Ip_NotifType). So when using these 2 functions you need to understand the meaning of the input parameters, namely:

- From FTM_PWM_IP_CHANNEL_0_NOTIFICATION to FTM_PWM_IP_CHANNEL_7_NOTIFICATION - Used for channel interrupts handling.
- FTM_PWM_IP_OVERFLOW_NOTIFICATION - Used for overflow interrupts handling
- FTM_PWM_IP_RELOAD_POINT_NOTIFICATION - Used for reload interrupts handling
- FTM_PWM_IP_FAULT_NOTIFICATION - Used for fault interrupts handling

3.7 Runtime errors

None.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

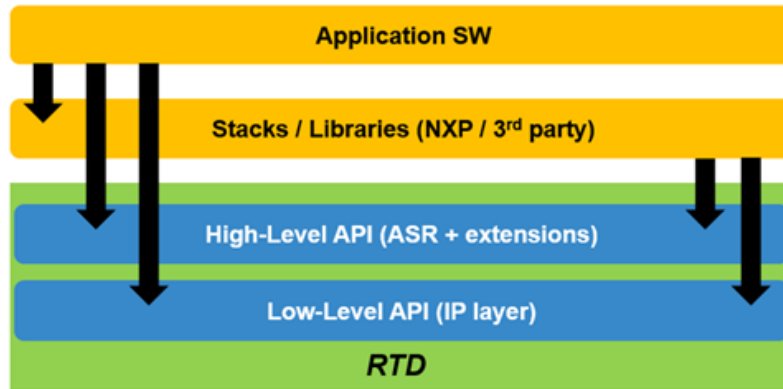


Figure 3.4 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

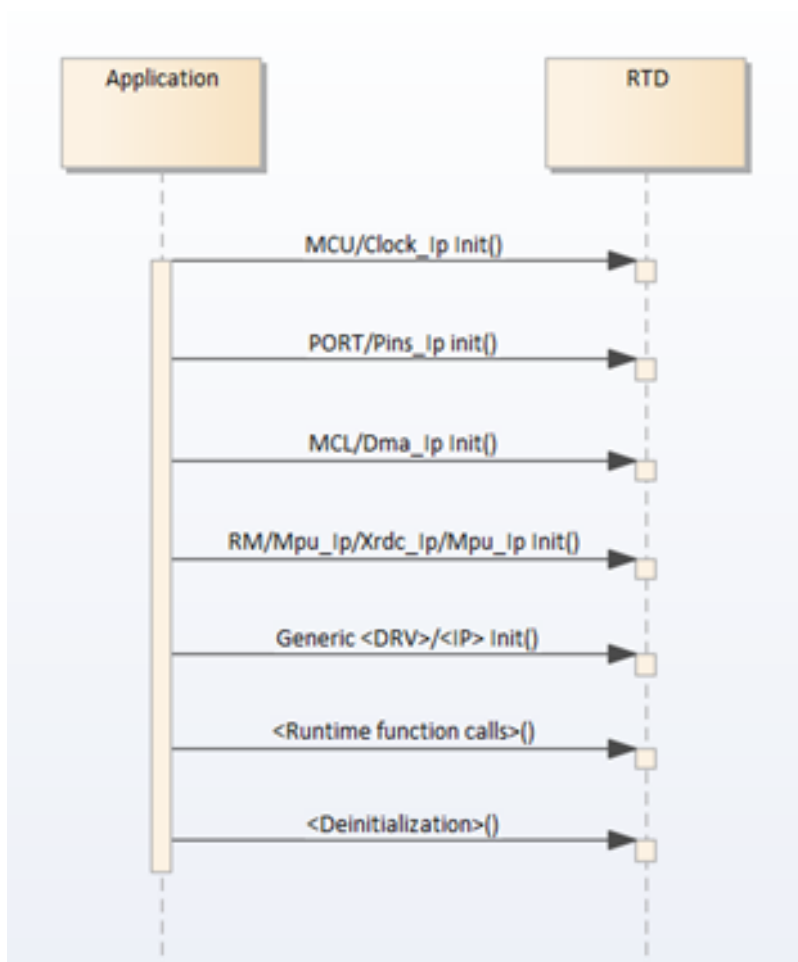


Figure 3.5 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

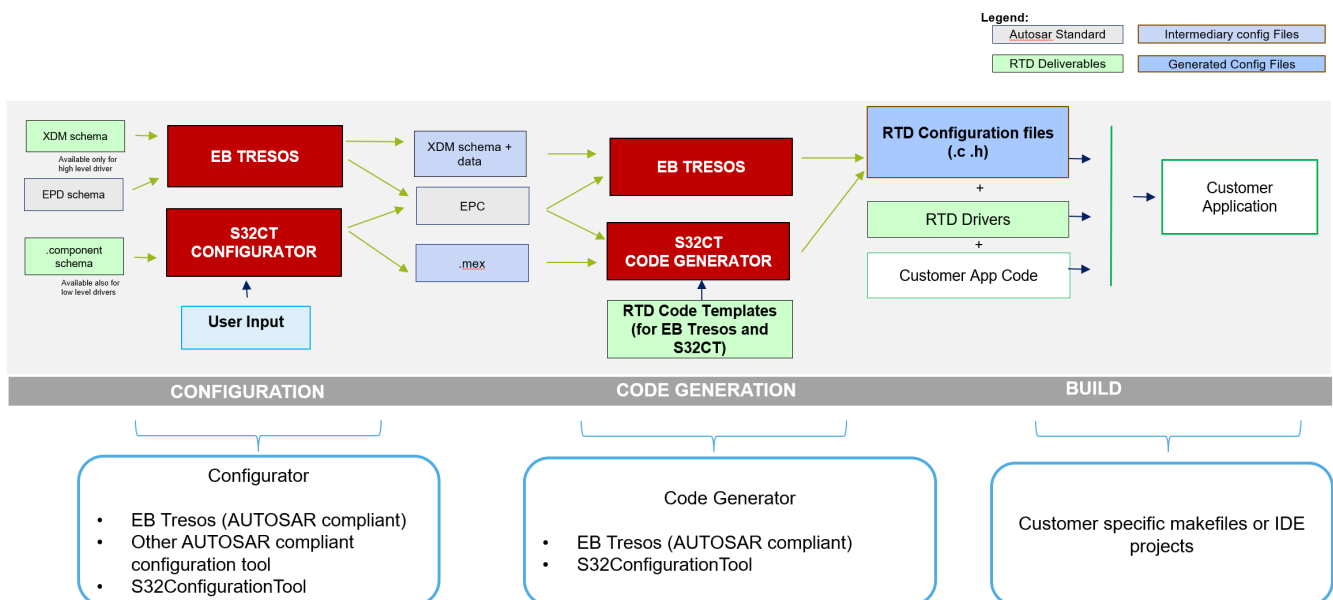


Figure 4.1 Configuration generation flow

- [Pwm](#)
- [FTM_PWM](#)
- [FLEXIO_PWM](#)

4.1 Pwm

- Module [Pwm](#)
 - Container [PwmChannelConfigSet](#)
 - * Container [PwmChannel](#)
 - Parameter [PwmChannelId](#)
 - Parameter [PwmChannelClass](#)
 - Parameter [PwmPeriodInTicks](#)
 - Parameter [PwmPeriodDefault](#)
 - Parameter [PwmDutycycleDefault](#)
 - Parameter [PwmPolarity](#)
 - Parameter [PwmIdleState](#)
 - Parameter [PwmNotification](#)
 - Reference [PwmChannelEcucPartitionRef](#)
 - Reference [PwmHwChannel](#)
 - Reference [PwmMcuClockReferencePoint](#)
 - * Container [PwmFtm](#)
 - Parameter [PwmHwInstance](#)
 - Container [PwmFtmClkCfg](#)
 - Parameter [PwmFtmClockSource](#)
 - Parameter [PwmFtmClockPrescaler](#)
 - Parameter [PwmFtmClockPrescalerAlternate](#)
 - Container [PwmFtmGlobalChCfg](#)
 - Parameter [PwmFtmCounterMode](#)
 - Parameter [PwmFtmPeriod](#)
 - Parameter [PwmFtmPeriodDither](#)
 - Parameter [PwmFtmDeadTime](#)
 - Parameter [PwmFtmDeadTimePrescaler](#)
 - Container [PwmFtmInstCfg](#)
 - Parameter [PwmFtmDebugMode](#)
 - Parameter [PwmFtmWriteProtection](#)
 - Parameter [PwmFtmInitTrigger](#)
 - Parameter [PwmFtmInitTriggerMode](#)
 - Parameter [PwmFtmOverflowIrq](#)
 - Parameter [PwmFtmReloadIrq](#)
 - Container [PwmFtmOverflowIrqCallback](#)
 - Parameter [PwmFtmOverflowIrqFunctionCallback](#)

- Parameter [PwmFtmOverflowIrqParameterCallback](#)
- Container [PwmFtmReloadIrqCallback](#)
 - Parameter [PwmFtmReloadIrqFunctionCallback](#)
 - Parameter [PwmFtmReloadIrqParameterCallback](#)
- Container [PwmFtmSyncCfg](#)
 - Parameter [PwmFtmSyncMode](#)
 - Parameter [PwmFtmHwTrig0](#)
 - Parameter [PwmFtmHwTrig1](#)
 - Parameter [PwmFtmHwTrig2](#)
 - Parameter [PwmFtmAutoHwTrig](#)
 - Parameter [PwmFtmMaxLoadPoint](#)
 - Parameter [PwmFtmMinLoadPoint](#)
 - Parameter [PwmFtmHalfCycleLoadPoint](#)
 - Parameter [PwmFtmHalfCycleLoadPointValue](#)
 - Parameter [PwmFtmReloadPointFrequency](#)
 - Parameter [PwmFtmInvertControl](#)
 - Parameter [PwmFtmSwOutControl](#)
 - Parameter [PwmFtmOutputMask](#)
 - Parameter [PwmFtmCounterInitial](#)
 - Parameter [PwmFtmCounterSync](#)
- Container [FtmGlobalFaultCfg](#)
 - Parameter [FtmFaultFunctionality](#)
 - Parameter [FtmFaultFilterValue](#)
 - Parameter [FtmFaultOutputState](#)
 - Parameter [FtmFaultIrqUsed](#)
 - Container [FtmFaultSettings](#)
 - Parameter [FtmFaultId](#)
 - Parameter [FtmFaultPolarity](#)
 - Parameter [FtmEnableFaultFilter](#)
 - Parameter [FtmFaultNotification](#)
- Container [PwmFtmSimCfg](#)
 - Parameter [PwmFtmObeControl](#)
- Container [PwmFtmCh](#)
 - Parameter [PwmFtmChId](#)
 - Parameter [PwmFtmChPulseType](#)
 - Parameter [PwmFtmChInitOutput](#)

- Parameter [PwmFtmChPolarity](#)
- Parameter [PwmFtmChDutyCycle](#)
- Parameter [PwmFtmChDutyCycleDither](#)
- Parameter [PwmFtmChInterrupt](#)
- Parameter [PwmFtmChPwmOutputEnable](#)
- Parameter [PwmFtmChSwCtrlEnable](#)
- Parameter [PwmFtmChExternTriggerEnable](#)
- Parameter [PwmFtmChSwCtrlValue](#)
- Parameter [PwmFtmChMatchLoadPointEnable](#)
- Parameter [PwmFtmPairChEnable](#)
- Container [PwmFtmChInterruptCallback](#)
 - Parameter [PwmFtmChInterruptFunctionCallback](#)
 - Parameter [PwmFtmChInterruptParameterCallback](#)
- Container [PwmFtmPairedCh](#)
 - Parameter [PwmFtmPairedChDeadtimeEnable](#)
 - Parameter [PwmFtmPairedChSync](#)
 - Parameter [PwmFtmPairedChExtTrig](#)
 - Parameter [PwmFtmPairedChComplementary](#)
 - Parameter [PwmFtmDutyDitheringPaired](#)
 - Parameter [PwmFtmPairedChCombineMode](#)
 - Parameter [PwmFtmPairedChComplementaryMode](#)
 - Parameter [PwmFtmPairedChPhaseShift](#)
- * Container [PwmFlexio](#)
 - Parameter [PwmHwInstance](#)
 - Container [PwmFlexioChannels](#)
 - Parameter [FlexioChId](#)
 - Parameter [FlexioPinId](#)
 - Parameter [FlexioChPrescaler](#)
 - Parameter [FlexioChPrescalerAlternate](#)
 - Parameter [FlexioChDutyCycle](#)
 - Parameter [FlexioChPeriod](#)
 - Parameter [FlexioChPolarity](#)
 - Reference [FlexioMclChRef](#)
 - Container [FlexioChIrqCallback](#)
 - Parameter [FlexioChIrqFunctionCallback](#)
 - Parameter [FlexioChIrqParameterCallback](#)

- * Container [PwmHwResourceConfig](#)
 - Parameter [PwmHwResourceId](#)
 - Parameter [PwmIsrEnable](#)
- Container [PwmGeneral](#)
 - * Parameter [PwmMultipartitionEnabled](#)
 - * Parameter [PwmDevErrorDetect](#)
 - * Parameter [PwmDutyCycleUpdatedEndperiod](#)
 - * Parameter [PwmPeriodUpdatedEndperiod](#)
 - * Parameter [PwmNotificationSupported](#)
 - * Parameter [PwmEnableUserModeSupport](#)
 - * Parameter [PwmLowPowerStatesSupport](#)
 - * Parameter [PwmPowerStateAsynchTransitionMode](#)
 - * Parameter [PwmEnableDualClockMode](#)
 - * Parameter [PwmEnableExternalTrigger](#)
 - * Parameter [PwmEnableMaskOutputs](#)
 - * Parameter [PwmFaultSupported](#)
 - * Parameter [PwmReloadNotificationSupported](#)
 - * Parameter [PwmMultiChannelSync](#)
 - * Parameter [PwmIndex](#)
 - * Reference [PwmEcucPartitionRef](#)
 - * Reference [PwmKernelEcucPartitionRef](#)
 - * Container [PwmPowerStateConfig](#)
 - Parameter [PwmPowerState](#)
 - Parameter [PwmPowerStateReadyCbkJRef](#)
- Container [PwmConfigurationOfOptApiServices](#)
 - * Parameter [PwmDeInitApi](#)
 - * Parameter [PwmGetOutputState](#)
 - * Parameter [PwmSetDutyCycle](#)
 - * Parameter [PwmSetOutputToIdle](#)
 - * Parameter [PwmSetPeriodAndDuty](#)
 - * Parameter [PwmVersionInfoApi](#)
 - * Parameter [PwmGetChannelStateApi](#)
 - * Parameter [PwmSetDutyCycle_NoUpdate](#)
 - * Parameter [PwmSetPeriodAndDuty_NoUpdate](#)
 - * Parameter [PwmSetDutyPhaseShift](#)
 - * Parameter [PwmSetChannelDeadTime](#)

- * Parameter [PwmForceOutputToZeroApi](#)
- * Parameter [PwmEnableTrigger](#)
- * Parameter [PwmDisableTrigger](#)
- * Parameter [PwmResetCounter](#)
- * Parameter [PwmFastUpdateApi](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Pwm

Configuration of Pwm (Pulse Width Modulation) module.

Included containers:

- [PwmChannelConfigSet](#)
- [PwmGeneral](#)
- [PwmConfigurationOfOptApiServices](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container PwmChannelConfigSet

Container contains the channel configuration parameter of the Pwm driver.

Included subcontainers:

- [PwmChannel](#)
- [PwmFtm](#)
- [PwmFlexio](#)
- [PwmHwResourceConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Container PwmChannel

Configuration of an individual Pwm channel.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.4 Parameter PwmChannelId

Channel ID of the Pwm channel. This value will be assigned to the symbolic name derived of the PwmChannel container short name.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.1.5 Parameter PwmChannelClass

Class of Pwm Channel.

PWM_FIXED_PERIOD - Period of the channel will not be changed.

Configuration Parameters

PWM_FIXED_PERIOD_SHIFTED - Period of the channel will not be changed, and support with phase shift feature.

PWM_VARIABLE_PERIOD - Period of the channel can be changed.

Due to Ftm hardware specific feature that the counter register and period register are shared common for all channels in one Ftm instance, therefore if current channel is configured *PWM_FIXED_PERIOD* class, but another channel in the same Ftm instance is configured *PWM_VARIABLE_PERIOD*, then when that channel call function to change period, current channel will change period, too.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	PWM_FIXED_PERIOD
literals	['PWM_FIXED_PERIOD', 'PWM_FIXED_PERIOD_SHIFTED', 'PWM_VARIABLE_PERIOD']

4.1.6 Parameter PwmPeriodInTicks

Check this option to configure Default Period unit in ticks, or uncheck this to configure Default Period unit in seconds.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.1.7 Parameter PwmPeriodDefault

Default period value of Pwm channel at initialization.

The measure unit are in ticks (if Period In Ticks checked), or in seconds (if unchecked)

Valid range: [0, 65534]

The maximum period tick is 65534 (instead of maximum value of period register 65535) in order to achieve perfect 0 or 100% duty cycle.

All channels which are in the same Ftm instance with current channel must have the same Default Period value, due to Ftm hardware specific feature that the period register is shared common for all channels in one Ftm instance.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	1.0
max	65534.0
min	0.0

4.1.8 Parameter PwmDutycycleDefault

Default value for duty cycle of Pwm channel at initialization.

0 represents for 0% duty cycle

16384 (0x4000) represents for 50% duty cycle

32768 (0x8000) represents for 100% duty cycle

Valid value: [0,32768]

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	16384
max	32768
min	0

4.1.9 Parameter PwmPolarity

Define the polarity of Pwm channel at initialization.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	PWM_HIGH
literals	['PWM_HIGH', 'PWM_LOW']

4.1.10 Parameter PwmIdleState

Define Pwm channel state when the output is set to idle.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	PWM_LOW
literals	['PWM_HIGH', 'PWM_LOW']

4.1.11 Parameter PwmNotification

User callback notification function.

This option is only activated when PwmGeneral/PwmNotificationSupported is checked.

Use NULL_PTR without any quotes to determine no notification function is used.

If the string is different from above, it will be used as the notification function name.

Notification does not apply to channel that its alignment type is PWM_CENTER_ALIGNED.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.12 Reference PwmChannelEcucPartitionRef

Maps a Pwm channel to zero or multiple ECUC partitions to limit the access to this channel group.

The ECUC partitions referenced are a subset of the ECUC partitions where the Pwm driver is mapped to.

When users choose **ENABLE** multipartition feature by checking **PwmMultipartitionEnabled** option, this will force to configure at least 1 ECUC partition in this list that is referenced from ECUC module; OR when **DISABLE** multipartition feature, user have to remove all ECUC partitions in this list.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.13 Reference PwmHwChannel

Select the hw channel on which the functionality of the current PWM channel will be implemented.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Pwm/PwmChannelConfigSet/PwmFtm/PwmFtmCh', '/TS_T40D2M30I0R0/Pwm/PwmChannelConfigSet/PwmFlexio/PwmFlexio↵ Channels']

4.1.14 Reference PwmMcuClockReferencePoint

Reference to the clock source configuration, which is set in the MCU driver configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSetting↵ Config/McuClockReferencePoint

4.1.15 Container PwmFtm

Configuration of a Ftm module available on the platform.

Included subcontainers:

- [PwmFtmClkCfg](#)
- [PwmFtmGlobalChCfg](#)
- [PwmFtmInstCfg](#)
- [PwmFtmSyncCfg](#)
- [FtmGlobalFaultCfg](#)
- [PwmFtmSimCfg](#)
- [PwmFtmCh](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.16 Parameter PwmHwInstance

Select the Ftm hardware module instance available on the current platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	Ftm_0
literals	['Ftm_0', 'Ftm_1', 'Ftm_2', 'Ftm_3']

4.1.17 Container PwmFtmClkCfg

Ftm Clock configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.18 Parameter PwmFtmClockSource

Select Ftm module clock source.

This option will be written into Clock Source Selection bitfield (CLKS) of Status and Control register (SC).

FTM_PWM_IP_CLOCK_SOURCE_NONE : No input clock is given to Ftm. Ftm counter is not operational.

FTM_PWM_IP_CLOCK_SOURCE_SYSTEMCLK : System clock is given to Ftm, which is referred to PER_CLK.

Configuration Parameters

FTM_PWM_IP_CLOCK_SOURCE_FIXEDCLK : An fixed frequency clock is given to Ftm. This selection is not supported in current platform.

FTM_PWM_IP_CLOCK_SOURCE_EXTERNALCLK : An external clock is given to Ftm, which is referred to Ftm_x_REF_CLK (x = 0 or 1).

Clock source and frequency are dependent on the value of the clock that is configured in MCU

and referred by MCU clock reference. Make sure that the correct reference is used for the configured clock.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	System_clock
literals	['No_clock', 'System_clock', 'Fixed_clock', 'External_clock']

4.1.19 Parameter PwmFtmClockPrescaler

Select the prescaler value for clock source.

This option will be written into Prescale Factor Selection bitfield (PS) of Status and Control register (SC).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Divide_by_1
literals	['Divide_by_1', 'Divide_by_2', 'Divide_by_4', 'Divide_by_8', 'Divide_by_16', 'Divide_by_32', 'Divide_by_64', 'Divide_by_128']

4.1.20 Parameter PwmFtmClockPrescalerAlternate

Select the prescaler value for clock source.

This option will be written into Prescale Factor Selection bitfield (PS) of Status and Control register (SC).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Divide_by_1
literals	['Divide_by_1', 'Divide_by_2', 'Divide_by_4', 'Divide_by_8', 'Divide_by_16', 'Divide_by_32', 'Divide_by_64', 'Divide_by_128']

4.1.21 Container PwmFtmGlobalChCfg

Ftm Clock configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.22 Parameter PwmFtmCounterMode

Select edge alignment type which is applied to all channels in current Ftm module.

This option will be written into Center-Aligned Pwm Select (CPWMS) of Status and Control register (SC).

FTM_PWM_IP_EDGE_ALIGNED : The leading edges of all Pwm signals are aligned with the beginning of the period.

Ftm counter operates in Up Counting mode.

FTM_PWM_IP_CENTER_ALIGNED : The pulse width centers for all Pwm channels are aligned with the value of CNTIN.

Ftm counter operates in Up-Down Counting mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	Edge_Aligned_mode
literals	['Edge_Aligned_mode', 'Center_Aligned_mode']

4.1.23 Parameter PwmFtmPeriod

User need to check the FTM frequency before enter the period in frequency of PWM. The min pwm frequency must be greater than (FTM counter frequency / 65536).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	65535
min	0

4.1.24 Parameter PwmFtmPeriodDither

The value of the dithering applied to the PWM frequency.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD

Property	Value
default Value	0
max	31
min	0

4.1.25 Parameter PwmFtmDeadTime

Deadtime counter used to create a offset between the output of two consecutive Ftm channels used complementary modes.

Complementary channels behave as a single channel with two outputs which may have a deadtime between them.

This option will be written into Extended Deadtime Value bitfield (DTVALEX) and Deadtime Value bitfield (DTVAL) in Deadtime Configuration register (DEADTIME).

Valid value: [0,1023]

For channels which are configured as independent this value has no effect.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	1023
min	0

4.1.26 Parameter PwmFtmDeadTimePrescaler

Deadtime prescaler.

This option will be written into Deadtime Prescaler Value (DTPS) in Deadtime Configuration register (DEADTIME).

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Divide_by_1
literals	['Divide_by_1', 'Divide_by_4', 'Divide_by_16']

4.1.27 Container PwmFtmInstCfg

Ftm Instance configuration settings.

Included subcontainers:

- [PwmFtmOverflowIrqCallback](#)
- [PwmFtmReloadIrqCallback](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.28 Parameter PwmFtmDebugMode

Select the Ftm behavior in Debug mode. Refer to Debug mode sub-chapter for more details.

This option will be written into Debug Mode (BDMODE) in Configuration register (CONF).

Mode 00

Counter - Stopped

Flag bit - Can be set

Output - Normal

Bufferd Registers - Writes bypass the registers buffers

Mode 01

Counter - Stopped

Flag bit - Is not set

Output - Forced to their safe value according to POLn bit

Bufferd Registers - Writes bypass the registers buffers

Mode 10

Counter - Stopped

Flag bit - Is not set

Output - Frozen when the chip enters Debug mode

Bufferd Registers - Writes bypass the registers buffers

Mode 11

Counter - Running

Flag bit - Can be set

Output - Normal

Bufferd Registers - Normal

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Debug_Mode_00
literals	['Debug_Mode_00', 'Debug_Mode_01', 'Debug_Mode_10', 'Debug_Mode_11']

4.1.29 Parameter PwmFtmWriteProtection

Enable/Disable the write protection of FTM register/bitfields.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.30 Parameter PwmFtmInitTrigger

Enable/Disable the generation of an initialization trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.31 Parameter PwmFtmInitTriggerMode

Select the generation of initialization trigger mode.

Trigger on Reload Point - Initialization trigger is generated when FTM counter reaches a reload point.

Trigger on Counter Update - Initialization trigger is generated when FTM counter is updated.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	Trigger_on_Reload_Point
literals	['Trigger_on_Reload_Point', 'Trigger_on_Counter_Update']

4.1.32 Parameter PwmFtmOverflowIrq

Enable/Disable the generation of an initialization trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.33 Parameter PwmFtmReloadIrq

Enable/Disable the generation of an initialization trigger.

Do not support for this implementation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.34 Container PwmFtmOverflowIrqCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.35 Parameter PwmFtmOverflowIrqFunctionCallback

User callback function.

NOTE: Use `NULL_PTR` without any quotes. If the used string is different from `NULL_PTR` it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.36 Parameter PwmFtmOverflowIrqParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.37 Container PwmFtmReloadIrqCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.38 Parameter PwmFtmReloadIrqFunctionCallback

User callback function.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.39 Parameter PwmFtmReloadIrqParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.40 Container PwmFtmSyncCfg

Ftm Synchronization configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.41 Parameter PwmFtmSyncMode

Configure FTM Sync to use Software or Hardware trigger modes

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	Software_sync_trigger
literals	['Software_sync_trigger', 'Hardware_sync_trigger']

4.1.42 Parameter PwmFtmHwTrig0

Enables/disables hardware trigger for register sync

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.43 Parameter PwmFtmHwTrig1

Enables/disables hardware trigger for register sync

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.44 Parameter PwmFtmHwTrig2

Enables/disables hardware trigger for register sync

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.45 Parameter PwmFtmAutoHwTrig

Enable/disable auto clear of triggers in hardware trigger mode

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.46 Parameter PwmFtmMaxLoadPoint

Enable/disable max counter value loading point

(for Center-Aligned Pwm, the Reload point is in middle of period)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.47 Parameter PwmFtmMinLoadPoint

Enable/disable min counter value loading point.

(for Center-Aligned Pwm, the Reload point is in end of period)

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.48 Parameter PwmFtmHalfCycleLoadPoint

Enable/disable half cycle loading point

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.49 Parameter PwmFtmHalfCycleLoadPoint Value

This option will be written into Half Cycle register (HCR).

After FTM counter reaches this value, a reload opportunity is generated if bit HCSEL in PWMLOAD register is enabled.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	65534
min	0

4.1.50 Parameter PwmFtmReloadPointFrequency

Select the number of enabled reload opportunities should happen until an enabled reload opportunity becomes a reload point.

This option will be written into Frequency of the Reload Opportunities bitfield (LDFQ) in Configuration register (CONF).

For safety, this option is activated only when PwmGeneral/PwmDutycycleUpdatedEndperiod is checked.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	31
min	0

4.1.51 Parameter PwmFtmInvertControl

Select when the Inverting Control register is synchronized.

Sync_disabled: synchronization for the Inverting Control register is disabled.

Sync_on_FTM_Clock_rising_edge: synchronization for the Inverting Control register is done on each FTM Clock rising edge.

Sync_on_FTM_Sync_Trigger: synchronization for the Inverting Control register is done on each FTM Sync Trigger.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Clock_rising_edge
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_Sync_Trigger']

4.1.52 Parameter PwmFtmSwOutControl

Select when the Software Output Control register is synchronized.

Sync_on_FTM_Clock_rising_edge: synchronization for the Software Output Control register is done on each FTM Clock rising edge.

Sync_on_FTM_Sync_Trigger: synchronization for the Software Output Control register is done on each FTM Sync Trigger.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Clock_rising_edge
literals	['Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_Sync_Trigger']

4.1.53 Parameter PwmFtmOutputMask

Select when the Output Mask register is synchronized.

Sync_disabled: synchronization for the Output Mask register is disabled.

Sync_on_FTM_Clock_rising_edge: synchronization for the Output Mask register is done on each FTM Clock rising edge.

Sync_on_FTM_Sync_Trigger: synchronization for the Output Mask register is done on each FTM Sync Trigger.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Clock_rising_edge
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_Sync_Trigger']

4.1.54 Parameter PwmFtmCounterInitial

Select when the Counter Initial, max counter, halfCycle and Counter Value register is synchronized.

Sync_disabled: synchronization for the FTM registers is disabled.

Sync_on_FTM_Clock_rising_edge: synchronization for the FTM registers is done on each FTM Clock rising edge.

Sync_on_FTM_Sync_Trigger: synchronization for the FTM registers is done on each FTM Sync Trigger.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Sync_Trigger
literals	['Sync_disabled', 'Sync_on_FTM_Clock_rising_edge', 'Sync_on_FTM_Sync_Trigger']

4.1.55 Parameter PwmFtmCounterSync

Select when the Ftm Counter register is synchronized.

Sync_disabled: synchronization for the Ftm Counter register is disabled.

Sync_on_FTM_Sync_Trigger: synchronization for the Ftm Counter register is done on each FTM Sync Trigger.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	Sync_on_FTM_Sync_Trigger
literals	['Sync_disabled', 'Sync_on_FTM_Sync_Trigger']

4.1.56 Container FtmGlobalFaultCfg

Global Fault configuration.

Autosar Requirements: Not specified.

Included subcontainers:

- [FtmFaultSettings](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.57 Parameter FtmFaultFunctionality

Selects the fault control mode

> *FAULT_DISABLED* -> Fault control disabled for all channels.

> *FAULT_EVEN_CHANNELS_MANUAL_CLEAR* -> Fault control enabled for even channels (0, 2, 4..) with manual fault clearing.

> *FAULT_ALL_CHANNELS_MANUAL_CLEAR* -> Fault control enabled for all channels with manual fault clearing.

> *FAULT_ALL_CHANNELS_AUTO_CLEAR* -> Fault control enabled for all channels with automatic fault clearing.

Note: For any automatic fault clear modes, a Fault Notification function should be defined. In automatic modes, the hardware shall

be configured into manual fault clearing and automatic fault clearing shall be implemented by software.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FAULT_DISABLED
literals	['FAULT_DISABLED', 'FAULT_EVEN_CHANNELS_MANUAL_CLEAR', 'FAULT_ALL_CHANNELS_MANUAL_CLEAR', 'FAULT_ALL_CHANNELS_AUTO_CLEAR']

4.1.58 Parameter FtmFaultFilterValue

Selects the filter value for the fault inputs.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.1.59 Parameter FtmFaultOutputState

Specifies the fault state for the PWM channel output during fault conditions

SAFE_VALUE: Outputs will be placed into safe values when fault events in ongoing (defined by polarity)

TRI_STATE: Outputs will be tri-stated when fault event is ongoing

Autosar Requirements: Not specified.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	SAFE_VALUE
literals	['SAFE_VALUE', 'TRI_STATE']

4.1.60 Parameter FtmFaultIrqUsed

Enable/Disable the generation of an the fault notification.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.61 Container FtmFaultSettings

Configuration of an individual Ftm fault input.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	4
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.62 Parameter FtmFaultId

Select the Ftm Fault Id available in module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FLT_0
literals	['FLT_0', 'FLT_1', 'FLT_2', 'FLT_3']

4.1.63 Parameter FtmFaultPolarity

Defines the polarity of the fault input.

Autosar Requirements: not specified.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	POLARITY_HIGH
literals	['POLARITY_HIGH', 'POLARITY_LOW']

4.1.64 Parameter FtmEnableFaultFilter

Enables/Disables the filter for the fault input.

Autosar Requirements: Not specified.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.65 Parameter FtmFaultNotification

User callback function.

Note: Please use NULL or NULL_PTR w/o any quotes. If the used string is different from NULL or NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.66 Container PwmFtmSimCfg

Ftm Sim configuration settings.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.67 Parameter PwmFtmObeControl

Unchecked - The FTM channel output is put to safe state when the FTM counter is enabled and the FTM channel output is enabled by Fault Control (FTM_MODE[FAULTM]≠0 and FTM_FLTCTRL[FSTATE]=0) and PWM is enabled (FTM_SC[PWMENn] = 1). Otherwise the channel output is tristated.

Checked - The FTM channel output state is retained when the channel is in output mode. The output channel is tristated when the channel is in input capture [DECAPEN=0, COMBINE=0, MSnB:MSnA=0] or dual edge capture mode [DECAPEN=1].

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	true

4.1.68 Container PwmFtmCh

Configuration of an individual Ftm hardware channel.

Included subcontainers:

- [PwmFtmChInterruptCallback](#)
- [PwmFtmPairedCh](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.69 Parameter PwmFtmChId

Select the Ftm hardware channel available in module. This value will be assigned to the symbolic name derived of the Pwm-Channel container short name.

NOTE: 6 channels from FlexTimer3(FTM3_Ch0 to FTM3_Ch5) are internally connected as PWM signals to GDU in all S32M24x

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CH_0
literals	['CH_0', 'CH_1', 'CH_2', 'CH_3', 'CH_4', 'CH_5', 'CH_6', 'CH_7']

4.1.70 Parameter PwmFtmChPulseType

The type of the generated PWM signal.

For Edge/Center aligned PWM:

High-true pulse - clear Output on match

Low-true pulse - set Output on match

For combined modes:

High-true pulse - set on channel (n) match, and clear on channel (n+1) match

Low-true pulse - clear on channel (n) match, and set on channel (n+1) match

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	High_true_pulse
literals	['High_true_pulse', 'Low_true_pulse']

4.1.71 Parameter PwmFtmChInitOutput

The initial state of the channel output is configured in PWM signal.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	LOW
literals	['LOW', 'HIGH']

4.1.72 Parameter PwmFtmChPolarity

Select operation mode of this Ftm channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Channel_active_HIGH
literals	['Channel_active_LOW', 'Channel_active_HIGH']

4.1.73 Parameter PwmFtmChDutyCycle

Define the duty cycle value (in tick) for this channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65534

Property	Value
min	0

4.1.74 Parameter PwmFtmChDutyCycleDither

Define the duty cycle dither value (in tick) for this channel.

Valid value: [0, 1023]

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	31
min	0

4.1.75 Parameter PwmFtmChInterrupt

Enable/Disable the generation of an initialization trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.76 Parameter PwmFtmChPwmOutputEnable

This option is used to enable/disable the output of the PWM signal on the channel pin via controlling the PWMENn bit field in the SC register.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	true

4.1.77 Parameter PwmFtmChSwCtrlEnable

Enable/Disable the software control for this channels output.

To avoid a spike pulse when configuring the duty cycle value of channel: 0% or 100% in case the multiple variants used, the software ouput control

need to be used. It will be handled directly in the driver code without depending on the value of this node, this will make the implementation more simple

as well as the usage of this feature easier.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.78 Parameter PwmFtmChExternTriggerEnable

Enable/Disable the generation of an external trigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.79 Parameter PwmFtmChSwCtrlValue

The software output control forces 0 or 1 to the channel output.

To avoid a spike pulse when configuring the duty cycle value of channel: 0% or 100% in case the multiple variants used, the software output control

need to be used. It will be handled directly in the driver code without depending on the value of this node, this will make the implementation more simple

as well as the usage of this feature easier.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.80 Parameter PwmFtmChMatchLoadPointEnable

Enable/Disable the channel match as an reload opportunity.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
default Value	false

4.1.81 Parameter PwmFtmPairChEnable

Enable the paired channel feature.

The channels n and $n+1$ will be used as a channel pair.

Note: Cannot be used for odd channels.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.82 Container PwmFtmChInterruptCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.83 Parameter PwmFtmChInterruptFunctionCallback

User callback function.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.84 Parameter PwmFtmChInterruptParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	NULL_PTR

4.1.85 Container PwmFtmPairedCh

Configuration of a paired Ftm hardware channel.

This will configure the channel $n+1$.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.86 Parameter PwmFtmPairedChDeadtimeEnable

Enable the deadtime delay for Ftm channel.

The deadtime delay insertion ensures that no two complementary signals (channels n and $n+1$) drive the active state at the same time.

This option is activated only when Ftm channel is using operation mode that has COMPLEMENTARY (COMBINED_COMPLEMENTARY, PHASE_SHIFTED_COMPLEMENTARY).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.87 Parameter PwmFtmPairedChSync

Enable the synchronization of the channel match value registers for this pair.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.1.88 Parameter PwmFtmPairedChExtTrig

Enable the generation of an external trigger for channel $n+1$ of this pair.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.89 Parameter PwmFtmPairedChComplementary

Enable the complementary mode for Ftm channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.90 Parameter PwmFtmDutyDitheringPaired

The PWM edge dithering in Combine/Modified Combine Mode can be done in the channel (n) match edge or in the channel (n+1) match edge.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	false

4.1.91 Parameter PwmFtmPairedChCombineMode

Select combine mode for the Ftm channel pair.

NOTE:Combine and Modified Combine mode are not supported in Center-Aligned mode

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Combine_modes_disabled
literals	['Combine_modes_disabled', 'Combined_Mode', 'Modified_Combine_Mode']

4.1.92 Parameter PwmFtmPairedChComplementaryMode

Select complementary mode of the Ftm channel n+1.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Invert_Output
literals	['Invert_Output', 'Duplicate_Output']

4.1.93 Parameter PwmFtmPairedChPhaseShift

Define the offset value (in tick) from 1 that the leading edge of pulse signal will start.

Valid value: [0, 65533]

NOTE:Phase Shift is not supported in Center-Aligned mode

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	65533
min	0

4.1.94 Container PwmFlexio

Configuration of an Flexio module available on the platform.

Included subcontainers:

- [PwmFlexioChannels](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD

4.1.95 Parameter PwmHwInstance

Select the hardware Flexio module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	Flexio_0
literals	['Flexio_0']

4.1.96 Container PwmFlexioChannels

List of Flexio channels available on the platform.

Included subcontainers:

- [FlexioChIrqCallback](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.97 Parameter FlexioChId

Select one of the Flexio channels available on the platform.

NOTE: This also selects the used timer.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	CH_0
literals	['CH_0', 'CH_1', 'CH_2', 'CH_3']

4.1.98 Parameter FlexioPinId

Select one of the Flexio Pins available on the platform.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Configuration Parameters

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	PIN_0
literals	['PIN_0', 'PIN_1', 'PIN_2', 'PIN_3', 'PIN_5', 'PIN_6', 'PIN_7']

4.1.99 Parameter FlexioChPrescaler

Select clock prescaler used for this Flexio channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	FLEXIO_PWM_IP_CLK_DIV_1
literals	['FLEXIO_PWM_IP_CLK_DIV_1', 'FLEXIO_PWM_IP_CLK_DIV_16', 'FLEXIO_PWM_IP_CLK_DIV_256']

4.1.100 Parameter FlexioChPrescalerAlternate

Select alternative clock prescaler used for this Flexio channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	FLEXIO_PWM_IP_CLK_DIV_1
literals	['FLEXIO_PWM_IP_CLK_DIV_1', 'FLEXIO_PWM_IP_CLK_DIV_16', 'FLEXIO_PWM_IP_CLK_DIV_256']

4.1.101 Parameter FlexioChDutyCycle

Value for duty cycle used for initialization.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	256
min	0

4.1.102 Parameter FlexioChPeriod

Period value used at initialization.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	512
min	0

4.1.103 Parameter FlexioChPolarity

Define the output polarity of the channel.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Configuration Parameters

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	FLEXIO_PWM_IP_ACTIVE_HIGH
literals	['FLEXIO_PWM_IP_ACTIVE_HIGH', 'FLEXIO_PWM_IP_ACTIVE_LOW']

4.1.104 Reference FlexioMclChRef

Select the Flexio MCL hw channel on which the the current PWM channel will be implemented.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels

4.1.105 Container FlexioChIrqCallback

Configure Notification function and parameter for interrupt handler callback.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.106 Parameter FlexioChIrqFunctionCallback

User callback function.

NOTE: Use `NULL_PTR` without any quotes. If the used string is different from `NULL_PTR` it will be used as the configured

function name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.107 Parameter FlexioChIrqParameterCallback

User callback parameter.

NOTE: Use NULL_PTR without any quotes. If the used string is different from NULL_PTR it will be used as the configured parameter name.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.108 Container PwmHwResourceConfig

List of all HW channel resources for Pwm module.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.109 Parameter PwmHwResourceId

Id of the HW available platform wide and usable by the Pwm module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FTM_0_CH_1
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_1_CH_0', 'FTM_1_CH_2', 'FTM_1_CH_3', 'FTM_1_CH_4', 'FTM_1_CH_6', 'FTM_1_CH_7', 'FTM_2_CH_0', 'FTM_2_CH_1', 'FTM_2_CH_2', 'FTM_2_CH_3', 'FTM_2_CH_5', 'FTM_2_CH_6', 'FTM_2_CH_7', 'FTM_3_CH_0', 'FTM_3_CH_1', 'FTM_3_CH_2', 'FTM_3_CH_3', 'FTM_3_CH_4', 'FTM_3_CH_5', 'FTM_3_CH_6', 'FTM_3_CH_7', 'FLEXIO_0_CH_0_PIN_0', 'FLEXIO_0_CH_0_PIN_1', 'FLEXIO_0_CH_0_PIN_2', 'FLEXIO_0_CH_0_PIN_3', 'FLEXIO_0_CH_0_PIN_4', 'FLEXIO_0_CH_0_PIN_5', 'FLEXIO_0_CH_0_PIN_6', 'FLEXIO_0_CH_0_PIN_7', 'FLEXIO_0_CH_1_PIN_0', 'FLEXIO_0_CH_1_PIN_1', 'FLEXIO_0_CH_1_PIN_2', 'FLEXIO_0_CH_1_PIN_3', 'FLEXIO_0_CH_1_PIN_4', 'FLEXIO_0_CH_1_PIN_5', 'FLEXIO_0_CH_1_PIN_6', 'FLEXIO_0_CH_1_PIN_7', 'FLEXIO_0_CH_2_PIN_0', 'FLEXIO_0_CH_2_PIN_1', 'FLEXIO_0_CH_2_PIN_2', 'FLEXIO_0_CH_2_PIN_3', 'FLEXIO_0_CH_2_PIN_4', 'FLEXIO_0_CH_2_PIN_5', 'FLEXIO_0_CH_2_PIN_6', 'FLEXIO_0_CH_2_PIN_7', 'FLEXIO_0_CH_3_PIN_0', 'FLEXIO_0_CH_3_PIN_1', 'FLEXIO_0_CH_3_PIN_2', 'FLEXIO_0_CH_3_PIN_3', 'FLEXIO_0_CH_3_PIN_4', 'FLEXIO_0_CH_3_PIN_5', 'FLEXIO_0_CH_3_PIN_6', 'FLEXIO_0_CH_3_PIN_7']

4.1.110 Parameter PwmIsrEnable

Status of the HW Interrupt (true - Interrupt shall be enable platform wide; false - Interrupt shall be disabled platform wide.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.111 Container PwmGeneral

Container used for storing the general configuration of Pwm.

Included subcontainers:

- [PwmPowerStateConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.112 Parameter PwmMultipartitionEnabled

Switch to enable/disable multipartition feature.

User can choose *ENABLE* multipartition feature by checking this option, this will force to configure at least 1 ECUC partition in *PwmChannelEcucPartitionRef*, and each Pwm channel in *PwmChannel* to configure at least 1 ECUC partition reference in *PwmChannelEcucPartitionRef* container to fulfill generating code condition; OR uncheck this option to *DISABLE*

multipartition feature, performing this action will force user to remove all ECUC partition reference in every Pwm channels contained

in *PwmChannel* and in *PwmChannelEcucPartitionRef*.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
default Value	false

4.1.113 Parameter PwmDevErrorDetect

Switch to enable/disable the development error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	true

4.1.114 Parameter PwmDutycycleUpdatedEndperiod

Switch to enable the update of the duty cycle parameter at the end of the current period.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.115 Parameter PwmPeriodUpdatedEndperiod

Switch to enable the update of the period parameter at the end of the current period.

In current implementation, this option is locked and always enable by default.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	true

4.1.116 Parameter PwmNotificationSupported

Switch to indicate that the notifications are supported.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.117 Parameter PwmEnableUserModeSupport

If enabled, the Pwm module will adapt to run from User Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.118 Parameter PwmLowPowerStatesSupport

If enabled, hardware offers low power mode and adds all power state management related APIs

(Pwm_SetPowerState, Pwm_GetCurrentPowerState, Pwm_GetTargetPowerState,

Pwm_PreparePowerState, Pwm_Main_PowerTransitionManager),

indicating if the hardware offers low power state management.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.119 Parameter PwmPowerStateAsynchTransitionMode

Enable the support of the Pwm driver to the asynchronous power state transition.

This feature is not supported and is rejected, as all hardware modules do not support asynchronous power state transitions.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.120 Parameter PwmEnableDualClockMode

Switch to enable/disable dual clock mode feature, which will add/remove the service Pwm_SetClockMode() from the code.

This feature is used when the prescaler value needs to be changed to maintain same period at different frequency.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.121 Parameter PwmEnableExternalTrigger

Enable external trigger generation.

This option will activate the use of PwmEnableTrigger and PwmDisableTrigger.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.122 Parameter PwmEnableMaskOutputs

Enable mask output feature, which will add/remove the services Pwm_MaskOutputs and Pwm_UnMaskOutputs from the code.

This option is activated only when PwmGeneral/PwmMultiChannelSync is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.123 Parameter PwmFaultSupported

This enables the fault functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.124 Parameter PwmReloadNotificationSupported

This enables the reload point interrupt.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.125 Parameter PwmMultiChannelSync

Enable the update synchronous feature for several channels.

This option will activate the use of PwmSetDutyCycle_NoUpdate, PwmSetPeriodAndDuty_NoUpdate, PwmSetDutyPhaseShift and PwmEnableMaskOutputs.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.126 Parameter PwmIndex

Specify the InstanceId of this module instance. If only one instance is present it shall have the ID 0.

In current implementation, this feature is not used.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	0
max	4294967295
min	0

4.1.127 Reference PwmEcucPartitionRef

Maps the Pwm driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

When users choose ENABLE multipartition feature by checking PwmMultipartitionEnabled option, this will force to configure at least 1 ECUC partition in this list that is referenced from ECUC module; OR when DISABLE multipartition feature, user have to remove all ECUC partitions in this list.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC

Property	Value
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.128 Reference PwmKernelEcucPartitionRef

Maps the Pwm kernel to zero or one ECUC partitions to assign the driver kernel to a certain core.

The ECUC partition referenced is a subset of the ECUC partitions where the Pwm driver is mapped to.

This feature is not supported and is rejected.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.129 Container PwmPowerStateConfig

Each instance of this parameter defines a power state and the callback to be called when this power state is reached.

This feature is not supported and is rejected, as all hardware modules do not support asynchronous power state transitions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.130 Parameter PwmPowerState

Each instance of this parameter describes a different power state supported by the Pwm hardware.

It should be defined by the hardware supplier and used by the Pwm Driver to reference specific HW configurations which set the Pwm HW module in the referenced power state.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	9223372036854775807
min	0

4.1.131 Parameter PwmPowerStateReadyCbRef

Each instance of this parameter contains a reference to a power mode callback defined in a CDD or IoHwAbs component.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.1.132 Container PwmConfigurationOfOptApiServices

Container used for storing the configuration of all optional API's.

Configuration Parameters

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.133 Parameter PwmDeInitApi

Add/remove the service `Pwm_DeInit()` from the code.

This option will toggle `PWM_DE_INIT_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	true

4.1.134 Parameter PwmGetOutputState

Add/remove the service `Pwm_GetOutputState()` from the code.

This option will toggle `PWM_GET_OUTPUT_STATE_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.135 Parameter PwmSetDutyCycle

Add/remove the service `Pwm_SetDutyCycle()` from the code.

This option will toggle `PWM_SET_DUTY_CYCLE_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.136 Parameter PwmSetOutputToIdle

Add/remove the service `Pwm_SetOutputToIdle()` from the code.

This option will toggle `PWM_SET_OUTPUT_TO_IDLE_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.137 Parameter PwmSetPeriodAndDuty

Add/remove the service `Pwm_SetPeriodAndDuty()` from the code.

This option will toggle `PWM_SET_PERIOD_AND_DUTY_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.138 Parameter PwmVersionInfoApi

Add/remove the service `Pwm_GetVersionInfo()` from the code.

This option will toggle `PWM_VERSION_INFO_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.139 Parameter PwmGetChannelStateApi

Add/remove the service `Pwm_GetChannelState()` from the code.

This option will toggle `PWM_GET_CHANNEL_STATE_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.140 Parameter PwmSetDutyCycle_NoUpdate

Add/remove the service `Pwm_SetDutyCycle_NoUpdate()` from the code.

This option will toggle `PWM_SET_DUTY_CYCLE_NO_UPDATE_API` define macro in `Pwm_Cfg.h` file.

This option is activated only when `PwmGeneral/PwmMultiChannelSync` is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.141 Parameter PwmSetPeriodAndDuty_NoUpdate

Add/remove the service `Pwm_SetPeriodAndDuty_NoUpdate()` from the code.

This option will toggle `PWM_SET_PERIOD_AND_DUTY_NO_UPDATE_API` define macro in `Pwm_Cfg.h` file.

This option is activated only when `PwmGeneral/PwmMultiChannelSync` is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.142 Parameter PwmSetDutyPhaseShift

Add/remove the service `Pwm_SetDutyPhaseShift()` from the code.

This option will toggle `PWM_SET_DUTY_PHASE_SHIFT_API` define macro in `Pwm_Cfg.h` file.

This option is activated only when `PwmGeneral/PwmMultiChannelSync` is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.143 Parameter PwmSetChannelDeadTime

Add/remove the service `Pwm_SetChannelDeadTime()` from the code.

This option will toggle `PWM_SETCHANNELDEADTIME_API` define macro.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.144 Parameter PwmForceOutputToZeroApi

Add/remove the service `Pwm_ForceOutputToZero()` from the code.

This option will toggle `PWM_FORCE_OUTPUT_TO_ZERO_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.145 Parameter PwmEnableTrigger

Add/remove the service `Pwm_EnableTrigger()` from the code.

This option will toggle `PWM_ENABLE_TRIGGER_API` define macro in `Pwm_Cfg.h` file.

This option is activated only when `PwmGeneral/PwmEnableExternalTrigger` is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.146 Parameter PwmDisableTrigger

Add/remove the service `Pwm_DisableTrigger()` from the code.

This option will toggle `PWM_DISABLE_TRIGGER_API` define macro in `Pwm_Cfg.h` file.

This option is activated only when `PwmGeneral/PwmEnableExternalTrigger` is enabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.147 Parameter PwmResetCounter

Add/remove the service `Pwm_ResetCounterEnable()` and `Pwm_ResetCounterDisable()` from the code.

This option will toggle `PWM_RESET_COUNTER_API` define macro in `Pwm_Cfg.h` file.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.148 Parameter PwmFastUpdateApi

Add/remove the service `Pwm_FastUpdateApi()` from the code.

This function may be used to update the duty cycle of FTM based channels with minimum overhead.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.149 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.150 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.151 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.152 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.153 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	121
max	121
min	121

4.1.154 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.155 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.156 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION

Configuration Parameters

Property	Value
defaultValue	0
max	0
min	0

4.1.157 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.158 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43

Property	Value
max	43
min	43

4.2 FTM_PWM

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Ftm_Pwm](#)
 - Container [PwmGeneral](#)
 - * Parameter [PwmDevErrorDetect](#)
 - * Parameter [PwmEnableUserModeSupport](#)
 - Container [PwmHwResourceConfig](#)
 - Container [PwmFtm](#)

4.2.1 Module Ftm_Pwm

Configuration of PWM (Pulse Width Modulation) module

Included containers:

- [PwmGeneral](#)
- [PwmHwResourceConfig](#)
- [PwmFtm](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3 FLEXIO_PWM

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Flexio_Pwm](#)
 - Container [PwmGeneral](#)
 - * Parameter [PwmDevErrorDetect](#)
 - * Parameter [PwmEnableUserModeSupport](#)
 - Container [PwmFlexio](#)
 - Container [PwmHwResourceConfig](#)

4.3.1 Module Flexio_Pwm

Configuration of PWM (Pulse Width Modulation) module

Included containers:

- [PwmGeneral](#)
- [PwmFlexio](#)
- [PwmHwResourceConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

FlexIO Pwm IPL	100
Ftm Pwm IPL	107
Pwm Driver	133

Chapter 6

Module Documentation

6.1 FlexIO Pwm IPL

6.1.1 Detailed Description

Data Structures

- struct [Flexio_Pwm_Ip_IplNotificationType](#)
Structure for notification. [More...](#)
- struct [Flexio_Pwm_Ip_HldNotificationType](#)
Structure for notification. [More...](#)
- struct [Flexio_Pwm_Ip_ChannelConfigType](#)
PWM configuration parameters structure. [More...](#)

Types Reference

- typedef void(* [Flexio_Pwm_Ip_IplCallbackType](#)) ([Flexio_Pwm_Ip_OccurredIRQType](#) IrqType, void *Param)
Notification function callback type for IPL.
- typedef void(* [Flexio_Pwm_Ip_HldCallbackType](#)) (uint8 Channel)
Notification function callback type for HLD.

Enum Reference

- enum [Flexio_Pwm_Ip_StatusType](#)
Status return codes.
- enum [Flexio_Pwm_Ip_InterruptType](#)
Interrupt types enabled for the channel.
- enum [Flexio_Pwm_Ip_OccurredIRQType](#)
Interrupt types enabled for the channel.

Function Reference

- [Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_InitChannel](#) (uint8 InstanceId, const [Flexio_Pwm_Ip_ChannelConfigType](#) *const UserCfg)
Initialize a flexio channel in pwm mode.

- [Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_DeInitChannel](#) (uint8 InstanceId, uint8 Channel)
Deinitialize a flexio channel.
- [Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_UpdatePeriodDuty](#) (uint8 InstanceId, uint8 Channel, uint16 Period, uint16 DutyCycle)
Set a new value for duty cycle and period of the channel.
- boolean [Flexio_Pwm_Ip_GetOutputState](#) (uint8 InstanceId, uint8 Channel)
Get the logic level of the channel output.
- [Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_UpdateInterruptMode](#) (uint8 InstanceId, uint8 Channel, [Flexio_Pwm_Ip_InterruptType](#) IrqMode)
Update the interrupt mode for a channel.
- uint16 [Flexio_Pwm_Ip_GetPeriod](#) (uint8 InstanceId, uint8 Channel)
Getting the period for a channel.

Variables

- const [Flexio_Pwm_Ip_ChannelConfigType](#) * [Flexio_Pwm_Ip_aState](#) [(FLEXIO_TIMCMP_COUNT)]
Array with pointers to user cfg structs.
- volatile uint8 [Flexio_Pwm_Ip_u8TimerIrqMask](#)
Interrupt enable masks for used timers.

6.1.2 Data Structure Documentation

6.1.2.1 struct Flexio_Pwm_Ip_IplNotificationType

Structure for notification.

The structure used to notification

Definition at line 203 of file Flexio_Pwm_Ip_Types.h.

Data Fields

Flexio_Pwm_Ip_IplCallbackType	CbFunction	Callback function pointer.
void *	CbParameter	Callback function parameter pointer.

6.1.2.2 struct Flexio_Pwm_Ip_HldNotificationType

Structure for notification.

The structure used to notification

Definition at line 217 of file Flexio_Pwm_Ip_Types.h.

Data Fields

Flexio_Pwm_Ip_HldCallbackType	CbFunction	Callback function pointer.
uint8	CbParameter	Callback function parameter pointer.

6.1.2.3 struct Flexio_Pwm_Ip_ChannelConfigType

PWM configuration parameters structure.

Flexio IP specific channel configuration structure type

Definition at line 231 of file Flexio_Pwm_Ip_Types.h.

Data Fields

uint8	TimerId	Flexio used timer index.
uint8	PinId	Flexio used pin index.
uint16	Period	Pwm period in ticks.
uint16	DutyCycle	Pwm duty cycle in ticks.
Flexio_Pwm_Ip_IplNotificationType	IplCallback	User notification callback for IPL.
Flexio_Pwm_Ip_HldNotificationType	HldCallback	User notification callback for HLD.

6.1.3 Types Reference

6.1.3.1 Flexio_Pwm_Ip_IplCallbackType

```
typedef void(* Flexio_Pwm_Ip_IplCallbackType) (Flexio\_Pwm\_Ip\_OccurredIRQType IrqType, void *Param)
```

Notification function callback type for IPL.

Definition at line 189 of file Flexio_Pwm_Ip_Types.h.

6.1.3.2 Flexio_Pwm_Ip_HldCallbackType

```
typedef void(* Flexio_Pwm_Ip_HldCallbackType) (uint8 Channel)
```

Notification function callback type for HLD.

Definition at line 195 of file Flexio_Pwm_Ip_Types.h.

6.1.4 Enum Reference

6.1.4.1 Flexio_Pwm_Ip_StatusType

```
enum Flexio\_Pwm\_Ip\_StatusType
```

Status return codes.

Common error codes will be a unified enumeration (C enum) that will contain all error codes (common and specific). There will be separate "error values spaces" (or slots), each of 256 positions, allocated per functionality.

Enumerator

FLEXIO_PWM_IP_STATUS_SUCCESS	Generic operation success status.
FLEXIO_PWM_IP_STATUS_ERROR	Generic operation failure status.
FLEXIO_PWM_IP_STATUS_BUSY	Generic operation busy status.

Enumerator

FLEXIO_PWM_IP_STATUS_TIMEOUT	Generic operation timeout status.
FLEXIO_PWM_IP_STATUS_UNSUPPORTED	Generic operation unsupported status.
FLEXIO_PWM_IP_WRONG_TIMER_PIN	FLEXIO unsuccessful attempt at acquiring the timer and pin.
FLEXIO_PWM_IP_WRONG_PERIOD_DUTY	FLEXIO wrong period for given duty cycle.

Definition at line 110 of file Flexio_Pwm_Ip_Types.h.

6.1.4.2 Flexio_Pwm_Ip_InterruptType

enum `Flexio_Pwm_Ip_InterruptType`

Interrupt types enabled for the channel.

This enumeration specifies interrupt type of Flexio

Enumerator

FLEXIO_PWM_IP_IRQ_DISABLED	Interrupt requests are disabled.
FLEXIO_PWM_IP_IRQ_ON_RISING_EDGE	Interrupt requests are generated on pin rising edge.
FLEXIO_PWM_IP_IRQ_ON_FALLING_EDGE	Interrupt requests are generated on pin falling edge.
FLEXIO_PWM_IP_IRQ_ON_BOTH_EDGES	Interrupt requests are generated on pin both edge.
FLEXIO_PWM_IP_IRQ_ON_PERIOD_END	Interrupt requests are generated on timer period end.

Definition at line 151 of file Flexio_Pwm_Ip_Types.h.

6.1.4.3 Flexio_Pwm_Ip_OccurredIRQType

enum `Flexio_Pwm_Ip_OccurredIRQType`

Interrupt types enabled for the channel.

This enumeration specifies interrupt type of Flexio

Enumerator

FLEXIO_PWM_IP_TIMER_IRQ	Timer flag generated the interrupt.
FLEXIO_PWM_IP_PIN_RISING_EDGE_IRQ	Pin rising edge flag generated the interrupt.
FLEXIO_PWM_IP_PIN_FALLING_EDGE_IRQ	Pin rising edge generated the interrupt.

Definition at line 171 of file Flexio_Pwm_Ip_Types.h.

6.1.5 Function Reference

6.1.5.1 Flexio_Pwm_Ip_InitChannel()

```
Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_InitChannel (
    uint8 InstanceId,
    const Flexio_Pwm_Ip_ChannelConfigType *const UserCfg )
```

Initialize a flexio channel in pwm mode.

The function will initialize one timer and pin of the selected flexio channel in pwm mode, with the configuration of the user. The interrupts will be disabled.

Parameters

in	<i>InstanceId</i>	The Flexio instance id
in	<i>UserCfg</i>	The channel configuration for the selected Flexio instance

Returns

FLEXIO_PWM_IP_STATUS_SUCCESS - if the initialization was successful

6.1.5.2 Flexio_Pwm_Ip_DeInitChannel()

```
Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_DeInitChannel (
    uint8 InstanceId,
    uint8 Channel )
```

Deinitialize a flexio channel.

The function will reset the timer and pin of the selected flexio channel.

Parameters

in	<i>InstanceId</i>	The Flexio instance id
in	<i>Channel</i>	The channel id for the selected Flexio instance

Returns

FLEXIO_PWM_IP_STATUS_SUCCESS - if the deinitialization was successful

6.1.5.3 Flexio_Pwm_Ip_UpdatePeriodDuty()

```
Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_UpdatePeriodDuty (
    uint8 InstanceId,
    uint8 Channel,
    uint16 Period,
    uint16 DutyCycle )
```

Set a new value for duty cycle and period of the channel.

The function will update the selected flexio channel with the new values for the duty cycle and period.

Parameters

in	<i>Instance↔ Id</i>	The Flexio instance id
in	<i>Channel</i>	The channel id for the selected Flexio instance
in	<i>Period</i>	The new value for the period
in	<i>DutyCycle</i>	The new value for the duty cycle

Returns

FLEXIO_PWM_IP_STATUS_SUCCESS - if the initialization was successfull

6.1.5.4 Flexio_Pwm_Ip_GetOutputState()

```
boolean Flexio_Pwm_Ip_GetOutputState (
    uint8 InstanceId,
    uint8 Channel )
```

Get the logic level of the channel ouput.

The function will return the logic level that the selected flexio channel is driving on on the output pin.

Parameters

in	<i>Instance↔ Id</i>	The Flexio instance id
in	<i>Channel</i>	The channel id for the selected Flexio instance

Returns

FLEXIO_PWM_IP_STATUS_SUCCESS - if the initialization was successfull

6.1.5.5 Flexio_Pwm_Ip_UpdateInterruptMode()

```
Flexio_Pwm_Ip_StatusType Flexio_Pwm_Ip_UpdateInterruptMode (
    uint8 InstanceId,
    uint8 Channel,
    Flexio_Pwm_Ip_InterruptType IrqMode )
```

Update the interrupt mode for a channel.

The function will set a new mode for the flag event response on the selected channel.

Parameters

in	<i>Instance↔ Id</i>	The Flexio instance id
in	<i>Channel</i>	The channel id for the selected Flexio instance
in	<i>IrqMode</i>	The new irq mode for the channel

Returns

FLEXIO_PWM_IP_STATUS_SUCCESS - if the initialization was successfull

6.1.5.6 Flexio_Pwm_Ip_GetPeriod()

```
uint16 Flexio_Pwm_Ip_GetPeriod (
    uint8 InstanceId,
    uint8 Channel )
```

Getting the period for a channel.
The function will get the period on the selected channel.

Parameters

in	<i>InstanceId</i>	The Flexio instance id
in	<i>Channel</i>	The channel id for the selected Flexio instance

Returns

uint16

6.1.6 Variable Documentation

6.1.6.1 Flexio_Pwm_Ip_aState

```
const Flexio_Pwm_Ip_ChannelConfigType* Flexio_Pwm_Ip_aState[ (FLEXIO_TIMCMP_COUNT) ] [extern]
```

Array with pointers to user cfg structs.

6.1.6.2 Flexio_Pwm_Ip_u8TimerIrqMask

```
volatile uint8 Flexio_Pwm_Ip_u8TimerIrqMask [extern]
```

Interrupt enable masks for used timers.

6.2 Ftm Pwm IPL

6.2.1 Detailed Description

Data Structures

- struct [Ftm_Pwm_Ip_CallbackCfgType](#)
Structure for notification. [More...](#)
- struct [Ftm_Pwm_Ip_SyncCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_InstanceCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_PairCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_ChannelConfigType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_FaultChCfgType](#)
Ftm configuration parameters structure. [More...](#)
- struct [Ftm_Pwm_Ip_UserCfgType](#)
Ftm configuration parameters structure. [More...](#)

Macros

- `#define FTM\_PWM\_IP\_MAX\_DUTY\_CYCLE`
Maximum value for PWM duty cycle.
- `#define FTM\_PWM\_IP\_NO\_EDGE`
FlexTimer edge interrupt.

Types Reference

- typedef void(* [Ftm_Pwm_Ip_CallbackType](#)) (uint8 CbParam)
Notification function callback type.
- typedef void(* [Ftm_Pwm_Ip_FaultCallbackType](#)) (void)
Fault channel notification typedef.

Enum Reference

- enum [Ftm_Pwm_Ip_StatusType](#)
Status return codes.
- enum [Ftm_Pwm_Ip_ChannelModeType](#)
FlexTimer operation mode.
- enum [Ftm_Pwm_Ip_ChannelStateType](#)
FlexTimer channel state.
- enum [Ftm_Pwm_Ip_PrescalerType](#)
Prescaler type.
- enum [Ftm_Pwm_Ip_PowerStateType](#)

Power state type.

- enum [Ftm_Pwm_Ip_NotifType](#)

Notification type.

- enum [Ftm_Pwm_Ip_CounterModeType](#)

FlexTimer operation mode.

- enum [Ftm_Pwm_Ip_ClkSourceType](#)

FlexTimer clock source selection.

- enum [Ftm_Pwm_Ip_PolarityType](#)

The polarity of the channel output is configured in PWM signal.

- enum [Ftm_Pwm_Ip_OutputStateType](#)

The initial state of the channel output is configured in PWM signal.

- enum [Ftm_Pwm_Ip_ClkPrescalerType](#)

FlexTimer pre-scaler factor selection for the clock source.

- enum [Ftm_Pwm_Ip_SyncType](#)

FTM sync source.

- enum [Ftm_Pwm_Ip_SyncModeType](#)

FTM sync mode.

- enum [Ftm_Pwm_Ip_DeadtimePrescalerType](#)

FlexTimer pre-scaler factor for the dead-time insertion.

- enum [Ftm_Pwm_Ip_DebugModeType](#)

Options for the FlexTimer behavior in BDM Mode.

- enum [Ftm_Pwm_Ip_InitTriggModeType](#)

FlexTimer PWM initialization trigger generation mode.

- enum [Ftm_Pwm_Ip_PairChPolarityType](#)

FlexTimer PWM channel (n+1) polarity for combine mode.

- enum [Ftm_Pwm_Ip_PairDitherEdgeType](#)

FlexTimer PWM Pair Edge Dithering for combine/mcombine modes.

- enum [Ftm_Pwm_Ip_FaultControlModeType](#)

Defines the FTM fault control mode.

- enum [Ftm_Pwm_Ip_FaultOutputStateType](#)

FlexTimer PWM Fault output state.

Function Reference

- void [Ftm_Pwm_Ip_Init](#) (uint8 Instance, const [Ftm_Pwm_Ip_UserCfgType](#) *UserCfg)

Initializes the FTM driver. The clock sources depend on the specified hardware. So that needs to check the external clock or the fixed frequency clock is available on device or not.

- void [Ftm_Pwm_Ip_DeInit](#) (uint8 Instance)

Shuts down the FTM driver.

- [Ftm_Pwm_Ip_StatusType](#) [Ftm_Pwm_Ip_UpdatePwmChannel](#) (uint8 Instance, uint8 Channel, uint16 FirstEdge, uint16 SecondEdge, boolean SoftwareTrigger)

This function updates the waveform output in PWM mode (duty cycle and phase).

- void [Ftm_Pwm_Ip_FastUpdatePwmDuty](#) (uint8 Instance, uint8 NumberOfChannels, const uint8 *Channels, const uint16 *Duty, boolean SoftwareTrigger)
This function will update the duty cycle of PWM output for multiple channels.
- [Ftm_Pwm_Ip_StatusType](#) [Ftm_Pwm_Ip_UpdatePwmPeriod](#) (uint8 Instance, uint32 NewValue, boolean SoftwareTrigger)
This function will update the new period in the frequency or in the counter value into mode register which modify the period of PWM signal on the Channel output.
- void [Ftm_Pwm_Ip_SwOutputControl](#) (uint8 Instance, uint8 Channel, [Ftm_Pwm_Ip_OutputStateType](#) OutputState, boolean ActiveState)
This function performs the update of the SWOCTRL register on request to set/clear output state of channel in idle state or output-forced.
- void [Ftm_Pwm_Ip_SyncUpdate](#) (uint8 Instance)
Ftm_Pwm_Ip_SyncUpdate.
- void [Ftm_Pwm_Ip_DisableNotification](#) (uint8 Instance, [Ftm_Pwm_Ip_NotifType](#) NotifType)
This function disables the user notifications.
- void [Ftm_Pwm_Ip_EnableNotification](#) (uint8 Instance, [Ftm_Pwm_Ip_NotifType](#) NotifType)
This function enables the user notifications.
- void [Ftm_Pwm_Ip_SetClockMode](#) (uint8 Instance, [Ftm_Pwm_Ip_PrescalerType](#) Prescaler)
This function sets the value of the prescaler on all FTM modules.
- void [Ftm_Pwm_Ip_ResetCounter](#) (uint8 Instance, boolean IsResetCnt)
This function enabel/disabled counter reset by [Pwm_SyncUpdate\(\)](#) call.
- void [Ftm_Pwm_Ip_MaskOutputChannels](#) (uint8 Instance, uint32 ChannelsMask, boolean SoftwareTrigger)
This function will mask the output of the channels and at match events will be ignored by the masked channels.
- void [Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel](#) (uint8 Instance, uint8 Channel, uint16 DutyCycle, boolean SoftwareTrigger)
This function sets the duty cycle for the specified PWM channel.
- void [Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty](#) (uint8 Instance, uint8 Channel, uint16 Period, uint16 Duty↵Cycle, boolean SoftwareTrigger)
This function sets the duty cycle and period for the specified PWM channel.
- [Ftm_Pwm_Ip_OutputStateType](#) [Ftm_Pwm_Ip_GetOutputState](#) (uint8 Instance, uint8 Channel)
This function used to get the current output state of the PWM channel.
- void [Ftm_Pwm_Ip_SetPowerState](#) (uint8 Instance, [Ftm_Pwm_Ip_PowerStateType](#) PowerState)
Put PWM channel based on FTM to target power mode.
- uint16 [Ftm_Pwm_Ip_GetChannelState](#) (uint8 Instance, uint8 Channel)
This function read the duty cycle value from the duty cycle array and returns this value.
- void [Ftm_Pwm_Ip_SetPhaseShift](#) (uint8 Instance, uint8 Channel, uint16 Period, uint16 PhaseShift, boolean SoftwareTrigger)
This function set phase shift value and also force duty cycle to 50%.
- void [Ftm_Pwm_Ip_SetDutyPhaseShift](#) (uint8 Instance, uint8 Channel, uint16 DutyCycle, uint16 PhaseShift, boolean SyncUpdate)
This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)
- void [Ftm_Pwm_Ip_UnMaskOutputChannels](#) (uint8 Instance, uint32 ChannelsMask, boolean Software↵Trigger)

This function will unmask the output of the channels and at match events will be ignored by the masked channels.

- void [Ftm_Pwm_Ip_EnableTrigger](#) (uint8 Instance, uint32 TriggerMask)

This function enables trigger generation for specific source.

- void [Ftm_Pwm_Ip_DisableTrigger](#) (uint8 Instance, uint32 TriggerMask)

This function disables trigger generation for specific source.

- void [Ftm_Pwm_Ip_SetChannelDeadTime](#) (uint8 Instance, uint8 Channel, uint16 DeadTimeVal)

This function is used to update the deadtime at runtime for Pwm channels.

Variables

- [Ftm_Pwm_Ip_FaultCallbackType](#) [Ftm_Pwm_Ip_pFaultIrqCallback](#) [(FTM_INSTANCE_COUNT)][(4U)]

Arrays for irq fault notifications.

- [Ftm_Pwm_Ip_ReloadCallbackCfgType](#) [Ftm_Pwm_Ip_pReloadIrqCallback](#) [((FTM_INSTANCE_COUNT))[(FTM_CONTROLS_COUNT)]]

Arrays for irq reload notifications of each Instance.

- [Ftm_Pwm_Ip_ChannelStateType](#) [Ftm_Pwm_Ip_aInstanceState](#) [((FTM_INSTANCE_COUNT))]

Arrays to store the state of each instance.

- [Ftm_Pwm_Ip_ChannelStateType](#) [Ftm_Pwm_Ip_aPreviousChannelState](#) [((FTM_INSTANCE_COUNT))[(FTM_CONTROLS_COUNT)]]

Arrays to store the previous state of each channel.

- [Ftm_Pwm_Ip_CallbackCfgType](#) [Ftm_Pwm_Ip_pOverflowIrqCallback](#) [((FTM_INSTANCE_COUNT))]

Arrays for irq overflow timer notifications of each Instance.

- [Ftm_Pwm_Ip_ChannelStateType](#) [Ftm_Pwm_Ip_aChannelState](#) [((FTM_INSTANCE_COUNT))[(FTM_CONTROLS_COUNT)]]

Arrays to store the state of channel.

- [Ftm_Pwm_Ip_EdgeInterruptType](#) [Ftm_Pwm_Ip_aNotifIrq](#) [((FTM_INSTANCE_COUNT))[(FTM_CONTROLS_COUNT)]]

Arrays to store the type of notification for each channel.

- uint16 [Ftm_Pwm_Ip_aPeriod](#) [((FTM_INSTANCE_COUNT))]

Arrays to store the period values for each Instance.

6.2.2 Data Structure Documentation

6.2.2.1 struct [Ftm_Pwm_Ip_CallbackCfgType](#)

Structure for notification.

The structure used to notification

Definition at line 345 of file [Ftm_Pwm_Ip_Types.h](#).

6.2.2.2 struct [Ftm_Pwm_Ip_SyncCfgType](#)

Ftm configuration parameters structure.

FlexTimer Sync and Load parameters

Definition at line 374 of file [Ftm_Pwm_Ip_Types.h](#).

Data Fields

Ftm_Pwm_Ip_SyncModeType	SyncMode	Configure FTM Sync to use Software or Hardware trigger modes
boolean	HwSync0	Enable/disable hardware 0 sync
boolean	HwSync1	Enable/disable hardware 1 sync
boolean	HwSync2	Enable/disable hardware 2 sync
boolean	HwTriggerAutoClear	Enable/disable auto clear of triggers in hardware trigger mode
boolean	MaxLoadPoint	Enable/disable maximum loading point
boolean	MinLoadPoint	Enable/disable minimum loading point
boolean	HalfCycleLoadPoint	Enable/disable half cycle loading point
uint16	HalfCycleLoadPointValue	The match value of FTM half cycle reload feature
uint8	LoadPointFreq	Value of how many reload points must happen before reloading.
Ftm_Pwm_Ip_SyncType	InverterSync	Configures INVCTRL sync
Ftm_Pwm_Ip_SyncType	OutRegSync	Configures SWOCTRL sync
Ftm_Pwm_Ip_SyncType	OutMaskSync	Configures OUTMASK sync
Ftm_Pwm_Ip_SyncType	InitCounterSync	Configures CNTIN sync
Ftm_Pwm_Ip_SyncType	CounterSync	Configures FTM Counter sync

6.2.2.3 struct Ftm_Pwm_Ip_InstanceCfgType

Ftm configuration parameters structure.

Ftm IP specific instance configuration structure type

Definition at line 402 of file Ftm_Pwm_Ip_Types.h.

Data Fields

Ftm_Pwm_Ip_ClkSourceType	ClkSrc	Clock source for Ftm instances
Ftm_Pwm_Ip_ClkPrescalerType	ClkPs	Pre-scaler for Ftm instances
Ftm_Pwm_Ip_ClkPrescalerType	AlternateClkPs	Alternative pre-scaler for Ftm instances
Ftm_Pwm_Ip_CounterModeType	CntMode	Mode of the counter
boolean	OverflowIrqEn	Selected the overflow interrupt
Ftm_Pwm_Ip_CallbackCfgType	OverflowCb	overflow callback
boolean	ReloadIrqEn	Selected the reload interrupt
Ftm_Pwm_Ip_ReloadCallbackCfgType	ReloadCb	Reload callback
Ftm_Pwm_Ip_DebugModeType	DebugMode	Debug Mode
boolean	WriteProtection	Selected write protection
boolean	InitTriggerEn	Selected trigger
Ftm_Pwm_Ip_InitTriggModeType	InitTrigMode	Chose generation of initialization trigger mode
uint16	PwmPeriod	Period of Ftm instances
uint8	PwmPeriodDither	Period Dither

Data Fields

uint16	DeadTimeValue	Deadtime Value
Ftm_Pwm_Ip_DeadtimePrescalerType	DeadTimePs	Deadtime pre-scaler Value
Ftm_Pwm_Ip_FaultControlModeType	FaultCtrMode	Defines the FTM fault control mode
Ftm_Pwm_Ip_FaultOutputStateType	FaultOutState	Fault output state
uint8	FaultInFilter	Fault Input Filter Value
boolean	FaultIrqEn	Fault interrupt enable
boolean	ObeCtrl	Selected the FTM channel output state
const Ftm_Pwm_Ip_SyncCfgType *	SyncCfg	Pointer to the configured channels for the FTM sync

6.2.2.4 struct Ftm_Pwm_Ip_PairCfgType

Ftm configuration parameters structure.

FlexTimer Channel Pair configuration parameters

Definition at line 441 of file Ftm_Pwm_Ip_Types.h.

Data Fields

uint8	PairId	FTM pair id channel.
boolean	DeadtimeEn	Selected dead-time for pair channel.
boolean	ComplementaryModeEn	Selected Complementary mode for pair channel.
Ftm_Pwm_Ip_PairChPolarityType	PairChPolarity	Polarity of pair channel.
boolean	PairExtTrigEn	Selected external trigger of pair channel.
boolean	PairSyncEn	Selected sync of pair channel.
uint16	PhaseShiftValue	the phase Shift Value
Ftm_Pwm_Ip_PairDitherEdgeType	DitherEdge	

6.2.2.5 struct Ftm_Pwm_Ip_ChannelConfigType

Ftm configuration parameters structure.

FlexTimer Channel configuration parameters

Definition at line 467 of file Ftm_Pwm_Ip_Types.h.

Data Fields

uint8	ChannelId	FTM channel ID.
Ftm_Pwm_Ip_ChannelModeType	ChannelMode	Mode of Ftm channel.
boolean	ChIrqEn	Selected for interrupt.
Ftm_Pwm_Ip_CallbackCfgType	ChannelCb	Callback for the ftm channels.
boolean	ChOutputEn	Enable/Disable the output of the PWM signal on the channel pin.
boolean	SwControlEn	Enable/Disable the software control for this channels output.

Data Fields

	boolean	SwControlVal	Channel Software Output Control Value.
	boolean	ExtTrigEn	Selected for external trigger.
	boolean	ChMatchLoadEn	Selected for external trigger.
	uint16	DutyCycle	Duty cycle value.
	uint8	DutyCycleDither	Duty cycle value in Dither.
	Ftm_Pwm_Ip_OutputStateType	InitOut	Output state of Ftm channel.
	Ftm_Pwm_Ip_PolarityType	Polarity	Polarity of Ftm channel.
	const Ftm_Pwm_Ip_PairCfgType *	PairCfg	The configuration of pair channel.

6.2.2.6 struct Ftm_Pwm_Ip_FaultChCfgType

Ftm configuration parameters structure.

FlexTimer fault configuration parameters

Definition at line 514 of file Ftm_Pwm_Ip_Types.h.

Data Fields

	uint8	FaultId	FTM fault ID is actived.
	Ftm_Pwm_Ip_PolarityType	FaultPol	Defines the polarity of the fault input.
	boolean	FaultFilEn	Fault Input Filter Enable
	Ftm_Pwm_Ip_FaultCallbackType	FaultCb	Pointer to fault notification function.

6.2.2.7 struct Ftm_Pwm_Ip_UserCfgType

Ftm configuration parameters structure.

FlexTimer User Configuration structure

Definition at line 532 of file Ftm_Pwm_Ip_Types.h.

Data Fields

const Ftm_Pwm_Ip_InstanceCfgType *	InstanceCfg	Pointer to the configured channels for the FTM instance.
const Ftm_Pwm_Ip_ChannelConfigType *const *	ConfiguredChArray	Pointer to the configured channels array for the FTM channel.
uint8	NoOfConfiguredCh	Number of the configured channels for the FTM channel.
const Ftm_Pwm_Ip_FaultChCfgType *const *	ConfiguredFaultArray	Pointer to the configured channels array for the FTM channel.
uint8	NoOfFaultCfg	Number of the configured channels for fault.

6.2.3 Macro Definition Documentation

6.2.3.1 FTM_PWM_IP_MAX_DUTY_CYCLE

```
#define FTM_PWM_IP_MAX_DUTY_CYCLE
```

Maximum value for PWM duty cycle.

Definition at line 121 of file Ftm_Pwm_Ip.h.

6.2.3.2 FTM_PWM_IP_NO_EDGE

```
#define FTM_PWM_IP_NO_EDGE
```

FlexTimer edge interrupt.

Definition at line 106 of file Ftm_Pwm_Ip_Types.h.

6.2.4 Types Reference

6.2.4.1 Ftm_Pwm_Ip_CallbackType

```
typedef void(* Ftm_Pwm_Ip_CallbackType) (uint8 CbParam)
```

Notification function callback type.

Definition at line 338 of file Ftm_Pwm_Ip_Types.h.

6.2.4.2 Ftm_Pwm_Ip_FaultCallbackType

```
typedef void(* Ftm_Pwm_Ip_FaultCallbackType) (void)
```

Fault channel notification typedef.

Definition at line 507 of file Ftm_Pwm_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Ftm_Pwm_Ip_StatusType

```
enum Ftm_Pwm_Ip_StatusType
```

Status return codes.

Enumerator

FTM_PWM_IP_STATUS_SUCCESS	Generic operation success status
FTM_PWM_IP_STATUS_ERROR	Generic operation failure status
FTM_PWM_IP_STATUS_BUSY	Generic operation busy status
FTM_PWM_IP_STATUS_TIMEOUT	Generic operation timeout status
FTM_PWM_IP_STATUS_UNSUPPORTED	Generic operation unsupported status

Definition at line 113 of file Ftm_Pwm_Ip_Types.h.

6.2.5.2 Ftm_Pwm_Ip_ChannelModeType

enum `Ftm_Pwm_Ip_ChannelModeType`

FlexTimer operation mode.

Enumerator

FTM_PWM_IP_MODE_EDGE_ALIGNED_HIGH	Edge aligned PWM High-true pulses (clear on match)
FTM_PWM_IP_MODE_EDGE_ALIGNED_LOW	Edge aligned PWM Low-true pulses (set on match)
FTM_PWM_IP_MODE_CENTER_ALIGNED_↔ HIGH	Center aligned PWM High-true pulses (clear on match)
FTM_PWM_IP_MODE_CENTER_ALIGNED_↔ LOW	Center aligned PWM Low-true pulses (set on match)
FTM_PWM_IP_MODE_COMBINE_HIGH	Combined channels PWM High-true pulses (set on channel (n) match, and clear on channel (n+1) match)
FTM_PWM_IP_MODE_COMBINE_LOW	Combined channels PWM Low-true pulses (clear on channel (n) match, and set on channel (n+1) match)
FTM_PWM_IP_MODE_MODIFIED_COMBIN↔ E_HIGH	Modified combined channel PWM High-true pulses (set on channel (n) match, and clear on channel (n+1) match)
FTM_PWM_IP_MODE_MODIFIED_COMBIN↔ E_LOW	Modified combined channel PWM Low-true pulses (clear on channel (n) match, and set on channel (n+1) match)
FTM_PWM_IP_MODE_NODEFINED	Non defined Mode

Definition at line 124 of file Ftm_Pwm_Ip_Types.h.

6.2.5.3 Ftm_Pwm_Ip_ChannelStateType

enum `Ftm_Pwm_Ip_ChannelStateType`

FlexTimer channel state.

Enumerator

FTM_PWM_IP_CHANNEL_UNINIT	The state's channel before initialization
FTM_PWM_IP_CHANNEL_RUNNING	The state's channel is running
FTM_PWM_IP_CHANNEL_IDLE	The state's channel is idle
FTM_PWM_IP_CHANNEL_OUTPUT_FORCED	The state's channel in force output channel mode

Definition at line 141 of file Ftm_Pwm_Ip_Types.h.

6.2.5.4 Ftm_Pwm_Ip_PrescalerType

enum `Ftm_Pwm_Ip_PrescalerType`

Prescaler type.

This enumeration specifies the possible types of prescalers used to configure base-clock timers

Enumerator

FTM_PWM_IP_PRIMARY_PRESCALER	Selected value is the default/primary prescaler.
FTM_PWM_IP_ALTERNATIVE_PRESCALER	Selected value is the alternative configured prescaler.

Definition at line 153 of file `Ftm_Pwm_Ip_Types.h`.

6.2.5.5 Ftm_Pwm_Ip_PowerStateType

enum `Ftm_Pwm_Ip_PowerStateType`

Power state type.

Power state currently active or set as target power state.

Enumerator

FTM_PWM_IP_FULL_POWER	PWM full power mode.
FTM_PWM_IP_LOW_POWER	PWM low power mode.
FTM_PWM_IP_NODEFINE_POWER	PWM no define power mode.

Definition at line 166 of file `Ftm_Pwm_Ip_Types.h`.

6.2.5.6 Ftm_Pwm_Ip_NotifType

enum `Ftm_Pwm_Ip_NotifType`

Notification type.

This enumeration defines the type of notification

Enumerator

FTM_PWM_IP_CHANNEL_0_NOTIFICATION	A notification will be generated when an event occurs on the channel 0.
FTM_PWM_IP_CHANNEL_1_NOTIFICATION	A notification will be generated when an event occurs on the channel 1.
FTM_PWM_IP_CHANNEL_2_NOTIFICATION	A notification will be generated when an event occurs on the channel 2.
FTM_PWM_IP_CHANNEL_3_NOTIFICATION	A notification will be generated when an event occurs on the channel 3.
FTM_PWM_IP_CHANNEL_4_NOTIFICATION	A notification will be generated when an event occurs on the channel 4.
FTM_PWM_IP_CHANNEL_5_NOTIFICATION	A notification will be generated when an event occurs on the channel 5.

Enumerator

FTM_PWM_IP_CHANNEL_6_NOTIFICATION	A notification will be generated when an event occurs on the channel 6.
FTM_PWM_IP_CHANNEL_7_NOTIFICATION	A notification will be generated when an event occurs on the channel 7.
FTM_PWM_IP_OVERFLOW_NOTIFICATION	A notification will be generated when the FTM counter passes the value in the MOD register.
FTM_PWM_IP_RELOAD_POINT_NOTIFICATION	A notification will be generated when a selected reload point happened.
FTM_PWM_IP_FAULT_NOTIFICATION	A notification will be generated when an event for the FTM peripheral fault channel has occurred.

Definition at line 181 of file Ftm_Pwm_Ip_Types.h.

6.2.5.7 Ftm_Pwm_Ip_CounterModeType

enum `Ftm_Pwm_Ip_CounterModeType`

FlexTimer operation mode.

Enumerator

FTM_PWM_IP_EDGE_ALIGNED	The FTM Counter is in UP Counting mode
FTM_PWM_IP_CENTER_ALIGNED	The FTM Counter is in UP-DOWN Counting mode

Definition at line 208 of file Ftm_Pwm_Ip_Types.h.

6.2.5.8 Ftm_Pwm_Ip_ClkSourceType

enum `Ftm_Pwm_Ip_ClkSourceType`

FlexTimer clock source selection.

Enumerator

FTM_PWM_IP_CLOCK_SOURCE_NONE	None use clock for FTM
FTM_PWM_IP_CLOCK_SOURCE_SYSTEMCLK	System clock
FTM_PWM_IP_CLOCK_SOURCE_FIXEDCLK	Fixed clock
FTM_PWM_IP_CLOCK_SOURCE_EXTERNALCLK	External clock

Definition at line 215 of file Ftm_Pwm_Ip_Types.h.

6.2.5.9 Ftm_Pwm_Ip_PolarityTypeenum `Ftm_Pwm_Ip_PolarityType`

The polarity of the channel output is configured in PWM signal.

Enumerator

FTM_PWM_IP_POLARITY_LOW	The channel polarity is active LOW
FTM_PWM_IP_POLARITY_HIGH	The channel polarity is active HIGH

Definition at line 228 of file `Ftm_Pwm_Ip_Types.h`.

6.2.5.10 Ftm_Pwm_Ip_OutputStateTypeenum `Ftm_Pwm_Ip_OutputStateType`

The initial state of the channel output is configured in PWM signal.

Enumerator

FTM_PWM_IP_OUTPUT_STATE_LOW	The channel output is LOW
FTM_PWM_IP_OUTPUT_STATE_HIGH	The channel output is HIGH

Definition at line 235 of file `Ftm_Pwm_Ip_Types.h`.

6.2.5.11 Ftm_Pwm_Ip_ClkPrescalerTypeenum `Ftm_Pwm_Ip_ClkPrescalerType`

FlexTimer pre-scaler factor selection for the clock source.

Enumerator

FTM_PWM_IP_CLOCK_DIV_1	Divide by 1
FTM_PWM_IP_CLOCK_DIV_2	Divide by 2
FTM_PWM_IP_CLOCK_DIV_4	Divide by 4
FTM_PWM_IP_CLOCK_DIV_8	Divide by 8
FTM_PWM_IP_CLOCK_DIV_16	Divide by 16
FTM_PWM_IP_CLOCK_DIV_32	Divide by 32
FTM_PWM_IP_CLOCK_DIV_64	Divide by 64
FTM_PWM_IP_CLOCK_DIV_128	Divide by 128

Definition at line 242 of file Ftm_Pwm_Ip_Types.h.

6.2.5.12 Ftm_Pwm_Ip_SyncType

enum [Ftm_Pwm_Ip_SyncType](#)

FTM sync source.

Enumerator

FTM_PWM_IP_SYNC_DISABLED	synchronization for the FTM register is disabled
FTM_PWM_IP_SYNC_ON_CLK	synchronization for the FTM register is done on each FTM Clock rising edge
FTM_PWM_IP_SYNC_ON_TRIGGER	synchronization for the FTM register is done on each FTM Sync Trigger

Definition at line 255 of file Ftm_Pwm_Ip_Types.h.

6.2.5.13 Ftm_Pwm_Ip_SyncModeType

enum [Ftm_Pwm_Ip_SyncModeType](#)

FTM sync mode.

Enumerator

FTM_PWM_IP_SYNC_SWTRIGGER	synchronization trigger for FTM is generated by software
FTM_PWM_IP_SYNC_HWTRIGGER	synchronization trigger for FTM is generated by hardware

Definition at line 263 of file Ftm_Pwm_Ip_Types.h.

6.2.5.14 Ftm_Pwm_Ip_DeadtimePrescalerType

enum [Ftm_Pwm_Ip_DeadtimePrescalerType](#)

FlexTimer pre-scaler factor for the dead-time insertion.

Enumerator

FTM_PWM_IP_DEADTIME_DIV_1	Divide by 1
FTM_PWM_IP_DEADTIME_DIV_4	Divide by 4
FTM_PWM_IP_DEADTIME_DIV_16	Divide by 16

Definition at line 270 of file Ftm_Pwm_Ip_Types.h.

6.2.5.15 Ftm_Pwm_Ip_DebugModeTypeenum `Ftm_Pwm_Ip_DebugModeType`

Options for the FlexTimer behavior in BDM Mode.

Enumerator

FTM_PWM_IP_BDM_MODE_00	FTM counter stopped, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers bypass the register buffers
FTM_PWM_IP_BDM_MODE_01	FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are forced to their safe value , writes to MOD,CNTIN and C(n)V registers bypass the register buffers
FTM_PWM_IP_BDM_MODE_10	FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are frozen when chip enters in BDM mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers
FTM_PWM_IP_BDM_MODE_11	FTM counter in functional mode, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers is in fully functional mode

Definition at line 278 of file `Ftm_Pwm_Ip_Types.h`.**6.2.5.16 Ftm_Pwm_Ip_InitTriggModeType**enum `Ftm_Pwm_Ip_InitTriggModeType`

FlexTimer PWM initialization trigger generation mode.

Enumerator

FTM_PWM_IP_INIT_TRIGG_RELOAD_POINT	Initialization trigger is generated when FTM counter reaches a reload point
FTM_PWM_IP_INIT_TRIGG_CNT_UPDATE	Initialization trigger is generated when FTM counter is updated

Definition at line 295 of file `Ftm_Pwm_Ip_Types.h`.**6.2.5.17 Ftm_Pwm_Ip_PairChPolarityType**enum `Ftm_Pwm_Ip_PairChPolarityType`

FlexTimer PWM channel (n+1) polarity for combine mode.

Enumerator

FTM_PWM_IP_MAIN_INVERTED	The channel (n+1) output is the inverse of the channel (n) output
FTM_PWM_IP_MAIN_DUPLICATED	The channel (n+1) output is the same as the channel (n) output

Definition at line 302 of file Ftm_Pwm_Ip_Types.h.

6.2.5.18 Ftm_Pwm_Ip_PairDitherEdgeType

enum `Ftm_Pwm_Ip_PairDitherEdgeType`

FlexTimer PWM Pair Edge Dithering for combine/mcombine modes.

Enumerator

FTM_PWM_IP_DITHER_LEADING_EDGE	The channel (n) dithering is used
FTM_PWM_IP_DITHER_TRAILING_EDGE	The channel (n+1) dithering is used

Definition at line 309 of file Ftm_Pwm_Ip_Types.h.

6.2.5.19 Ftm_Pwm_Ip_FaultControlModeType

enum `Ftm_Pwm_Ip_FaultControlModeType`

Defines the FTM fault control mode.

Enumerator

FTM_PWM_IP_FAULT_DISABLED	Fault control disabled for all channels.
FTM_PWM_IP_FAULT_EVEN_CHANNELS_↔ MANUAL_CLEAR	Fault control enabled for even channels (0, 2, 4..) with manual fault clearing.
FTM_PWM_IP_FAULT_ALL_CHANNELS_M↔ ANUAL_CLEAR	Fault control enabled for all channels with manual fault clearing.
FTM_PWM_IP_FAULT_ALL_CHANNELS_A↔ UTO_CLEAR	Fault control enabled for all channels with automatic fault clearing.

Definition at line 317 of file Ftm_Pwm_Ip_Types.h.

6.2.5.20 Ftm_Pwm_Ip_FaultOutputStateType

enum `Ftm_Pwm_Ip_FaultOutputStateType`

FlexTimer PWM Fault output state.

Enumerator

FTM_PWM_IP_OUT_SAFE_VALUE	FTM outputs will be placed into safe values
FTM_PWM_IP_OUT_TRI_STATE	FTM outputs will be tri-stated

Definition at line 326 of file Ftm_Pwm_Ip_Types.h.

6.2.6 Function Reference

6.2.6.1 Ftm_Pwm_Ip_Init()

```
void Ftm_Pwm_Ip_Init (
    uint8 Instance,
    const Ftm_Pwm_Ip_UserCfgType * UserCfg )
```

Initializes the FTM driver. The clock sources depend on the specified hardware. So that needs to check the external clock or the fixed frequency clock is available on device or not.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>info</i>	The FTM user configuration structure, see #Ftm_Pwm_Ip_UserConfigType.
out	<i>state</i>	The FTM state structure of the driver.

Returns

void

6.2.6.2 Ftm_Pwm_Ip_DeInit()

```
void Ftm_Pwm_Ip_DeInit (
    uint8 Instance )
```

Shuts down the FTM driver.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
----	-----------------	-------------------------------------

Returns

void

6.2.6.3 Ftm_Pwm_Ip_UpdatePwmChannel()

```
Ftm_Pwm_Ip_StatusType Ftm_Pwm_Ip_UpdatePwmChannel (
    uint8 Instance,
    uint8 Channel,
    uint16 FirstEdge,
    uint16 SecondEdge,
    boolean SoftwareTrigger )
```

This function updates the waveform output in PWM mode (duty cycle and phase).

@Note: Regarding the type of updating PWM in the duty cycle, if the expected duty is 100% then the value that is to be written to hardware will be exceed value of period. It means that the FTM counter will not match the value of the CV register in this case.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The channel number. In combined mode, the code finds the channel.
in	<i>FirstEdge</i>	Duty cycle or first edge time for PWM mode. Can take value between 0 - FTM_PWM_IP_MAX_DUTY_CYCLE(0 = 0% from period and FTM_PWM_IP_MAX_DUTY_CYCLE = 100% from period) Or value in ticks for the first of the PWM mode in which can have value between 0 and ftmPeriod is stored in the state structure.
in	<i>SecondEdge</i>	Second edge time - only for combined mode. Can take value between 0 - FTM_PWM_IP_MAX_DUTY_CYCLE(0 = 0% from period and FTM_PWM_IP_MAX_DUTY_CYCLE = 100% from period). Or value in ticks for the second of the PWM mode in which can have value between 0 and ftmPeriod is stored in the state structure.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.
- FTM_PWM_STATUS_ERROR : Error occurred.

6.2.6.4 Ftm_Pwm_Ip_FastUpdatePwmDuty()

```
void Ftm_Pwm_Ip_FastUpdatePwmDuty (
    uint8 Instance,
    uint8 NumberOfChannels,
    const uint8 * Channels,
    const uint16 * Duty,
    boolean SoftwareTrigger )
```

This function will update the duty cycle of PWM output for multiple channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>NumberOfChannels</i>	The number of channels which should be updated.
in	<i>Channels</i>	The list of channels which should be updated.
in	<i>Duty</i>	The list of duty cycles for selected channels.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.
- FTM_PWM_STATUS_ERROR : Error occurred.

6.2.6.5 Ftm_Pwm_Ip_UpdatePwmPeriod()

```
Ftm_Pwm_Ip_StatusType Ftm_Pwm_Ip_UpdatePwmPeriod (
    uint8 Instance,
    uint32 NewValue,
    boolean SoftwareTrigger )
```

This function will update the new period in the frequency or in the counter value into mode register which modify the period of PWM signal on the Channel output.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>NewValue</i>	The frequency or the counter value which will select with modified value for PWM signal. If the type of update in the duty cycle, the newValue parameter must be value between 1U and maximum is the frequency of the FTM counter. If the type of update in ticks, the newValue parameter must be value between 1U and 0xFFFFU.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

operation status

- FTM_PWM_STATUS_SUCCESS : Completed successfully.
- FTM_PWM_STATUS_ERROR : Error occurred.

6.2.6.6 Ftm_Pwm_Ip_SwOutputControl()

```
void Ftm_Pwm_Ip_SwOutputControl (
    uint8 Instance,
    uint8 Channel,
    Ftm_Pwm_Ip_OutputStateType OutputState,
    boolean ActiveState )
```

This function performs the update of the SWOCTRL register on request to set/clear output state of channel in idle state or output-forced.

This function can be called from Pwm_Ftm_SetOutputToIdle, Pwm_Ftm_DeInit and Pwm_Ftm_ForceOutputToZero functions

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index
in	<i>OutputState</i>	PWM output level
in	<i>ActiveState</i>	Activation state of the SW Output Control TRUE - Enable SW Output Control FALSE - Disable SW Output Control

Returns

void

6.2.6.7 Ftm_Pwm_Ip_SyncUpdate()

```
void Ftm_Pwm_Ip_SyncUpdate (
    uint8 Instance )
```

Ftm_Pwm_Ip_SyncUpdate.

This function will allow synchronized loading of the duty registers for all the channels of a given FTM module

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
----	-----------------	-------------------------------------

Returns

void

6.2.6.8 Ftm_Pwm_Ip_DisableNotification()

```
void Ftm_Pwm_Ip_DisableNotification (
    uint8 Instance,
    Ftm_Pwm_Ip_NotifType NotifType )
```

This function disables the user notifications.

This function disables the user notifications for the corresponding type of notification

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>NotifType</i>	The type of notification

Returns

void

6.2.6.9 Ftm_Pwm_Ip_EnableNotification()

```
void Ftm_Pwm_Ip_EnableNotification (
    uint8 Instance,
    Ftm_Pwm_Ip_NotifType NotifType )
```

This function enables the user notifications.

This function enables the user notifications for the corresponding type of notification

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>NotifType</i>	The type of notification

Returns

void

6.2.6.10 Ftm_Pwm_Ip_SetClockMode()

```
void Ftm_Pwm_Ip_SetClockMode (
    uint8 Instance,
    Ftm_Pwm_Ip_PrescalerType Prescaler )
```

This function sets the value of the prescaler on all FTM modules.

This function sets the prescaler depending on the value ePrescaler parameter.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Prescaler</i>	Prescaler values

Returns

void

6.2.6.11 Ftm_Pwm_Ip_ResetCounter()

```
void Ftm_Pwm_Ip_ResetCounter (
    uint8 Instance,
    boolean IsResetCnt )
```

This function enable/disable counter reset by [Pwm_SyncUpdate\(\)](#) call.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>IsResetCnt</i>	Reset counter or not

Returns

void

6.2.6.12 Ftm_Pwm_Ip_MaskOutputChannels()

```
void Ftm_Pwm_Ip_MaskOutputChannels (
    uint8 Instance,
    uint32 ChannelsMask,
    boolean SoftwareTrigger )
```

This function will mask the output of the channels and at match events will be ignored by the masked channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>ChannelsMask</i>	The mask which will select which channels will ignore match events.

Parameters

in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.
----	------------------------	--

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.

6.2.6.13 Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel()

```
void Ftm_Pwm_Ip_UpdatePwmDutyCycleChannel (
    uint8 Instance,
    uint8 Channel,
    uint16 DutyCycle,
    boolean SoftwareTrigger )
```

This function sets the duty cycle for the specified PWM channel.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The FTM peripheral channel number.
in	<i>DutyCycle</i>	PWM duty cycle value [0-0x8000]
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

void

6.2.6.14 Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty()

```
void Ftm_Pwm_Ip_UpdatePwmPeriodAndDuty (
    uint8 Instance,
    uint8 Channel,
    uint16 Period,
    uint16 DutyCycle,
    boolean SoftwareTrigger )
```

This function sets the duty cycle and period for the specified PWM channel.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The FTM peripheral channel number.
in	<i>Period</i>	The period will be updated
in	<i>DutyCycle</i>	PWM duty cycle value [0-0x8000]
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

void

6.2.6.15 Ftm_Pwm_Ip_GetOutputState()

```
Ftm_Pwm_Ip_OutputStateType Ftm_Pwm_Ip_GetOutputState (
    uint8 Instance,
    uint8 Channel )
```

This function used to get the current output state of the PWM channel.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index

Returns

Pwm_Ftm_OutputStateType FTM_PWM_OUTPUT_STATE_LOW The output pin is in LOW logic level
 FTM_PWM_OUTPUT_STATE_HIGH The output pin is in HIGH logic level

6.2.6.16 Ftm_Pwm_Ip_SetPowerState()

```
void Ftm_Pwm_Ip_SetPowerState (
    uint8 Instance,
    Ftm_Pwm_Ip_PowerStateType PowerState )
```

Put PWM channel based on FTM to target power mode.

This function will disable clock if target power mode is Low power, and enable clock if target power mode is full power

- nPowerState value is either PWM_LOW_POWER or PWM_FULL_POWER
- the values associated to either types are configured by tooling and the physical values associated are written to hardware registry.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>PowerState</i>	Target mode value

Returns

none

6.2.6.17 Ftm_Pwm_Ip_GetChannelState()

```
uint16 Ftm_Pwm_Ip_GetChannelState (
    uint8 Instance,
    uint8 Channel )
```

This function read the duty cycle value from the duty cycle array and returns this value.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index

Returns

uint16 Duty cycle value

6.2.6.18 Ftm_Pwm_Ip_SetPhaseShift()

```
void Ftm_Pwm_Ip_SetPhaseShift (
    uint8 Instance,
    uint8 Channel,
    uint16 Period,
    uint16 PhaseShift,
    boolean SoftwareTrigger )
```

This function set phase shift value and also force duty cycle to 50%.

In order to have Phase-Shifted Full-Bridge controller, Pwm_SetPhaseShift is introduced. This function bases on FTM Combine mode with C(n) and C(n+1) combine to generate leading edge and trailing edge. Pwm_SetPhaseShift allows to set both phase shift value and period, the duty value is fixed to 50%.

Parameters

in	<i>Instance</i>	FTM hardware module index
in	<i>Channel</i>	FTM hardware channel index
in	<i>Period</i>	PWM signal period value
in	<i>PhaseShift</i>	Phase shift value
in	<i>SoftwareTrigger</i>	Buffer value be loaded or not

Returns

void

6.2.6.19 Ftm_Pwm_Ip_SetDutyPhaseShift()

```
void Ftm_Pwm_Ip_SetDutyPhaseShift (
    uint8 Instance,
    uint8 Channel,
    uint16 DutyCycle,
    uint16 PhaseShift,
    boolean SyncUpdate )
```

This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)

Ftm_Pwm_Ip_SetDutyPhaseShift allows to set both phase shift and duty cycle value, The phase shift is the offset of the leading edge of the signal in respect to period starting point.

Parameters

in	<i>Instance</i>	FTM hardware module index
----	-----------------	---------------------------

Parameters

in	<i>Channel</i>	FTM hardware channel index
in	<i>DutyCycle</i>	Pwm duty cycle value 0x0000 for 0% ... 0x8000 for 100%
in	<i>PhaseShift</i>	Phase shift value (in ticks)
in	<i>SyncUpdate</i>	TRUE Set the phase shift and duty cycle value base on the synchronization when calling Pwm_SyncUpdate. FALSE Set phase shift and duty cycle value immediately

Returns

void

6.2.6.20 Ftm_Pwm_Ip_UnMaskOutputChannels()

```
void Ftm_Pwm_Ip_UnMaskOutputChannels (
    uint8 Instance,
    uint32 ChannelsMask,
    boolean SoftwareTrigger )
```

This function will unmask the output of the channels and at match events will be ignored by the masked channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>ChannelsMask</i>	The mask which will select which channels will ignore match events.
in	<i>SoftwareTrigger</i>	If true a software trigger is generate to update PWM parameters.

Returns

success

- FTM_PWM_STATUS_SUCCESS : Completed successfully.

6.2.6.21 Ftm_Pwm_Ip_EnableTrigger()

```
void Ftm_Pwm_Ip_EnableTrigger (
    uint8 Instance,
    uint32 TriggerMask )
```

This function enables trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

<i>Instance</i>	FTM hardware module index TriggerMask Bit mask will be clear for FTM_EXTTRIG.
-----------------	---

Returns

void

6.2.6.22 Ftm_Pwm_Ip_DisableTrigger()

```
void Ftm_Pwm_Ip_DisableTrigger (
    uint8 Instance,
    uint32 TriggerMask )
```

This function disables trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

<i>Instance</i>	FTM hardware module index TriggerMask Bit mask will be clear for FTM_EXTTRIG.
-----------------	---

Returns

void

6.2.6.23 Ftm_Pwm_Ip_SetChannelDeadTime()

```
void Ftm_Pwm_Ip_SetChannelDeadTime (
    uint8 Instance,
    uint8 Channel,
    uint16 DeadTimeVal )
```

This function is used to update the deadtime at runtime for Pwm channels.

Parameters

in	<i>Instance</i>	The FTM peripheral instance number.
in	<i>Channel</i>	The FTM hardware channel index
in	<i>DeadTimeVal</i>	Deadtime value

Returns

void

6.2.7 Variable Documentation

6.2.7.1 Ftm_Pwm_Ip_pFaultIrqCallback

```
Ftm_Pwm_Ip_FaultCallbackType Ftm_Pwm_Ip_pFaultIrqCallback[ (FTM_INSTANCE_COUNT) ][ (4U) ] [extern]
```

Arrays for irq fault notifications.

6.2.7.2 Ftm_Pwm_Ip_pReloadIrqCallback

`Ftm_Pwm_Ip_ReloadCallbackCfgType Ftm_Pwm_Ip_pReloadIrqCallback[ ((FTM_INSTANCE_COUNT)) ]` [extern]

Arrays for irq reload notifications of each Instance.

6.2.7.3 Ftm_Pwm_Ip_aInstanceState

`Ftm_Pwm_Ip_ChannelStateType Ftm_Pwm_Ip_aInstanceState[ ((FTM_INSTANCE_COUNT)) ]` [extern]

Arrays to store the state of each instance.

6.2.7.4 Ftm_Pwm_Ip_aPreviousChannelState

`Ftm_Pwm_Ip_ChannelStateType Ftm_Pwm_Ip_aPreviousChannelState[ ((FTM_INSTANCE_COUNT)) ][ (FTM_CONTROLS_COUNT) ]` [extern]

Arrays to store the previous state of each channel.

6.2.7.5 Ftm_Pwm_Ip_pOverflowIrqCallback

`Ftm_Pwm_Ip_CallbackCfgType Ftm_Pwm_Ip_pOverflowIrqCallback[ ((FTM_INSTANCE_COUNT)) ]` [extern]

Arrays for irq overflow timer notifications of each Instance.

6.2.7.6 Ftm_Pwm_Ip_aChannelState

`Ftm_Pwm_Ip_ChannelStateType Ftm_Pwm_Ip_aChannelState[ ((FTM_INSTANCE_COUNT)) ][ (FTM_CONTROLS_COUNT) ]` [extern]

Arrays to store the state of channel.

6.2.7.7 Ftm_Pwm_Ip_aNotifIrq

`Ftm_Pwm_Ip_EdgeInterruptType Ftm_Pwm_Ip_aNotifIrq[ ((FTM_INSTANCE_COUNT)) ][ (FTM_CONTROLS_COUNT) ]` [extern]

Arrays to store the type of notification for each channel.

6.2.7.8 Ftm_Pwm_Ip_aPeriod

`uint16 Ftm_Pwm_Ip_aPeriod[ ((FTM_INSTANCE_COUNT)) ]` [extern]

Arrays to store the period values for each Instance.

6.3 Pwm Driver

6.3.1 Detailed Description

Data Structures

- struct [Pwm_ChannelConfigType](#)
Pwm channel high level configuration structure. [More...](#)
- struct [Pwm_ConfigType](#)
Pwm high level configuration structure. [More...](#)

Macros

- [#define PWM_DUTY_CYCLE_100_U16](#)
100% duty cycle
- [#define PWM_E_INIT_FAILED](#)
API Pwm_Init service called with wrong parameter.
- [#define PWM_E_UNINIT](#)
API service used without module initialization.
- [#define PWM_E_PARAM_CHANNEL](#)
API service used with an invalid channel Identifier.
- [#define PWM_E_PERIOD_UNCHANGEABLE](#)
Usage of unauthorized PWM service on PWM channel configured a fixed period.
- [#define PWM_E_ALREADY_INITIALIZED](#)
API Pwm_Init service called while the PWM driver has already been initialized.
- [#define PWM_E_PARAM_POINTER](#)
Generated when a NULL_PTR pointer is passed to Pwm_GetVersionInfo function.
- [#define PWM_E_NOT_DISENGAGED](#)
Generated when Pwm_SetPowerState is called while the PWM module is still in use.
- [#define PWM_E_POWER_STATE_NOT_SUPPORTED](#)
The requested power state is not supported by the PWM module.
- [#define PWM_E_TRANSITION_NOT_POSSIBLE](#)
Generated The requested power state is not reachable from the current one.
- [#define PWM_E_PERIPHERAL_NOT_PREPARED](#)
Generated when Pwm_SetPowerState has been called without having called the API Pwm_PreparePowerState before.
- [#define PWM_E_PERIODVALUE](#)
Pwm_SetPeriodAndDuty called with invalid period range.
- [#define PWM_E_PARAM_NOTIFICATION](#)
Invalid polarity selected for edge notification.
- [#define PWM_E_PARAM_NOTIFICATION_NULL](#)
NULL_PTR function is configured as notification callback.
- [#define PWM_E_DUTYCYCLE_RANGE](#)
Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty called with invalid duty cycle range.
- [#define PWM_E_CHANNEL_OFFSET_VALUE](#)

- Generated when the configured offset for the OPWMB channel is more than the period of the associated MCB channel.
- #define `PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE`
Generated when the requested offset value plus the current requested duty cycle leads to programming event B over the Period value leading to unexpected behavior of the PWM signal.
- #define `PWM_E_PARAM_INSTANCE`
Generated when the module id is more than the number of module that supported by this platform.
- #define `PWM_E_UNEXPECTED_ISR`
Generated when an ISR has been triggered.
- #define `PWM_E_PARAM_PHASESHIFT_RANGE`
Generated when requested phase shift value greater than 0x4000 (50%)
- #define `PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED`
Generated when given channel does not support phase shift feature.
- #define `PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED`
Generated when given channel does not support duty synchronous feature. (PHASE_SHIFTED_SYNCED and P↔HASE_SHIFTED_COMPLEMENTARY)
- #define `PWM_E_TRIGGER_MASK`
Generated when bit mask is not compatible with hardware register.
- #define `PWM_E_FORCE_OUTPUT_NOT_SUPPORTED`
Generated when given channel does not support force output to zero feature.
- #define `PWM_E_PARAM_CONFIG`
Generated when the requested resource is configured to be unavailable on the current partition.
- #define `PWM_E_DEADTIME_RANGE`
Generated when the configured dead time value is not valid.
- #define `PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED`
Generated when the configured dead time value is not valid.
- #define `PWM_E_GETOUTPUTSTATE_NOT_SUPPORTED`
Generated when given channel does not support getting the state of output signal feature.
- #define `PWM_INIT_ID`
API service ID of Pwm_Init function.
- #define `PWM_DEINIT_ID`
API service ID of Pwm_DeInit function.
- #define `PWM_SETDUTYCYCLE_ID`
API service ID of Pwm_SetDutyCycle function.
- #define `PWM_SETPERIODANDDDUTY_ID`
API service ID of Pwm_SetPeriodAndDuty function.
- #define `PWM_SETOUTPUTTOIDLE_ID`
API service ID of Pwm_SetOutputToIdle function.
- #define `PWM_GETOUTPUTSTATE_ID`
API service ID of Pwm_GetOutputState function.
- #define `PWM_DISABLENOTIFICATION_ID`
API service ID of Pwm_DisableNotification function.

- `#define PWM_ENABLENOTIFICATION_ID`
API service ID of *Pwm_EnableNotification* function.
- `#define PWM_GETVERSIONINFO_ID`
API service ID of *Pwm_GetVersionInfo* function.
- `#define PWM_SETPOWERSTATE_ID`
API service ID of *Pwm_SetPowerState* function.
- `#define PWM_GETCURRENTPOWERSTATE_ID`
API service ID of *Pwm_GetCurrentPowerState* function.
- `#define PWM_GETTARGETPOWERSTATE_ID`
API service ID of *Pwm_GetTargetPowerState* function.
- `#define PWM_PREPAREPOWERSTATE_ID`
API service ID of *Pwm_PrepPowerState* function.
- `#define PWM_MAIN_POWERTRANSITIONMANAGER_ID`
API service ID of *Pwm_Main_PowerTransitionManager* function.
- `#define PWM_GETCHANNELSTATE_ID`
API service ID of *Pwm_GetChannelState* function.
- `#define PWM_FORCEOUTPUTTOZERO_ID`
API service ID of *Pwm_ForceOutputToZero* function.
- `#define PWM_SETCLOCKMODE_ID`
API service ID of *Pwm_SetClockMode* function.
- `#define PWM_SYNCUPDATE_ID`
API service ID of *Pwm_SyncUpdate* function.
- `#define PWM_SETPERIODANDDDUTY_NO_UPDATE_ID`
API service ID of *Pwm_SetPeriodAndDuty_NoUpdate* function.
- `#define PWM_SETDUTYCYCLE_NO_UPDATE_ID`
API service ID of *Pwm_SetDutyCycle_NoUpdate* function.
- `#define PWM_SETCHANNELDEADTIME_ID`
API service ID of *Pwm_SetChannelDeadTime* function.
- `#define PWM_MASKOUTPUT_ID`
API service ID of *Pwm_MaskOutputs* function.
- `#define PWM_UNMASKOUTPUT_ID`
API service ID of *Pwm_UnMaskOutputs* function.
- `#define PWM_ENABLETRIGGER_ID`
API service ID of *Pwm_EnableTrigger* function.
- `#define PWM_DISABLETRIGGER_ID`
API service ID of *Pwm_DisableTrigger* function.
- `#define PWM_RESETCOUNTERENABLE_ID`
API service ID of *Pwm_ResetCounterEnable* function.
- `#define PWM_RESETCOUNTERDISABLE_ID`
API service ID of *Pwm_ResetCounterDisable* function.
- `#define PWM_DISABLERELOADNOTIF_ID`

API service ID of Pwm_DisableReloadNotification function.

- #define [PWM_ENABLERELOADNOTIF_ID](#)

API service ID of Pwm_EnableReloadNotification function.

- #define [PWM_FASTUPDATEDUTYCYCLE_ID](#)

API service ID of Pwm_EnableReloadNotification function.

- #define [PWM_SETDUTYPHASESHIFT_ID](#)

API service ID of Pwm_SetDutyPhaseShift function.

- #define [PWM_DISABLEFAULTNOTIF_ID](#)

API service ID of Pwm_DisableFaultNotification function.

- #define [PWM_ENABLEFAULTNOTIF_ID](#)

API service ID of Pwm_EnableFaultNotification function.

Types Reference

- typedef uint8 [Pwm_ChannelType](#)

PWM channel type.

- typedef uint8 [Pwm_InstanceType](#)

PWM channel type.

- typedef Pwm_Ipw_PeriodType [Pwm_PeriodType](#)

PWM period type.

- typedef Pwm_Ipw_DutyType [Pwm_DutyType](#)

PWM duty type.

- typedef void(* [Pwm_NotifyType](#)) (void)

Channel notification typedef.

Enum Reference

- enum [Pwm_OutputStateType](#)

Output signal level.

- enum [Pwm_EdgeNotificationType](#)

Edge notification type.

- enum [Pwm_ChannelClassType](#)

PWM channel class type.

- enum [Pwm_PowerStateType](#)

Power state type.

- enum [Pwm_PowerStateRequestResultType](#)

Result of power state type.

- enum [Pwm_PrescalerType](#)

Prescaler type.

Function Reference

- void [Pwm_Init](#) (const [Pwm_ConfigType](#) *ConfigPtr)
This function initializes the Pwm driver.
- void [Pwm_DeInit](#) (void)
This function deinitializes the Pwm driver.
- void [Pwm_SetDutyCycle](#) ([Pwm_ChannelType](#) ChannelNumber, uint16 DutyCycle)
This function sets the dutycycle for the specified Pwm channel.
- void [Pwm_SetPeriodAndDuty](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_PeriodType](#) Period, uint16 DutyCycle)
This function sets the period and the dutycycle for the specified Pwm channel.
- void [Pwm_SetOutputToIdle](#) ([Pwm_ChannelType](#) ChannelNumber)
This function sets the generated pwm signal to the idle value configured.
- [Pwm_OutputStateType](#) [Pwm_GetOutputState](#) ([Pwm_ChannelType](#) ChannelNumber)
This function returns the signal output state.
- void [Pwm_EnableNotification](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_EdgeNotificationType](#) Notification)
This function enables the user notifications.
- void [Pwm_DisableNotification](#) ([Pwm_ChannelType](#) ChannelNumber)
This function disables the user notifications.
- void [Pwm_GetVersionInfo](#) ([Std_VersionInfoType](#) *versioninfo)
This function returns Pwm driver version details.
- uint16 [Pwm_GetChannelState](#) ([Pwm_ChannelType](#) ChannelNumber)
This function returns the duty cycle of the channel passed as parameter.
- void [Pwm_ForceOutputToZero](#) ([Pwm_ChannelType](#) ChannelNumber, boolean bForce)
This function enables-disables the forcing of the output of a given channel to logic 0.
- void [Pwm_SetClockMode](#) ([Pwm_PrescalerType](#) Prescaler)
Implementation specific function to change the peripheral clock frequency.
- void [Pwm_SetChannelDeadTime](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_PeriodType](#) DeadTimeTicks)
- void [Pwm_SetDutyCycle_NoUpdate](#) ([Pwm_ChannelType](#) ChannelNumber, uint16 DutyCycle)
This function sets the values of dutycycle for the specified Pwm channel but without updating the PWM output.
- void [Pwm_SetPeriodAndDuty_NoUpdate](#) ([Pwm_ChannelType](#) ChannelNumber, [Pwm_PeriodType](#) Period, uint16 DutyCycle)
This function sets the values of the period and the dutycycle for the specified Pwm channel into the hardware buffers but without updating the PWM output..
- void [Pwm_SyncUpdate](#) (uint8 ModuleId)
Implementation specific function to updates duty synchronization.
- void [Pwm_SetDutyPhaseShift](#) ([Pwm_ChannelType](#) ChannelNumber, uint16 DutyCycle, [Pwm_DutyType](#) PhaseShift, boolean SyncUpdate)
This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)
- void [Pwm_EnableTrigger](#) (uint8 TriggerHostId, uint16 TriggerMask)
This function enable trigger generation for specific source.
- void [Pwm_DisableTrigger](#) (uint8 TriggerHostId, uint16 TriggerMask)

This function disable trigger generation for specific source.

- void [Pwm_ResetCounterEnable](#) (uint8 ModuleId)

This function shall enable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.

- void [Pwm_ResetCounterDisable](#) (uint8 ModuleId)

This function shall disable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.

- void [Pwm_FastUpdateDutyCycle](#) (uint8 ModuleId, uint8 NumberOfChannels, const uint8 *Channels, const uint16 *DutyCycle, boolean SyncUpdate)

This function will update the duty cycle of PWM output for multiple channels with minimum overhead.

- void [Pwm_MaskOutputs](#) (uint8 ModuleId, uint16 ChannelsMask, boolean UpdateType)

This function force channels output to their inactive state (as immediate or synchronized base on API parameter UpdateType).

- void [Pwm_UnMaskOutputs](#) (uint8 ModuleId, uint16 ChannelsMask, boolean UpdateType)

This function puts channels output to normal operation state (as immediate or synchronized base on API parameter UpdateType).

- void [Pwm_EnableReloadNotification](#) (uint8 ModuleId)

This function enables the user reload notifications.

- void [Pwm_DisableReloadNotification](#) (uint8 ModuleId)

This function disables the user reload notifications.

- void [Pwm_EnableFaultNotification](#) (uint8 ModuleId)

This function enables the user fault notifications.

- void [Pwm_DisableFaultNotification](#) (uint8 ModuleId)

This function disables the user fault notifications.

- Std_ReturnType [Pwm_SetPowerState](#) ([Pwm_PowerStateRequestResultType](#) *Result)

Enters the already prepared power state.

- Std_ReturnType [Pwm_GetCurrentPowerState](#) ([Pwm_PowerStateType](#) *CurrentPowerState, [Pwm_PowerStateRequestResultType](#) *Result)

Get the current power state of the Pwm HW unit.

- Std_ReturnType [Pwm_GetTargetPowerState](#) ([Pwm_PowerStateType](#) *TargetPowerState, [Pwm_PowerStateRequestResultType](#) *Result)

Get the target power state of the Pwm HW unit.

- Std_ReturnType [Pwm_PreparePowerState](#) ([Pwm_PowerStateType](#) PowerState, [Pwm_PowerStateRequestResultType](#) *Result)

Starts the needed process to allow the Pwm HW module to enter the requested power state.

6.3.2 Data Structure Documentation

6.3.2.1 struct [Pwm_ChannelConfigType](#)

Pwm channel high level configuration structure.

Definition at line 658 of file Pwm.h.

Data Fields

- const `Pwm_ChannelType` `ChannelId`
Id for the logical channel.
- const `Pwm_ChannelClassType` `PwmChannelClass`
Channel class type: Variable/Fixed period.
- const `Pwm_IpwChannelConfigType` `IpwChannelCfg`
The type of ip channel configured.
- const `Pwm_OutputStateType` `ChannelIdleState`
The state of the channel output in idle mode.

Field Documentation**ChannelId**

const `Pwm_ChannelType` `ChannelId`

Id for the logical channel.

Definition at line 661 of file `Pwm.h`.

PwmChannelClass

const `Pwm_ChannelClassType` `PwmChannelClass`

Channel class type: Variable/Fixed period.

Definition at line 663 of file `Pwm.h`.

IpwChannelCfg

const `Pwm_IpwChannelConfigType` `IpwChannelCfg`

The type of ip channel configured.

Definition at line 665 of file `Pwm.h`.

ChannelIdleState

const `Pwm_OutputStateType` `ChannelIdleState`

The state of the channel output in idle mode.

Definition at line 667 of file `Pwm.h`.

6.3.2.2 struct Pwm_ConfigType

Pwm high level configuration structure.

Definition at line 677 of file `Pwm.h`.

Data Fields

- const `Pwm_ChannelType NumChannels`
Number of Pwm configured channels.
- const `Pwm_ChannelConfigType(* PwmChannelsConfig )[]`
Pointer to the list of Pwm configured channels.
- const `Pwm_InstanceType NumInstances`
Number of Pwm configured instances.
- const `Pwm_IpwInstanceConfigType(* PwmInstancesConfig )[]`
Pointer to the list of Pwm configured channels.
- const `Pwm_ChannelType HwToLogicChannelMap [(36U)]`
Index table to translate HW channels to logical used to process interrupts for notifications.

Field Documentation

NumChannels

```
const Pwm_ChannelType NumChannels
```

Number of Pwm configured channels.

Definition at line 684 of file Pwm.h.

PwmChannelsConfig

```
const Pwm_ChannelConfigType(* PwmChannelsConfig)[]
```

Pointer to the list of Pwm configured channels.

Definition at line 686 of file Pwm.h.

NumInstances

```
const Pwm_InstanceType NumInstances
```

Number of Pwm configured instances.

Definition at line 689 of file Pwm.h.

PwmInstancesConfig

```
const Pwm_IpwInstanceConfigType(* PwmInstancesConfig)[]
```

Pointer to the list of Pwm configured channels.

Definition at line 691 of file Pwm.h.

HwToLogicChannelMap

```
const Pwm_ChannelType HwToLogicChannelMap[ (36U) ]
```

Index table to translate HW channels to logical used to process interrupts for notifications.

Definition at line 699 of file Pwm.h.

6.3.3 Macro Definition Documentation

6.3.3.1 PWM_DUTY_CYCLE_100_U16

```
#define PWM_DUTY_CYCLE_100_U16
```

100% duty cycle

Errors and exceptions that will be detected by the PWM driver generated when Pwm_SetDutyCycle or Pwm_↔ SetPeriodAndDuty are called with a value for duty cycle out of valid range [0x0000, 0x8000]

Definition at line 144 of file Pwm.h.

6.3.3.2 PWM_E_INIT_FAILED

```
#define PWM_E_INIT_FAILED
```

API Pwm_Init service called with wrong parameter.

Errors and exceptions that will be detected by the PWM driver

Definition at line 151 of file Pwm.h.

6.3.3.3 PWM_E_UNINIT

```
#define PWM_E_UNINIT
```

API service used without module initialization.

Errors and exceptions that will be detected by the PWM driver

Definition at line 157 of file Pwm.h.

6.3.3.4 PWM_E_PARAM_CHANNEL

```
#define PWM_E_PARAM_CHANNEL
```

API service used with an invalid channel Identifier.

Errors and exceptions that will be detected by the PWM driver

Definition at line 163 of file Pwm.h.

6.3.3.5 PWM_E_PERIOD_UNCHANGEABLE

```
#define PWM_E_PERIOD_UNCHANGEABLE
```

Usage of unauthorized PWM service on PWM channel configured a fixed period.

Errors and exceptions that will be detected by the PWM driver

Definition at line 169 of file Pwm.h.

6.3.3.6 PWM_E_ALREADY_INITIALIZED

```
#define PWM_E_ALREADY_INITIALIZED
```

API Pwm_Init service called while the PWM driver has already been initialized.

Errors and exceptions that will be detected by the PWM driver

Definition at line 175 of file Pwm.h.

6.3.3.7 PWM_E_PARAM_POINTER

```
#define PWM_E_PARAM_POINTER
```

Generated when a NULL_PTR pointer is passed to Pwm_GetVersionInfo function.

Errors and exceptions that will be detected by the PWM driver

Definition at line 181 of file Pwm.h.

6.3.3.8 PWM_E_NOT_DISENGAGED

```
#define PWM_E_NOT_DISENGAGED
```

Generated when Pwm_SetPowerState is called while the PWM module is still in use.

Errors and exceptions that will be detected by the PWM driver

Definition at line 187 of file Pwm.h.

6.3.3.9 PWM_E_POWER_STATE_NOT_SUPPORTED

```
#define PWM_E_POWER_STATE_NOT_SUPPORTED
```

The requested power state is not supported by the PWM module.

Errors and exceptions that will be detected by the PWM driver

Definition at line 193 of file Pwm.h.

6.3.3.10 PWM_E_TRANSITION_NOT_POSSIBLE

```
#define PWM_E_TRANSITION_NOT_POSSIBLE
```

Generated The requested power state is not reachable from the current one.

Errors and exceptions that will be detected by the PWM driver

Definition at line 199 of file Pwm.h.

6.3.3.11 PWM_E_PERIPHERAL_NOT_PREPARED

```
#define PWM_E_PERIPHERAL_NOT_PREPARED
```

Generated when Pwm_SetPowerState has been called without having called the API Pwm_PreparePowerState before.

Errors and exceptions that will be detected by the PWM driver

Definition at line 206 of file Pwm.h.

6.3.3.12 PWM_E_PERIODVALUE

```
#define PWM_E_PERIODVALUE
```

Pwm_SetPeriodAndDuty called with invalid period range.

Generated when Pwm_SetPeriodAndDuty is called with a value for period out of valid range [0x0000, PWM_MAX_PERIOD]

Definition at line 213 of file Pwm.h.

6.3.3.13 PWM_E_PARAM_NOTIFICATION

```
#define PWM_E_PARAM_NOTIFICATION
```

Invalid polarity selected for edge notification.

Will be generated when an invalid polarity, edge notification is requested for one PWM channel. Due to the limitations that are present in the eMIOS implementation not all the polarity notifications combinations can be supported.

Definition at line 221 of file Pwm.h.

6.3.3.14 PWM_E_PARAM_NOTIFICATION_NULL

```
#define PWM_E_PARAM_NOTIFICATION_NULL
```

NULL_PTR function is configured as notification callback.

Will be generated when a NULL_PTR function is configured as notification callback for one PWM channel and Pwm_EnableNotification is called for that channel

Definition at line 228 of file Pwm.h.

6.3.3.15 PWM_E_DUTYCYCLE_RANGE

```
#define PWM_E_DUTYCYCLE_RANGE
```

Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty called with invalid duty cycle range.

Generated when Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty are called with a value for duty cycle out of valid range [0x0000, 0x8000]

Definition at line 235 of file Pwm.h.

6.3.3.16 PWM_E_CHANNEL_OFFSET_VALUE

```
#define PWM_E_CHANNEL_OFFSET_VALUE
```

Generated when the configured offset for the OPWMB channel is more than the period of the associated MCB channel.

Errors and exceptions that will be detected by the PWM driver

Definition at line 241 of file Pwm.h.

6.3.3.17 PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE

```
#define PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE
```

Generated when the requested offset value plus the current requested duty cycle leads to programming event B over the Period value leading to unexpected behavior of the PWM signal.

Errors and exceptions that will be detected by the PWM driver

Definition at line 248 of file Pwm.h.

6.3.3.18 PWM_E_PARAM_INSTANCE

```
#define PWM_E_PARAM_INSTANCE
```

Generated when the module id is more than the number of module that supported by this platform.

Errors and exceptions that will be detected by the PWM driver

Definition at line 255 of file Pwm.h.

6.3.3.19 PWM_E_UNEXPECTED_ISR

```
#define PWM_E_UNEXPECTED_ISR
```

Generated when an ISR has been triggered.

1. when the driver is not initialized
2. for a HW channel that is not used by any logic channel
3. for a logic channel that has no notification configured

Errors and exceptions that will be detected by the PWM driver

Definition at line 263 of file Pwm.h.

6.3.3.20 PWM_E_PARAM_PHASESHIFT_RANGE

```
#define PWM_E_PARAM_PHASESHIFT_RANGE
```

Generated when requested phase shift value greater than 0x4000 (50%)

Pwm_SetPhaseShift only works with Combine channel (COMBINED_SYNCED or COMBINED_COMPLEMENTARY) Which do not support matching at next cycle. The duty cycle is always fixed value at 50%, so Phase Shift cannot greater than 50%

Definition at line 271 of file Pwm.h.

6.3.3.21 PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED

```
#define PWM_E_CHANNEL_PHASE_SHIFT_NOT_SUPPORTED
```

Generated when given channel does not support phase shift feature.

For FTM, only combine mode (COMBINED_SYNCED and COMBINED_COMPLEMENTARY) is supported. For eMIOS, only OPWMB, OPWM and OPWMT mode is supported.

Definition at line 278 of file Pwm.h.

6.3.3.22 PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED

```
#define PWM_E_DUTY_SYNCHRONOUS_NOT_SUPPORTED
```

Generated when given channel does not support duty synchronous feature. (PHASE_SHIFTED_SYNCED and PHASE_SHIFTED_COMPLEMENTARY)

For FTM, please note that Modified Combine mode does not support synchronous update. Therefore Pwm_SetDutyCycle_NoUpdate and Pwm_SetPeriodAndDuty_NoUpdate should not be called in this case. For eMIOS, please note that channels using DAOC mode or channels in idle state does not support.

Definition at line 289 of file Pwm.h.

6.3.3.23 PWM_E_TRIGGER_MASK

```
#define PWM_E_TRIGGER_MASK
```

Generated when bit mask is not compatible with hardware register.

Definition at line 294 of file Pwm.h.

6.3.3.24 PWM_E_FORCE_OUTPUT_NOT_SUPPORTED

```
#define PWM_E_FORCE_OUTPUT_NOT_SUPPORTED
```

Generated when given channel does not support force output to zero feature.

Only channels in FlexPWM and FTM module are supported.

Definition at line 300 of file Pwm.h.

6.3.3.25 PWM_E_PARAM_CONFIG

```
#define PWM_E_PARAM_CONFIG
```

Generated when the requested resource is configured to be unavailable on the current partition.

Only multi-partition configuration is available.

Definition at line 305 of file Pwm.h.

6.3.3.26 PWM_E_DEADTIME_RANGE

```
#define PWM_E_DEADTIME_RANGE
```

Generated when the configured dead time value is not valid.

Errors and exceptions that will be detected by the PWM driver

Definition at line 311 of file Pwm.h.

6.3.3.27 PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED

```
#define PWM_E_SETOUTPUTTOIDLE_NOT_SUPPORTED
```

Generated when the configured dead time value is not valid.

Errors and exceptions that will be detected by the PWM driver

Definition at line 317 of file Pwm.h.

6.3.3.28 PWM_E_GETOUTPUTSTATE_NOT_SUPPORTED

```
#define PWM_E_GETOUTPUTSTATE_NOT_SUPPORTED
```

Generated when given channel does not support getting the state of output signal feature.

Errors and exceptions that will be detected by the PWM driver

Definition at line 323 of file Pwm.h.

6.3.3.29 PWM_INIT_ID

```
#define PWM_INIT_ID
```

API service ID of Pwm_Init function.

Parameters used when raising an error/exception

Definition at line 329 of file Pwm.h.

6.3.3.30 PWM_DEINIT_ID

```
#define PWM_DEINIT_ID
```

API service ID of Pwm_DeInit function.

Parameters used when raising an error/exception

Definition at line 335 of file Pwm.h.

6.3.3.31 PWM_SETDUTYCYCLE_ID

```
#define PWM_SETDUTYCYCLE_ID
```

API service ID of Pwm_SetDutyCycle function.

Parameters used when raising an error/exception

Definition at line 341 of file Pwm.h.

6.3.3.32 PWM_SETPERIODANDDUTY_ID

```
#define PWM_SETPERIODANDDUTY_ID
```

API service ID of Pwm_SetPeriodAndDuty function.

Parameters used when raising an error/exception

Definition at line 347 of file Pwm.h.

6.3.3.33 PWM_SETOUTPUTTOIDLE_ID

```
#define PWM_SETOUTPUTTOIDLE_ID
```

API service ID of Pwm_SetOutputToIdle function.

Parameters used when raising an error/exception

Definition at line 353 of file Pwm.h.

6.3.3.34 PWM_GETOUTPUTSTATE_ID

```
#define PWM_GETOUTPUTSTATE_ID
```

API service ID of Pwm_GetOutputState function.

Parameters used when raising an error/exception

Definition at line 359 of file Pwm.h.

6.3.3.35 PWM_DISABLENOTIFICATION_ID

```
#define PWM_DISABLENOTIFICATION_ID
```

API service ID of Pwm_DisableNotification function.

Parameters used when raising an error/exception

Definition at line 365 of file Pwm.h.

6.3.3.36 PWM_ENABLENOTIFICATION_ID

```
#define PWM_ENABLENOTIFICATION_ID
```

API service ID of Pwm_EnableNotification function.

Parameters used when raising an error/exception

Definition at line 371 of file Pwm.h.

6.3.3.37 PWM_GETVERSIONINFO_ID

```
#define PWM_GETVERSIONINFO_ID
```

API service ID of Pwm_GetVersionInfo function.

Parameters used when raising an error/exception

Definition at line 377 of file Pwm.h.

6.3.3.38 PWM_SETPOWERSTATE_ID

```
#define PWM_SETPOWERSTATE_ID
```

API service ID of Pwm_SetPowerState function.

Parameters used when raising an error/exception

Definition at line 383 of file Pwm.h.

6.3.3.39 PWM_GETCURRENTPOWERSTATE_ID

```
#define PWM_GETCURRENTPOWERSTATE_ID
```

API service ID of Pwm_GetCurrentPowerState function.

Parameters used when raising an error/exception

Definition at line 389 of file Pwm.h.

6.3.3.40 PWM_GETTARGETPOWERSTATE_ID

```
#define PWM_GETTARGETPOWERSTATE_ID
```

API service ID of Pwm_GetTargetPowerState function.

Parameters used when raising an error/exception

Definition at line 395 of file Pwm.h.

6.3.3.41 PWM_PREPAREPOWERSTATE_ID

```
#define PWM_PREPAREPOWERSTATE_ID
```

API service ID of Pwm_PreparePowerState function.

Parameters used when raising an error/exception

Definition at line 401 of file Pwm.h.

6.3.3.42 PWM_MAIN_POWERTRANSITIONMANAGER_ID

```
#define PWM_MAIN_POWERTRANSITIONMANAGER_ID
```

API service ID of Pwm_Main_PowerTransitionManager function.

Parameters used when raising an error/exception

Definition at line 407 of file Pwm.h.

6.3.3.43 PWM_GETCHANNELSTATE_ID

```
#define PWM_GETCHANNELSTATE_ID
```

API service ID of Pwm_GetChannelState function.

Parameters used when raising an error/exception

Definition at line 413 of file Pwm.h.

6.3.3.44 PWM_FORCEOUTPUTTOZERO_ID

```
#define PWM_FORCEOUTPUTTOZERO_ID
```

API service ID of Pwm_ForceOutputToZero function.

Parameters used when raising an error/exception

Definition at line 418 of file Pwm.h.

6.3.3.45 PWM_SETCLOCKMODE_ID

```
#define PWM_SETCLOCKMODE_ID
```

API service ID of Pwm_SetClockMode function.

Parameters used when raising an error/exception

Definition at line 423 of file Pwm.h.

6.3.3.46 PWM_SYNCUPDATE_ID

```
#define PWM_SYNCUPDATE_ID
```

API service ID of Pwm_SyncUpdate function.

Parameters used when raising an error/exception

Definition at line 429 of file Pwm.h.

6.3.3.47 PWM_SETPERIODANDDUTY_NO_UPDATE_ID

```
#define PWM_SETPERIODANDDUTY_NO_UPDATE_ID
```

API service ID of Pwm_SetPeriodAndDuty_NoUpdate function.

Parameters used when raising an error/exception

Definition at line 435 of file Pwm.h.

6.3.3.48 PWM_SETDUTYCYCLE_NO_UPDATE_ID

```
#define PWM_SETDUTYCYCLE_NO_UPDATE_ID
```

API service ID of Pwm_SetDutyCycle_NoUpdate function.

Parameters used when raising an error/exception

Definition at line 441 of file Pwm.h.

6.3.3.49 PWM_SETCHANNELDEADTIME_ID

```
#define PWM_SETCHANNELDEADTIME_ID
```

API service ID of Pwm_SetChannelDeadTime function.

Parameters used when raising an error/exception

Definition at line 447 of file Pwm.h.

6.3.3.50 PWM_MASKOUTPUT_ID

```
#define PWM_MASKOUTPUT_ID
```

API service ID of Pwm_MaskOutputs function.

Parameters used when raising an error/exception

Definition at line 453 of file Pwm.h.

6.3.3.51 PWM_UNMASKOUTPUT_ID

```
#define PWM_UNMASKOUTPUT_ID
```

API service ID of Pwm_UnMaskOutputs function.

Parameters used when raising an error/exception

Definition at line 459 of file Pwm.h.

6.3.3.52 PWM_ENABLETRIGGER_ID

```
#define PWM_ENABLETRIGGER_ID
```

API service ID of Pwm_EnableTrigger function.

Parameters used when raising an error/exception

Definition at line 465 of file Pwm.h.

6.3.3.53 PWM_DISABLETRIGGER_ID

```
#define PWM_DISABLETRIGGER_ID
```

API service ID of Pwm_DisableTrigger function.

Parameters used when raising an error/exception

Definition at line 471 of file Pwm.h.

6.3.3.54 PWM_RESETCOUNTERENABLE_ID

```
#define PWM_RESETCOUNTERENABLE_ID
```

API service ID of Pwm_ResetCounterEnable function.

Parameters used when raising an error/exception

Definition at line 477 of file Pwm.h.

6.3.3.55 PWM_RESETCOUNTERDISABLE_ID

```
#define PWM_RESETCOUNTERDISABLE_ID
```

API service ID of Pwm_ResetCounterDisable function.

Parameters used when raising an error/exception

Definition at line 483 of file Pwm.h.

6.3.3.56 PWM_DISABLERELOADNOTIF_ID

```
#define PWM_DISABLERELOADNOTIF_ID
```

API service ID of Pwm_DisableReloadNotification function.

Parameters used when raising an error/exception

Definition at line 489 of file Pwm.h.

6.3.3.57 PWM_ENABLERELOADNOTIF_ID

```
#define PWM_ENABLERELOADNOTIF_ID
```

API service ID of Pwm_EnableReloadNotification function.

Parameters used when raising an error/exception

Definition at line 495 of file Pwm.h.

6.3.3.58 PWM_FASTUPDATEDUTYCYCLE_ID

```
#define PWM_FASTUPDATEDUTYCYCLE_ID
```

API service ID of Pwm_EnableReloadNotification function.

Parameters used when raising an error/exception

Definition at line 501 of file Pwm.h.

6.3.3.59 PWM_SETDUTYPHASESHIFT_ID

```
#define PWM_SETDUTYPHASESHIFT_ID
```

API service ID of Pwm_SetDutyPhaseShift function.

Parameters used when raising an error/exception

Definition at line 506 of file Pwm.h.

6.3.3.60 PWM_DISABLEFAULTNOTIF_ID

```
#define PWM_DISABLEFAULTNOTIF_ID
```

API service ID of Pwm_DisableFaultNotification function.

Parameters used when raising an error/exception

Definition at line 511 of file Pwm.h.

6.3.3.61 PWM_ENABLEFAULTNOTIF_ID

```
#define PWM_ENABLEFAULTNOTIF_ID
```

API service ID of Pwm_EnableFaultNotification function.

Parameters used when raising an error/exception

Definition at line 517 of file Pwm.h.

6.3.4 Types Reference**6.3.4.1 Pwm_ChannelType**

```
typedef uint8 Pwm_ChannelType
```

PWM channel type.

Definition at line 625 of file Pwm.h.

6.3.4.2 Pwm_InstanceType

```
typedef uint8 Pwm_InstanceType
```

PWM channel type.

Definition at line 630 of file Pwm.h.

6.3.4.3 Pwm_PeriodType

```
typedef Pwm_Ipw_PeriodType Pwm_PeriodType
```

PWM period type.

Definition at line 636 of file Pwm.h.

6.3.4.4 Pwm_DutyType

```
typedef Pwm_Ipw_DutyType Pwm_DutyType
```

PWM duty type.

Definition at line 643 of file Pwm.h.

6.3.4.5 Pwm_NotifyType

```
typedef void(* Pwm_NotifyType) (void)
```

Channel notification typedef.

Definition at line 651 of file Pwm.h.

6.3.5 Enum Reference

6.3.5.1 Pwm_OutputStateType

```
enum Pwm_OutputStateType
```

Output signal level.

This enumeration specifies the return type of Pwm_GetOutputState

Enumerator

PWM_HIGH	PWM level is logic high.
PWM_LOW	PWM level is logic low.

Definition at line 527 of file Pwm.h.

6.3.5.2 Pwm_EdgeNotificationType

```
enum Pwm_EdgeNotificationType
```

Edge notification type.

This enumeration defines the type of edge transition that can generate a notification

Enumerator

PWM_RISING_EDGE	A notification will be generated on the rising edge.
PWM_FALLING_EDGE	A notification will be generated on the falling edge.
PWM_BOTH_EDGES	A notification will be generated on any state transition.

Definition at line 541 of file Pwm.h.

6.3.5.3 Pwm_ChannelClassType

```
enum Pwm_ChannelClassType
```

PWM channel class type.

This field will specify what parameters can be altered for the selected channel

Enumerator

PWM_VARIABLE_PERIOD	The period and duty cycle can be altered.
---------------------	---

Enumerator

PWM_FIXED_PERIOD	Only the duty cycle can be altered.
PWM_FIXED_PERIOD_SHIFTED	Only the duty cycle can be altered.

Definition at line 557 of file Pwm.h.

6.3.5.4 Pwm_PowerStateType

enum `Pwm_PowerStateType`

Power state type.

Power state currently active or set as target power state.

Enumerator

PWM_FULL_POWER	PWM full power mode.
PWM_LOW_POWER	PWM low power mode.
PWM_NODEFINE_POWER	PWM no define power mode.

Definition at line 573 of file Pwm.h.

6.3.5.5 Pwm_PowerStateRequestResultType

enum `Pwm_PowerStateRequestResultType`

Result of power state type.

Result of the requests related to power state transitions.

Enumerator

PWM_SERVICE_ACCEPTED	Power state change executed.
PWM_NOT_INIT	Module not initialized.
PWM_SEQUENCE_ERROR	Wrong API call sequence.
PWM_HW_FAILURE	The HW module has a failure which prevents it to enter the required power state.
PWM_POWER_STATE_NOT_SUPP	Module does not support the requested power state.
PWM_TRANS_NOT_POSSIBLE	Module cannot transition directly from the current power state to the requested power state.

Definition at line 589 of file Pwm.h.

6.3.5.6 Pwm_PrescalerType

enum `Pwm_PrescalerType`

Prescaler type.

This enumeration specifies the possible types of prescalers used to configure base-clock timers

Enumerator

PWM_PRIMARY_PRESCALER	Selected value is the default/primary prescaler.
PWM_ALTERNATIVE_PRESCALER	Selected value is the alternative configured prescaler.

Definition at line 610 of file Pwm.h.

6.3.6 Function Reference

6.3.6.1 Pwm_Init()

```
void Pwm_Init (
    const Pwm_ConfigType * ConfigPtr )
```

This function initializes the Pwm driver.

The function Pwm_Init shall initialize all internals variables and the used PWM structure of the microcontroller according to the parameters specified in configPtr. If the duty cycle parameter equals:

- 0% or 100% : Then the PWM output signal shall be in the state according to the configured polarity parameter;
- >0% and <100%: Then the PWM output signal shall be modulated according to parameters period, duty cycle and configured polarity.

The function Pwm_SetDutyCycle shall update the duty cycle always at the end of the period if supported by the implementation and configured with PwmDutyCycleUpdatedEndperiod.

The driver shall avoid spikes on the PWM output signal when updating the PWM period and duty.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error PWM_E_PERIOD_UNCHANGEABLE if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The function Pwm_Init shall disable all notifications. The reason is that the users of these notifications may not be ready. They can call Pwm_EnableNotification to start notifications.

The function Pwm_Init shall only initialize the configured resources and shall not touch resources that are not configured in the configuration file.

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection is enabled, calling the routine Pwm_Init while the PWM driver and hardware are already initialized will cause a development error PWM_E_ALREADY_INITIALIZED. The desired functionality shall be left without any action.

For pre-compile and link time configuration variants, a NULL pointer shall be passed to the initialization routine. In this case the check for this NULL pointer has to be omitted.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ConfigPtr</i>	Pointer to PWM top configuration structure
----	------------------	--

Returns

void

6.3.6.2 Pwm_DeInit()

```
void Pwm_DeInit (
    void )
```

This function deinitializes the Pwm driver.

The function Pwm_DeInit shall deinitialize the PWM module.

The function Pwm_DeInit shall set the state of the PWM output signals to the idle state.

The function Pwm_DeInit shall disable PWM interrupts and PWM signal edge notifications.

The function Pwm_DeInit shall be pre-compile time configurable On-Off by the configuration parameter PwmDeInitApi function prototype.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Returns

void

6.3.6.3 Pwm_SetDutyCycle()

```
void Pwm_SetDutyCycle (
    Pwm_ChannelType ChannelNumber,
    uint16 DutyCycle )
```

This function sets the dutycycle for the specified Pwm channel.

The function Pwm_SetDutyCycle shall set the duty cycle of the PWM channel.

The function Pwm_SetDutyCycle shall set the PWM output state according to the configured polarity parameter, when the duty cycle = 0% or 100%.

The function Pwm_SetDutyCycle shall modulate the PWM output signal according to parameters period, duty cycle and configured polarity, when the duty cycle > 0 % and < 100%.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error

PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value.

As an implementation guide, the following source code example is given:

```
AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;
```

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled.

The detailed description of the detected errors can be found in chapter

Error classification and chapter API specification (see PWM_SWS).

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.3.6.4 Pwm_SetPeriodAndDuty()

```
void Pwm_SetPeriodAndDuty (
    Pwm_ChannelType ChannelNumber,
    Pwm_PeriodType Period,
    uint16 DutyCycle )
```

This function sets the period and the dutycycle for the specified Pwm channel.

The function Pwm_SetPeriodAndDuty shall set the duty cycle of the PWM channel.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error PWM_E_PERIOD_UNCHANGEABLE if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value.

As an implementation guide, the following source code example is given:

```
AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;
```

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Period</i>	Pwm signal period value
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.3.6.5 Pwm_SetOutputToIdle()

```
void Pwm_SetOutputToIdle (
    Pwm_ChannelType ChannelNumber )
```

This function sets the generated pwm signal to the idle value configured.

The function Pwm_SetOutputToIdle shall set immediately the PWM output to the configured Idle state.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

After the call of the function Pwm_SetOutputToIdle, variable period type channels shall be reactivated either using the Api [Pwm_SetPeriodAndDuty\(\)](#) to activate the PWM channel with the new passed period or Api [Pwm_SetDutyCycle\(\)](#) to activate the PWM channel with the old period.

After the call of the function Pwm_SetOutputToIdle, fixed period type channels shall be reactivated using only the API [Pwm_SetDutyCycle\(\)](#) to activate the PWM channel with the old period.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

void

6.3.6.6 Pwm_GetOutputState()

```
Pwm_OutputStateType Pwm_GetOutputState (
    Pwm_ChannelType ChannelNumber )
```

This function returns the signal output state.

The function Pwm_GetOutputState shall read the internal state of the PWM output signal and return it as defined in the diagram below (see PWM_SWS).

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter channelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter channelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

Due to real time constraint and setting of the PWM channel (project dependant), the output state can be modified just after the call of the service Pwm_GetOutputState.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	ChannelNumber	Pwm Channel Id in the configuration
----	---------------	-------------------------------------

Returns

Pwm_OutputStateType Pwm signal output logic value

Return values

PWM_LOW	The output state of PWM channel is low
PWM_HIGH	The output state of PWM channel is high

6.3.6.7 Pwm_EnableNotification()

```
void Pwm_EnableNotification (
    Pwm_ChannelType ChannelNumber,
    Pwm_EdgeNotificationType Notification )
```


This function enables the user notifications.

The function `Pwm_EnableNotification` shall enable the PWM signal edge notification according to notification parameter. If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PA↔RAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Notification</i>	Notification type to be enabled

Returns

void

6.3.6.8 Pwm_DisableNotification()

```
void Pwm_DisableNotification (
    Pwm_ChannelType ChannelNumber )
```

This function disables the user notifications.

If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PA↔RAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

All functions from the PWM module except `Pwm_Init`, `Pwm_DeInit` and `Pwm_GetVersionInfo` shall be re-entrant for different PWM channel numbers. In order to keep a simple module implementation, no check of PWM088 must be performed by the module. The function `Pwm_DisableNotification` shall be pre compile time configurable On-Off by the configuration parameter: `PwmNotificationSupported`.

Module Documentation

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

void

6.3.6.9 Pwm_GetVersionInfo()

```
void Pwm_GetVersionInfo (
    Std_VersionInfoType * versioninfo )
```

This function returns Pwm driver version details.

The function Pwm_GetVersionInfo shall return the version information of this module. The version information includes: Module Id, Vendor Id, Vendor specific version number.

Parameters

out	<i>versioninfo</i>	Pointer to Std_VersionInfoType output variable
-----	--------------------	--

Returns

void

6.3.6.10 Pwm_GetChannelState()

```
uint16 Pwm_GetChannelState (
    Pwm_ChannelType ChannelNumber )
```

This function returns the duty cycle of the channel passed as parameter.

The function Pwm_GetChannelState shall return the DutyCycle of the channel. In case the channel is idle, the returned value will be zero.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
----	----------------------	-------------------------------------

Returns

uint16 DutyCycle of the requested channel

6.3.6.11 Pwm_ForceOutputToZero()

```
void Pwm_ForceOutputToZero (
    Pwm_ChannelType ChannelNumber,
    boolean bForce )
```

This function enables-disables the forcing of the output of a given channel to logic 0.

The function `Pwm_GetChannelState` shall return the `DutyCycle` of the channel. In case the channel is idle, the returned value will be zero.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>bForce</i>	Boolean value to state if the output of the channel will be forced to zero logic or not

Returns

void

6.3.6.12 Pwm_SetClockMode()

```
void Pwm_SetClockMode (
    Pwm_PrescalerType Prescaler )
```

Implementation specific function to change the peripheral clock frequency.

This function is useful to set the prescalers that divide the PWM channels clock frequency.

Parameters

in	<i>Prescaler</i>	Prescaler type
----	------------------	----------------

Returns

void

6.3.6.13 Pwm_SetChannelDeadTime()

```
void Pwm_SetChannelDeadTime (
    Pwm_ChannelType ChannelNumber,
    Pwm_PeriodType DeadTimeTicks )
```

This function is used to update the deadtime at runtime for Pwm channels.

Parameters

in	<i>ChannelNumber</i>	Pwm channel id
in	<i>DeadTimeTicks</i>	Dead Time value in ticks

Returns

void

6.3.6.14 Pwm_SetDutyCycle_NoUpdate()

```
void Pwm_SetDutyCycle_NoUpdate (
    Pwm_ChannelType ChannelNumber,
```

```
uint16 DutyCycle )
```

This function sets the values of dutycycle for the specified Pwm channel but without updating the PWM output.

The function `Pwm_SetDutyCycle_NoUpdate` shall set the duty cycle of the PWM channel to the corresponding hardware buffers without updating the wave form on the output pin. This feature will allow a pre-buffering of new PWM duty cycle values for several channel, which can all be updated synchronously by calling `Pwm_SyncUpdate`.

The function `Pwm_SetDutyCycle_NoUpdate` shall set the PWM output state according to the configured polarity parameter, when the duty cycle = 0% or 100%. The function `Pwm_SetDutyCycle_NoUpdate` shall modulate the PWM output signal according to parameters period, duty cycle and configured polarity, when the duty cycle > 0 % and < 100%.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PARAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value. As an implementation guide, the following source code example is given: `AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;`

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.3.6.15 Pwm_SetPeriodAndDuty_NoUpdate()

```
void Pwm_SetPeriodAndDuty_NoUpdate (
    Pwm_ChannelType ChannelNumber,
    Pwm_PeriodType Period,
    uint16 DutyCycle )
```

This function sets the values of the period and the dutycycle for the specified Pwm channel into the hardware buffers but without updating the PWM output..

The function `Pwm_SetPeriodAndDuty_NoUpdate` shall set the period and duty cycle of the PWM channel to

the corresponding hardware buffers without updating the wave form on the output pin. This feature will allow a pre-buffering of new PWM duty cycle values for several channel, which can all be updated synchronously by calling `Pwm_SyncUpdate`.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error `PWM_E_PERIOD_UNCHANGEABLE` if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter `channelNumber` and raise development error `PWM_E_PARAM_CHANNEL` if the parameter `channelNumber` is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function `Pwm_GetOutputState`.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value. As an implementation guide, the following source code example is given: `AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;`

If the `PwmDevErrorDetect` switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see `PWM_SWS`).

If development error detection for the Pwm module is enabled, if any function (except `Pwm_Init`) is called before `Pwm_Init` has been called, the called function shall raise development error `PWM_E_UNINIT`.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>Period</i>	Pwm signal period value
in	<i>DutyCycle</i>	Pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Returns

void

6.3.6.16 `Pwm_SyncUpdate()`

```
void Pwm_SyncUpdate (
    uint8 ModuleId )
```

Implementation specific function to updates duty synchronization.

This function is used to update duty synchronization for channels in given module, this should be called after calling `Pwm_SetPeriodAndDuty_NoUpdate()` or `Pwm_SetDutyCycle_NoUpdate()` API.

Module Documentation

Parameters

in	<i>Module↔ Id</i>	pwm module id(instance ID) Ex : PWM_EMIOs_INSTANCE_0 PWM_EMIOs_INSTANCE_1 ... PWM_FTM_INSTANCE_0
----	-----------------------	---

Returns

void

6.3.6.17 Pwm_SetDutyPhaseShift()

```
void Pwm_SetDutyPhaseShift (
    Pwm_ChannelType ChannelNumber,
    uint16 DutyCycle,
    Pwm_DutyType PhaseShift,
    boolean SyncUpdate )
```

This function set phase shift and duty cycle value (as immediate or synchronized base on API parameter SyncUpdate)

Pwm_SetDutyPhaseShift allows to set both phase shift and duty cycle value, The phase shift is the offset of the leading edge of the signal in respect to period starting point.

Parameters

in	<i>ChannelNumber</i>	Pwm Channel Id in the configuration
in	<i>DutyCycle</i>	Pwm duty cycle value 0x0000 for 0% ... 0x8000 for 100%
in	<i>PhaseShift</i>	Phase shift value (in ticks)
in	<i>SyncUpdate</i>	Update duty and phases shift value synchronization for channels in given module or not TRUE Set the phase shift and duty cycle value base on the synchronization when calling Pwm_SyncUpdate. FALSE Set phase shift and duty cycle value immediately

Returns

void

6.3.6.18 Pwm_EnableTrigger()

```
void Pwm_EnableTrigger (
    uint8 TriggerHostId,
    uint16 TriggerMask )
```

This function enable trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

in	<i>Trigger↔ HostId</i>	Pwm hardware module id
in	<i>TriggerMask</i>	Bit mask will be set to enable trigger with corresponding sources.

Returns

void

6.3.6.19 Pwm_DisableTrigger()

```
void Pwm_DisableTrigger (
    uint8 TriggerHostId,
    uint16 TriggerMask )
```

This function disable trigger generation for specific source.

Corresponding bits with trigger source as bellow: Bit 0 Channel 2 Trigger Enable Bit 1 Channel 3 Trigger Enable Bit 2 Channel 4 Trigger Enable Bit 3 Channel 5 Trigger Enable Bit 4 Channel 0 Trigger Enable Bit 5 Channel 1 Trigger Enable Bit 6 Initialization Trigger Enable

Parameters

in	<i>TriggerHostId</i>	Pwm hardware module id
in	<i>TriggerMask</i>	Bit mask will be cleared to disable trigger with corresponding sources.

Returns

void

6.3.6.20 Pwm_ResetCounterEnable()

```
void Pwm_ResetCounterEnable (
    uint8 ModuleId )
```

This function shall enable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.3.6.21 Pwm_ResetCounterDisable()

```
void Pwm_ResetCounterDisable (
    uint8 ModuleId )
```

This function shall disable the PWM timer HW counter reset by [Pwm_SyncUpdate\(\)](#) function.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Module Documentation

Returns

void

6.3.6.22 Pwm_FastUpdateDutyCycle()

```
void Pwm_FastUpdateDutyCycle (
    uint8 ModuleId,
    uint8 NumberOfChannels,
    const uint8 * Channels,
    const uint16 * DutyCycle,
    boolean SyncUpdate )
```

This function will update the duty cycle of PWM output for multiple channels with minimum overhead.

Parameters

in	<i>ModuleId</i>	The FTM peripheral instance number.
in	<i>NumberOfChannels</i>	The number of channels which should be updated.
in	<i>Channels</i>	The list of channels which should be updated.
in	<i>DutyCycle</i>	The list of duty cycles for selected channels.
in	<i>SyncUpdate</i>	Update duty cycle immediately or wait for a trigger event TRUE Update immediately FALSE Wait for a trigger event

Returns

void

6.3.6.23 Pwm_MaskOutputs()

```
void Pwm_MaskOutputs (
    uint8 ModuleId,
    uint16 ChannelsMask,
    boolean UpdateType )
```

This function force channels output to their inactive state (as immediate or synchronized base on API parameter UpdateType).

Corresponding bits with channel will be masked: Bit 0 Channel 0 Output Mask Bit 1 Channel 1 Output Mask Bit 2 Channel 2 Output Mask Bit 3 Channel 3 Output Mask Bit 4 Channel 4 Output Mask Bit 5 Channel 5 Output Mask

Parameters

in	<i>ModuleId</i>	Pwm Hardware module id
in	<i>ChannelsMask</i>	Bitmask's used to select corresponding channels.
in	<i>UpdateType</i>	The update is immediate or wait for a trigger event. TRUE Update immediately FALSE Wait for a trigger event (based on the synchronization when calling Pwm_SyncUpdate)

Returns

void

6.3.6.24 Pwm_UnMaskOutputs()

```
void Pwm_UnMaskOutputs (
    uint8 ModuleId,
    uint16 ChannelsMask,
    boolean UpdateType )
```

This function puts channels output to normal operation state (as immediate or synchronized base on API parameter UpdateType).

Corresponding bits with channel will be masked: Bit 0 Channel 0 Output Mask Bit 1 Channel 1 Output Mask Bit 2 Channel 2 Output Mask Bit 3 Channel 3 Output Mask Bit 4 Channel 4 Output Mask Bit 5 Channel 5 Output Mask

Parameters

in	<i>ModuleId</i>	Pwm Hardware module id
in	<i>ChannelsMask</i>	Bitmask's used to select corresponding channels.
in	<i>UpdateType</i>	The update is immediate or wait for a trigger event. TRUE Update immediately FALSE Wait for a trigger event (based on the synchronization when calling Pwm_SyncUpdate)

Returns

void

6.3.6.25 Pwm_EnableReloadNotification()

```
void Pwm_EnableReloadNotification (
    uint8 ModuleId )
```

This function enables the user reload notifications.

The function Pwm_EnableReloadNotification shall enable the PWM reload notification If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ModuleId and raise development error PWM_E_PARAM_↵ INSTANCE if the parameter ModuleId is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>Module↵ Id</i>	Pwm hardware module id
----	-------------------	------------------------

Returns

void

6.3.6.26 Pwm_DisableReloadNotification()

```
void Pwm_DisableReloadNotification (
    uint8 ModuleId )
```

This function disables the user reload notifications.

The function Pwm_DisableReloadNotification shall disable the PWM reload notification If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ModuleId and raise development error PWM_E_PARAM_↵ INSTANCE if the parameter ModuleId is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>Module↵ Id</i>	Pwm hardware module id
----	-----------------------	------------------------

Returns

void

6.3.6.27 Pwm_EnableFaultNotification()

```
void Pwm_EnableFaultNotification (
    uint8 ModuleId )
```

This function enables the user fault notifications.

The function Pwm_EnableFaultNotification shall enable the PWM fault notification If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ModuleId and raise development error PWM_E_PARAM_↵ INSTANCE if the parameter ModuleId is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.3.6.28 Pwm_DisableFaultNotification()

```
void Pwm_DisableFaultNotification (
    uint8 ModuleId )
```

This function disables the user fault notifications.

The function Pwm_DisableFaultNotification shall disable the PWM fault notification If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ModuleId and raise development error PWM_E_PARAM_INVALID_INSTANCE if the parameter ModuleId is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the PwmDevErrorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Parameters

in	<i>ModuleId</i>	Pwm hardware module id
----	-----------------	------------------------

Returns

void

6.3.6.29 Pwm_SetPowerState()

```
Std_ReturnType Pwm_SetPowerState (
    Pwm_PowerStateRequestResultType * Result )
```

Enters the already prepared power state.

Module Documentation

This API configures the Pwm module so that it enters the already prepared power state, chosen between a predefined set of configured ones.

Parameters

out	<i>Result</i>	Pointer to a variable to store the result of this function
-----	---------------	--

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Power Mode changed.
<i>E_NOT_OK</i>	Request rejected.

6.3.6.30 Pwm_GetCurrentPowerState()

```
Std_ReturnType Pwm_GetCurrentPowerState (  
    Pwm_PowerStateType * CurrentPowerState,  
    Pwm_PowerStateRequestResultType * Result )
```

Get the current power state of the Pwm HW unit.

This API returns the current power state of the Pwm HW unit.

Parameters

out	<i>CurrentPowerState</i>	The current power mode of the Pwm HW Unit is returned in this parameter
out	<i>Result</i>	Pointer to a variable to store the result of this function

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Mode could be read.
<i>E_NOT_OK</i>	Service is rejected.

6.3.6.31 Pwm_GetTargetPowerState()

```
Std_ReturnType Pwm_GetTargetPowerState (  
    Pwm_PowerStateType * TargetPowerState,  
    Pwm_PowerStateRequestResultType * Result )
```

Get the target power state of the Pwm HW unit.

This API returns the target power state of the Pwm HW unit.

Parameters

out	<i>TargetPowerState</i>	The Target power mode of the Pwm HW Unit is returned in this parameter.
out	<i>Result</i>	Pointer to a variable to store the result of this function.

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Mode could be read.
<i>E_NOT_OK</i>	Service is rejected.

6.3.6.32 Pwm_PreparePowerState()

```
Std_ReturnType Pwm_PreparePowerState (
    Pwm_PowerStateType PowerState,
    Pwm_PowerStateRequestResultType * Result )
```

Starts the needed process to allow the Pwm HW module to enter the requested power state.

This API starts the needed process to allow the Pwm HW module to enter the requested power state.

Parameters

in	<i>PowerState</i>	The target power state intended to be attained.
out	<i>Result</i>	Pointer to a variable to store the result of this function.

Returns

Std_ReturnType Standard return type.

Return values

<i>E_OK</i>	Mode could be read.
<i>E_NOT_OK</i>	Service is rejected.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.